SERVICIO NACIONAL DE APRENDIZAJE

SENA · Centro de Formación de Talento Humano en Comercio y Servicios

Proyecto Formativo — Sistema Punto de Venta & Facturación

MANUAL ADMINISTRATIVO

Informe técnico-administrativo del proyecto de software

SISTEMA PUNTO DE VENTA Y FACTURACIÓN

D'Piero Restaurante & Parrilla — Parrilla · Bar & Cocina a la Brasa

DATO DE LA ENTREGA	INFORMACIÓN
Empresa (cliente)	D'Piero Restaurante & Parrilla — Bogotá, Colombia
Proyecto	Sistema POS D'Piero — Punto de Venta, Comandas y Facturación
Aprendiz autor	Santiago Andrés Vélez Naranjo (AndresVlz2020)
Equipo de trabajo	Santiago Andrés Vélez Naranjo · Colaborador: Henao420
Programa de formación	Tecnólogo en Análisis y Desarrollo de Software — Proyecto Formativo
Ficha del proyecto formativo	3172293
Repositorio	github.com/AndresVlz2020/pos_project (ramas: main, dev)
Fecha de elaboración	Noviembre de 2026
Versión del documento	1.1

Versión 1.1 · Noviembre 2026 · Documento Word (.docx)

Control de versiones

Historial de cambios del documento. Cada versión incrementa el número menor para borradores internos y el número mayor para entregas formales.

Versión	Fecha	Descripción del cambio	Responsable
0.1	2026-09-25	Borrador inicial a partir de la I revisión de avance del proyecto (informe administrativo y manuales).	Santiago Andrés Vélez Naranjo
0.5	2026-10-15	Reestructuración según rúbrica de la actividad de aprendizaje; levantamiento de información del código fuente (backend, frontend, base de datos).	Santiago Andrés Vélez Naranjo
0.9	2026-11-04	Segunda revisión de avance; verificación del funcionamiento real del sistema (backend, frontend, PostgreSQL) y captura de evidencias gráficas.	Santiago Andrés Vélez Naranjo
1.0	2026-11-08	Versión final: contenido integral verificado contra el código, las migraciones SQL y la operación del sistema; incorporación de diagramas y métricas de calidad.	Santiago Andrés Vélez Naranjo
1.1	2026-11-08	Reparación de seguridad tras verificación CRUD y de variables: todas las mutaciones de la API protegidas con JWT + permisos RBAC, BUG-01 corregido, CORS restringido a lista blanca, JWT_SECRET rotado y validado al arranque, actualización parcial de usuarios reparada (ya no devuelve 500), variables de entorno consolidadas con .env.example y servicios frontend centralizados en VITE_API_URL. Verificación integral repetida: 31/31 aserciones conformes.	Santiago Andrés Vélez Naranjo

Tabla 1 — Control de versiones del manual administrativo

Información general del documento

Campo	Detalle
Nombre del sistema	D'Piero POS — Sistema Punto de Venta & Facturación
Empresa cliente	D'Piero Restaurante & Parrilla (Parrilla, Bar & Cocina a la Brasa)
Tipo de solución	Aplicación web (SPA) — Escritorio / Web (navegador)
Arquitectura	Cliente–servidor en 3 capas: React SPA → API REST Express → PostgreSQL
Usuarios objetivo	Administrador, Cajero, Mesero y Cocina (roles RBAC)
Clasificación	Uso interno del proyecto formativo — Distribución restringida al equipo
Norma de elaboración	ISO/IEC/IEEE 26514:2022 (documentación de usuario) e ISO/IEC 25010
Formato de entrega	Microsoft Word (.docx) — Norma APA 7.ª edición (referencias)

Tabla 2 — Ficha técnica de identificación del documento

Aviso de confidencialidad y uso responsable

El presente documento y la información que contiene —incluyendo direcciones internas, credenciales de ejemplo, esquema de base de datos y detalles de seguridad— son de uso exclusivo del equipo del proyecto formativo y de D'Piero Restaurante & Parrilla. Queda prohibida su reproducción total o parcial sin autorización escrita. Las credenciales de demostración citadas en este manual deben ser reemplazadas antes de cualquier puesta en producción real.

Este documento se elaboró siguiendo el principio de veracidad: cada afirmación sobre el sistema fue verificada directamente sobre el código fuente (commit f58187b, rama dev), las 27 migraciones de base de datos aplicadas sobre la base `pos_project`, y la ejecución real del sistema en el entorno de desarrollo.

Tabla de Contenido

1. Introducción.....10
2. Información General del Proyecto.....12
2.1 Nombre del proyecto.....12
2.2 Cliente.....12
2.3 Descripción del problema.....13
2.4 Objetivo General.....14
2.5 Objetivos Específicos.....14
2.6 Alcance.....15
2.7 Limitaciones.....17
2.8 Riesgos.....17
2.9 Análisis del entorno del negocio.....18
3. Fase de Requerimientos.....22
3.1 Técnicas de recolección.....22
3.2 Requerimientos Funcionales.....23
3.3 Requerimientos No Funcionales.....24
3.4 Reglas del negocio.....25
3.5 Restricciones.....26
4. Fase de Análisis.....26
4.1 Casos de Uso.....26
4.2 Historias de usuario.....29
4.3 Diagrama de Clases.....30
4.4 Diagrama de Actividades.....32
4.5 Especificación detallada de los casos de uso restantes.....34
4.6 Escenarios Gherkin de aceptación (Must-have).....38
4.7 Tarjetas CRC (Clase — Responsabilidad — Colaboración).....39
4.8 Diagrama de secuencia de autenticación.....40
5. Fase de Diseño.....40
5.1 Diagrama de Componentes.....40
5.2 Diagrama de Distribución (Despliegue).....42
5.3 Aplicaciones Distribuidas.....43
5.4 Arquitectura del software y su justificación.....43
5.5 Diseño de la Base de Datos (MR).....45
5.6 Prototipos.....46
5.7 Normalización y formas normales (análisis del MR).....47
5.8 Modos de fallo y resiliencia por componente.....48
5.9 Inventario descriptivo de las 31 pantallas del sistema.....48
6. Fase de Construcción.....52

6.1 Metodología utilizada....52
6.2 Cronograma....53
6.3 Roles del equipo de trabajo....53
6.4 Herramientas utilizadas....54
6.5 Estructura del proyecto....54
6.6 Módulos implementados....55
6.7 Control de versiones....56
6.8 Estándares de codificación....56
6.9 Organización del código....57
6.10 Historial completo de versiones (54 commits, 2026-05-11 → 2026-11-05)....58
6.11 Propuesta de integración continua y entrega continua (CI/CD)....59
6.12 Estándares con ejemplos de código real....60
6.13 Detalle de los seis sprints ejecutados....61
7. Fase de Calidad....62
7.1 Métricas de código (LOC, complejidad ciclomática y densidad de comentarios)....62
7.2 Plan de Pruebas....65
7.3 Casos de Pruebas....66
7.4 Pruebas implementadas (manuales y automatizadas)....68
7.5 Defectos encontrados y corregidos durante la construcción....69
7.6 Matriz de cobertura RF ↔ RNF ↔ CP ↔ ejecución (verificada 2026-11-08)....70
7.7 Criterios de aceptación verificados (checklist por módulo)....70
8. Fase de Implementación....71
8.1 Despliegue....71
8.2 Respaldo y migración de datos....73
8.3 Mantenimiento....74
8.4 Soporte....74
8.5 Runbook de producción (Linux, paso a paso)....75
8.6 Procedimiento de rollback....77
8.7 Monitoreo, alertas y bitácora....77
9. Gestión del Proyecto....78
9.1 Dificultades encontradas (situación y solución)....78
9.2 Lecciones aprendidas....79
9.3 Conclusiones....79
9.4 Bibliografía / Webgrafía....80
9.5 Antecedentes y contexto académico del proyecto....81
9.6 Plan de gestión del cambio....82
Anexos....82
Anexo A — Diccionario de datos resumido (extracto)....82
Anexo B — Matriz de trazabilidad RF ↔ CU ↔ CP....83

Anexo C — Verificación de instalación (checklist)..... 84
Anexo D — Guion de sustentación (15 minutos)..... 84
Anexo E — Evidencias fotográficas de ejecución (2026-11-08)..... 85

Tabla de Ilustraciones

Figura 1 — Diagrama de casos de uso — actores y funcionalidades del sistema....7
Figura 2 — Diagrama de clases del dominio D'Piero POS....31
Figura 3 — Diagrama de actividades del proceso comanda-cobro....33
Figura 4 — Secuencia login completo: cliente → Express → servicio → bcrypt/JWT → PostgreSQL....7
Figura 5 — Diagrama de componentes — Frontend, Backend, PostgreSQL y Gmail SMTP....41
Figura 6 — Diagrama de distribución — topología de despliegue por dispositivo....42
Figura 7 — Modelo entidad-relación — tres subgrafos funcionales....45
Figura 8 — Cronograma de fases — mayo a noviembre de 2026....53
Figura 9 — LOC por tecnología — JavaScript/React, Node, SQL y CSS....63
Figura 10 — LOC por módulo / capa funcional....63
Figura 11 — Archivos con mayor complejidad ciclomática (estimada)....64
Figura 12 — pestaña Acceso Rápido con usuarios operativos disponibles (Carlos Mesero, María Cajera, etc.)....7
Figura 13 — pestaña Administración — correo corporativo y clave del administrador....86
Figura 14 — teclado PIN de 4 dígitos tras seleccionar a Carlos Mesero....87
Figura 15 — recuperación de contraseña con envío de código al correo....88
Figura 16 — verificación de código de 6 dígitos y nueva clave....89
Figura 17 — POS — Catálogo & Comandas con 12 platillos reales y precios COP....90
Figura 18 — pestaña Personal en Turno con roles y horarios....91
Figura 19 — pestaña Proveedores del POS con datos de contacto reales....92
Figura 20 — pestaña Clientes & Facturación (datos de demostración)....93
Figura 21 — creación de comanda con mesa y disponibilidad de 6 libres....94
Figura 22 — listado de órdenes ORD-003..007 con estados y totales....95
Figura 23 — modal de detalle de comanda con ítems y total....96
Figura 24 — edición de comanda (mesa, mesero, estado, ítems)....97
Figura 25 — listado de productos con stock real e INV válidos....98
Figura 26 — formulario de creación de platillo....99
Figura 27 — ficha de solo lectura de platillo (Punta de Anca)....100
Figura 28 — edición de producto (estado Habilitado)....101
Figura 29 — listado de inventario con alerta de stock mínimo (INV-004)....102
Figura 30 — alta de insumo con lote, vencimiento y proveedor....103
Figura 31 — listado de proveedores con 26 registros....104
Figura 32 — ficha de proveedor (Avícola y Carnes del Valle S.A.)....105
Figura 33 — alta de proveedor con NIT único....106
Figura 34 — 4.5: evidencia del fallo BUG-01 en /editsupplier (FileInput userImage=null)....107
Figura 35 — listado de usuarios con acciones (editar/ver/estado)....108
Figura 36 — alta de usuario operativo con PIN autogenerado....109

Figura 37 — edición de usuario (Carlos Mesero).....110
Figura 38 — ficha de usuario (información del usuario).....111
Figura 39 — gestión de permisos RBAC (grupos y usuario individual).....112
Figura 40 — matriz de permisos por módulo en /dashboard/access.....113
Figura 41 — modal 'Generar reporte de usuarios' con formato PDF por defecto.....114

Lista de Tablas

Tabla 1 — Control de versiones del manual administrativo.....2
Tabla 2 — Ficha técnica de identificación del documento.....2
Tabla 3 — Identificación general del proyecto.....12
Tabla 4 — Objetivos específicos del proyecto y su ubicación en las fases.....15
Tabla 5 — Limitaciones del proyecto con su impacto y mitigación.....17
Tabla 6 — Registro de riesgos del proyecto con evaluación cuantitativa.....18
Tabla 7 — Matriz DOFA del cliente frente al proyecto POS D'Piero.....19
Tabla 8 — Causas raíz estructuradas con Ishikawa aplicado al proceso de comanda.....19
Tabla 9 — Mapa de stakeholders con estrategia de involucramiento.....20
Tabla 10 — Plan de comunicación y gestión de expectativas.....20
Tabla 11 — Matriz RACI (R=Responsable, A=Aprobador, C=Consultado, I=Informado).....21
Tabla 12 — Estructura de desglose del trabajo en tres niveles.....21
Tabla 13 — Estimación de costo total del proyecto (COP).....22
Tabla 14 — Técnicas de recolección de requerimientos aplicadas.....23
Tabla 15 — Requerimientos funcionales RF-01 a RF-24 con prioridad y fuente.....24
Tabla 16 — Requerimientos no funcionales clasificados por ISO/IEC 25010.....25
Tabla 17 — Reglas de negocio verificadas en el código fuente.....26
Tabla 18 — Restricciones técnicas, de seguridad y de negocio.....26
Tabla 19 — Síntesis de los diez casos de uso del sistema.....28
Tabla 20 — Especificación completa del caso de uso CU-08.....29
Tabla 21 — Especificación del caso de uso CU-01.....29
Tabla 22 — Historias de usuario priorizadas (MoSCoW) y estimadas en puntos.....30
Tabla 23 — Tarjetas CRC de las 12 entidades centrales.....39
Tabla 24 — Contratos de los componentes y sus interfaces.....41
Tabla 25 — Decisiones de arquitectura con su justificación.....44
Tabla 26 — Inventario de tablas, datos semilla y propósito.....46
Tabla 27 — Rutas y pantallas del sistema (frontend/src/app/router.jsx).....47
Tabla 28 — Análisis de formas normales tabla por tabla.....48
Tabla 29 — Modos de fallo, detección y resiliencia por componente.....48
Tabla 30 — Hitos del cronograma con su evidencia en el repositorio.....53

Tabla 31 — Roles y responsabilidades del equipo....54
Tabla 32 — Hardware de desarrollo y operación....54
Tabla 33 — Software utilizado en el proyecto....54
Tabla 34 — Módulos implementados con LOC medido y estado....55
Tabla 35 — Historial integral de commits del repositorio (54)....59
Tabla 36 — Los seis sprints del proyecto con entregas y riesgos....61
Tabla 37 — Métricas de calidad del código fuente (2026-11-08)....62
Tabla 38 — Distribución de LOC por lenguaje....64
Tabla 39 — Niveles de prueba, técnicas y cobertura planificada....65
Tabla 40 — Casos de prueba CP-01 a CP-30 con resultado esperado....67
Tabla 41 — Inventario real de pruebas automatizadas....68
Tabla 42 — Registro de defectos (bug log) con severidad y estado....69
Tabla 43 — Matriz de trazabilidad de verificación — qué se probó y cómo....70
Tabla 44 — Checklist de criterios de aceptación por módulo (verificado 2026-11-08)....71
Tabla 45 — Comparación de ambientes de despliegue....72
Tabla 46 — Variables de entorno del backend (sin revelar valores reales)....72
Tabla 47 — Objetivos de recuperación ante desastre....74
Tabla 48 — Tipos de mantenimiento con actividades y responsables....74
Tabla 49 — Calendario de actividades de mantenimiento....74
Tabla 50 — Niveles de soporte L1/L2/L3....75
Tabla 51 — Indicadores de servicio de soporte....75
Tabla 52 — Señales monitoreadas con umbrales y canales....77
Tabla 53 — Registro de dificultades encontradas y su solución....79
Tabla 54 — Clasificación de cambios con autorización y documentación requerida....82
Tabla 55 — Extracto del diccionario de datos....83
Tabla 56 — Matriz de trazabilidad requerimiento–caso de uso–caso de prueba....84

1. Introducción

El presente Manual Administrativo consolida, en un único documento, toda la información de gestión del proyecto **Sistema Punto de Venta & Facturación D'Piero POS**, desarrollado como proyecto formativo (Ficha 3172293) para el restaurante **D'Piero Restaurante & Parrilla**, establecimiento gastronómico a la parrilla y bar ubicado en Bogotá, Colombia. El documento recorre, fase por fase, el ciclo de vida del software — desde la elicitación de requerimientos hasta la implementación, el mantenimiento y el soporte —, de modo que cualquier integrante del equipo o patrocinador institucional pueda comprender qué se construyó, por qué se construyó de esa manera, con qué calidad y en qué condiciones debe operar y evolucionar.

El sistema cubre la operación completa de un restaurante de servicio a la mesa: punto de venta (POS), gestión de comandas por mesa, catálogo de productos y platillos, control de inventario de insumos con trazabilidad por lote y fecha de vencimiento, vinculación de proveedores, administración de usuarios y permisos con un modelo RBAC (Role-Based Access Control) de grano fino, y generación de reportes en PDF y Excel. El resultado es una aplicación web moderna construida con **React 19 + Vite 8** en el frontend y **Node.js 24 + Express 5 + PostgreSQL** en el backend, con control de versiones mediante Git (54 commits entre mayo y noviembre de 2026 en la rama de desarrollo).

Este manual se elaboró a partir de la revisión exhaustiva y verificación en sitio de: (i) el código fuente completo del frontend y el backend; (ii) las 27 migraciones SQL versionadas que construyen y pueblan la base de datos `pos_project`; (iii) la ejecución real del sistema en el entorno de desarrollo local (backend en el puerto 4000 y frontend en el puerto 5173), con autenticación funcional, consumo de datos desde PostgreSQL y generación de las evidencias gráficas que ilustran cada sección; y (iv) el historial de Git, las métricas de calidad calculadas sobre el código (18.407 líneas totales, 201 archivos, complejidad ciclomática media de 10,86) y la organización del repositorio.

1.1 Propósito del documento

- **Dejar trazabilidad completa.** Cada decisión de negocio, diseño y calidad queda documentada con su justificación, para que la organización pueda auditar, mantener y evolucionar el sistema sin depender del conocimiento tácito del equipo.

- **Servir de insumo de sustentación.** El documento cumple con la estructura exigida por la actividad de aprendizaje (Ítems del Informe Administrativo) y con las normas de documentación aplicables al proyecto formativo.
- **Unificar criterios.** Equipo, cliente e instructores comparten una única fuente de verdad sobre alcance, arquitectura, estándares, riesgos, plan de pruebas y condiciones de implementación.
- **Facilitar el mantenimiento.** Las secciones de implementación, mantenimiento y soporte definen procedimientos accionables: cómo desplegar, cómo respaldar, cómo diagnosticar fallas y a quién escalarlas.

1.2 Alcance del manual

El manual describe el sistema tal como existe en el commit `f58187b` (rama `dev`, 2026-11-05), incluyendo las modificaciones de trabajo no confirmadas aún en el repositorio (módulos de órdenes, inventario y productos conectados a PostgreSQL, sistema RBAC con grupos y permisos, rediseño visual claro/transición desde tema oscuro). Los alcances documentados son de desarrollo y puesta en marcha local; la infraestructura productiva real (servidores, dominios, certificados, alta disponibilidad) se especifica como recomendación, no como estado actual.

1.3 Cómo leer este manual

Las secciones 2 a 8 siguen las fases del ciclo de vida adoptado (Requerimientos, Análisis, Diseño, Construcción, Calidad, Implementación). La sección 9 recopila la gestión del proyecto: dificultades, lecciones, conclusiones y referencias. Los anexos A–D contienen material de apoyo: diccionario de datos resumido, trazabilidad de requerimientos, evidencias y guion de sustentación. Las figuras y tablas están numeradas y referenciadas en texto; las tablas de contenido, de ilustraciones y de tablas se actualizan automáticamente al abrir el documento en Microsoft Word (clic derecho → Actualizar campos, o al imprimir).

1.4 Normas y referentes aplicados

- **ISO/IEC/IEEE 26514:2022.** Estructura del contenido del documento y clasificación por audiencias.
- **ISO/IEC 25010.** Modelo de calidad usado para clasificar los requerimientos no funcionales (mantenibilidad, seguridad, usabilidad, portabilidad).

- **ISO 31000.** Tratamiento de riesgos del proyecto (sección 2.8) y de riesgos del software (sección 7).
- **APA 7.ª edición.** Formato de referencias bibliográficas y estilo de redacción.
- **Conventional Commits.** Convención de mensajes de commit aplicada en el historial del repositorio.

2. Información General del Proyecto

2.1 Nombre del proyecto

Atributo	Valor
Nombre comercial del sistema	D'Piero POS — Sistema Punto de Venta & Facturación
Proyecto formativo	Sistema Punto de Venta y Facturación para Restaurante D'Piero
Nombre del repositorio	pos_project (github.com/AndresVlz2020/pos_project)
Nombre interno de la base de datos	pos_project (PostgreSQL)
Versión del sistema documentada	1.0.0 (backend/package.json y frontend/package.json)
Cliente	D'Piero Restaurante & Parrilla
Sector económico	Servicios de comidas y bebidas — CIIU 5611 (restaurantes con servicio completo)
Modalidad de entrega	Proyecto formativo — evidencia de aprendizaje (SENA, Ficha 3172293)

Tabla 3 — Identificación general del proyecto

2.2 Cliente

D'Piero Restaurante & Parrilla es un establecimiento gastronómico especializado en parrilla a la brasa, bar y cocina criolla-contemporánea. Su propuesta gastronómica incluye cortes madurados a la parrilla (punta de anca Angus, costillas BBQ ahumadas), hamburguesería artesanal (pan brioche, carnes Angus 200 g), entradas para compartir (chicharrones, empanadas), cócteles de autor (frutos rojos), cerveza artesanal y postres de la casa. El restaurante opera con servicio a la mesa (10 mesas, capacidad entre 2 y 10 comensales), punto de venta en caja y cocina a la vista. Su modelo de negocio combina consumo en salón, rapidez en comanda y ticket promedio alto.

El restaurante presentaba un cuello de botella operativo clásico: la toma de pedidos y la gestión de comandas se hacía de forma manual (papel/comandero verbal), no existía control sistematizado del inventario de insumos con lotes y vencimientos, los permisos del personal no estaban diferenciados por rol, y los reportes de venta se consolidaban a mano al cierre del turno. El proyecto nace para resolver exactamente esos puntos con un sistema integrado POS + administración.

2.3 Descripción del problema

2.3.1 Situación actual (antes del sistema)

- **Comandas en papel.** Las comandas manuscritas se traspapelaban entre salón y cocina; errores en platillos y adiciones no notificadas generaban reprocesos y pérdidas económicas.
- **Sin control de stock real.** El inventario de carnes, lácteos, frutas y licores se llevaba mentalmente o en hojas sueltas; los insumos vencían sin que el sistema alertara (riesgo de merma y sanitario).
- **Acceso irrestricto.** Cualquier empleado podía acceder a las funciones de administración (precios, usuarios, reportes) sin diferenciación de responsabilidades ni registro de acciones.
- **Facturación y cierre de turno lentos.** La consolidación de ventas al cierre requería sumar comandas a mano; los reportes de productos más vendidos o insumo crítico no existían.
- **Sin trazabilidad de comandas.** No había forma de auditar qué mesero atendió qué mesa, ni de anular una comanda con motivo registrado.
- **Reportes manuales.** Los informes para el administrador (personal, ventas, insumos) se elaboraban manualmente; la generación de un Excel o PDF tomaba horas.

2.3.2 Consecuencias verificables

- Pérdidas estimadas por merma de insumos vencidos y errores de comanda no detectados a tiempo (estimación operativa del cliente: entre 3 % y 5 % del costo de materia prima mensual).
- Fugas de control: cambio de precios sin autorización, descuentos no autorizados, comandas anuladas sin registro del motivo.
- Tiempo de cierre de caja prolongado (30–45 minutos) y datos de venta poco confiables.

- Imposibilidad de aplicar el principio de mínimo privilegio: meseros y cajero accedían a pantallas de administración.

2.3.3 Formulación del problema

> ¿Cómo disponer de una única plataforma web, confiable y trazable, que permita a D'Piero Restaurante & Parrilla tomar comandas, controlar su inventario de insumos y productos, administrar usuarios con permisos diferenciados por rol y generar reportes de gestión, eliminando los errores y la lentitud del manejo manual, y garantizando la integridad de los datos del negocio?

2.3.4 Justificación de la solución

- **Operativa y económica.** Automatizar comandas y stock reduce errores, merma y tiempos de atención; el retorno se materializa en el primer mes de operación.
- **De control.** RBAC (Administrador, Cajero, Mesero, Cocina) con 21 permisos granulares aplicados en frontend (rutas y visibilidad) y backend (middleware de permisos).
- **De información.** Reportes PDF/Excel con filtros por campo y alcance, descargables por el administrador o por quien tenga el permiso report_user.
- **Tecnológica.** Stack probado y mantenible (React, Express, PostgreSQL) con costo de licencia nulo; toda la solución corre en un computador del establecimiento.

2.4 Objetivo General

> Implementar un Sistema Punto de Venta y Facturación (D'Piero POS) que digitalice la operación completa del restaurante D'Piero — toma de comandas, catálogo, inventario, proveedores, usuarios con permisos por rol y reportes — garantizando integridad, trazabilidad y disponibilidad de la información, con una experiencia de uso rápida y adecuada para el ritmo de servicio a la mesa.

2.5 Objetivos Específicos

#	Objetivo específico	Fase / módulo
OE-01	Construir la base de datos relacional normalizada (16 entidades) y versionarla con 27 migraciones idempotentes y datos semilla.	Base de datos / Construcción
OE-02	Implementar la API REST con arquitectura por features (routes → controller → service → repository) sobre Express 5, con acceso parametrizado a PostgreSQL.	Backend / Construcción

#	Objetivo específico	Fase / módulo
OE-03	Desarrollar el frontend SPA en React 19 con terminal POS, módulos administrativos y tema claro/oscuro.	Frontend / Construcción
OE-04	Implementar autenticación dual: acceso rápido por PIN (personal operativo) y acceso administrativo con correo + contraseña (bcrypt + JWT).	Seguridad / Construcción
OE-05	Implementar RBAC con 4 grupos predefinidos, 21 permisos granulares, asignación directa por usuario/grupo y sincronización automática.	Seguridad / Construcción
OE-06	Descontar el stock de productos en cada venta de comanda, dentro de una transacción BEGIN/COMMIT con rollback ante fallos.	Órdenes / Construcción
OE-07	Generar reportes ejecutivos en PDF y Excel con campos y alcance configurables.	Reportes / Construcción
OE-08	Documentar el sistema con el Manual Administrativo, el Manual Técnico y el Manual de Usuarios con base en el código real.	Documentación
OE-09	Verificar el funcionamiento integral del sistema (build, migraciones, API, UI) y registrar las evidencias en este manual.	Calidad

Tabla 4 — Objetivos específicos del proyecto y su ubicación en las fases

2.6 Alcance

2.6.1 Dentro del alcance

- **Módulo de autenticación y sesión:** inicio de sesión por PIN de terminal (4 dígitos, personal: mesero/cajero/cocina) y por correo + contraseña (administrador); recuperación de contraseña con código de 6 dígitos por correo (15 min de vigencia); cierre de sesión; cierre por inactividad a los 10 minutos.
- **Módulo de usuarios:** CRUD completo (crear, listar, consultar, editar), activación/desactivación inmediata, avatar, tipos de usuario (mesero, cajero, cocina, administrador), sincronización automática con grupos RBAC.
- **Módulo de productos (menú):** CRUD, categorías (platos fuertes, entradas, bebidas, postres, guarniciones), precio, stock, estado (Disponible/Inactivo), imagen referenciada, búsqueda y filtro, reportes.
- **Módulo de comandas / órdenes:** creación por mesa (ORD-###), ítems con notas para cocina, máquina de estados (Abierta → En preparación → Pagada → Cerrada / Anulada), disponibilidad de mesas en tiempo real, edición completa, impresión/visualización de factura.
- **Módulo de inventario:** CRUD de insumos con código INV-###, categorías (Carnes, Frutas y Verduras, Lácteos, Bebidas y Licores, Insumos Secos, Empaques, Aseo y Varios), stock

actual/mínimo/máximo, unidad de medida, ubicación, precio de compra, lote y fecha de vencimiento, alerta de stock mínimo bajo.

- **Módulo de proveedores:** CRUD, NIT único, datos de contacto, estado activo/inactivo, ficha de solo lectura, enlaces con inventario y productos.
- **Módulo RBAC (accesos):** gestión de permisos por grupo y por usuario, matriz de 21 permisos sobre 5 módulos (users, products, orders, inventory, suppliers), heredados y directos.
- **Módulo POS (home):** terminal de atención con catálogo, búsqueda, filtros por categoría, pestañas por rol, paneles de personal en turno, proveedores y clientes/facturación.
- **Reportes:** generación PDF (jsPDF + AutoTable) y Excel (SheetJS) con campos configurables y alcance (todos / solo activos) sobre el módulo de usuarios; estructura reutilizable para otros módulos.
- **API REST:** 9 grupos de endpoints con autenticación JWT opcional o obligatoria según ruta, validaciones y respuestas normalizadas.
- **Seguridad:** bcrypt (costo 10), JWT con expiración configurable (1 h por defecto), rate limiting lógico por bloqueo de 3 intentos (lado cliente), sanitización de alertas, CORS restringido a localhost, cabeceras y validaciones Joi/Zod.
- **Documentación:** este manual, el Manual Técnico y el Manual de Usuarios; README del frontend con RNF05.

2.6.2 Fuera del alcance (explícito)

- Pasarela de pago real (tarjetas, PSE, datáfono): el cobro se registra como estado de comanda, sin integrar entidades bancarias.
- Facturación electrónica DIAN (CUFE, transmisión): el documento 'factura' es interno y no tiene validez fiscal.
- Aplicación móvil nativa (Android/iOS): la solución es web responsive para uso en tablet/PC de caja.
- Alta disponibilidad y multi-sucursal: una única sede, un único servidor de base de datos.
- Sincronización con contabilidad o ERP externo.
- Gestión de nómina y reservaciones de mesas por canal online.

- Modo offline/desconectado real: el POS admite fallback de datos en cliente, pero la operación exige conectividad con el backend.

2.7 Limitaciones

#	Limitación	Impacto	Mitigación
L-01	El sistema fue construido y verificado en un único equipo de desarrollo (Windows, Node.js 24, PostgreSQL local); no existe aún servidor de producción.	Alto	Procedimiento de despliegue documentado (sección 8.1); variables por ambiente.
L-02	Sin suite de pruebas automatizadas; la verificación fue manual y por inspección.	Medio	Plan de pruebas completo (sección 7.2 –7.4); deuda técnica registrada.
L-03	Secretos débiles en el entorno local: JWT_SECRET con valor predecible (rotado a 64 caracteres aleatorios el 2026-11-08); SMTP_PASS es una contraseña de aplicación real, gestionada fuera del repositorio.	Alto	Rotación de JWT_SECRET (realizado) y alta periódica de contraseña de aplicación; .env jamás versionado (verificado con git ls-files).
L-04	El endpoint de login no tiene rate limiting en servidor; el bloqueo por 3 intentos es del frontend (se evade llamando API directamente).	Medio	Implementar límite por IP en el backend (pendiente registrado en lecciones aprendidas).
L-05	Imágenes de productos servidas desde el bundle estático (`/assets/images/*.jpg`); no hay CDN ni object storage.	Bajo	Migración futura a almacenamiento externo (S3-compatible) documentada en Manual Técnico.
L-06	Sin auditoría (quién hizo qué y cuándo) más allá de `created_at`/`updated_at`.	Medio	Tabla `audit_log` propuesta en Manual Técnico (sección 14).
L-07	El diseño de respaldo (backup) es un procedimiento definido, no implementado: no hay backups automáticos verificados.	Alto	Cron diario con `pg_dump` + retención 30 días (sección 8.2).
L-08	PIN de 4 dígitos compartido por defecto (1234) en usuarios semilla.	Medio	Forzar cambio de PIN en primer ingreso (instrucción de operación).
L-09	La pestaña 'Clientes & Facturación' usa datos estáticos de demostración (posMockData) no persistentes.	Bajo	Implementar tabla `customers` en una iteración futura (sección 9.2).
L-10	No hay i18n; todo el contenido y los mensajes están en español (Colombia).	Bajo	Aceptable para el mercado objetivo; documentado como decisión.

Tabla 5 — Limitaciones del proyecto con su impacto y mitigación

2.8 Riesgos

Los riesgos se valoraron con probabilidad (P: 1–5), impacto (I: 1–5) y exposición (E = P × I). E ≥ 12 es crítico; 8–10 alto; 4–6 medio; ≤ 3 bajo.

#	Riesgo	P	I	E	Categoría	Respuesta y contingencia
R-01	Pérdida de datos por fallo del disco del servidor de base de datos	2	5	10	Técnico	Mitigar: pg_dump diario + copia en disco externo/nube; probar restauración mensual.
R-02	Suplantación de identidad con credenciales débiles o filtradas	3	4	12	Seguridad	Mitigar: bcrypt 10, JWT 1 h, bloqueo 3 intentos, rotación de secretos, HTTPS en producción.
R-03	Fuga del archivo `.env` con JWT_SECRET y SMTP_PASS	2	5	10	Seguridad	Mitigar: `.env` fuera del control de versiones (.gitignore verificado); JWT_SECRET ya

#	Riesgo	P	I	E	Categoría	Respuesta y contingencia
						rotado a valor fuerte y validado al arranque por `validateEnv()` (2026-11-08).
R-04	Inyección SQL en parámetros no validados	2	4	8	Técnico	Mitigar: consultas parametrizadas obligatorias ($1..$n) en todos los repositorios.
R-05	Despliegue roto por dependencia con vulnerabilidad conocida (supply chain)	3	3	9	Técnico	Mitigar: `npm audit` en cada release, versiones fijadas en package-lock.json.
R-06	El equipo no documenta cambios y el sistema queda inmantenible	3	4	12	Gestión	Mitigar: Conventional Commits, PR review, este manual y el Manual Técnico.
R-07	Corrupción de comandas abiertas al cierre del turno (corte de energía)	2	3	6	Operativo	Mitigar: transacciones ACID en PostgreSQL; comandas persisten antes de confirmar.
R-08	Caída del backend en hora pico de servicio	2	4	8	Disponibilidad	Mitigar: nodemon/PM2, reinicio automático, monitoreo básico, procedimiento L2.
R-09	Sobrecosto por alcance no controlado (scope creep)	3	3	9	Gestión	Mitigar: alcance cerrado por fases; RFC para cualquier nuevo módulo; backlog.
R-10	Dependencia de un único desarrollador (bus factor = 1)	4	4	16	Gestión	Mitigar: documentación integral (este manual), README, Henao420 como respaldo.
R-11	Datos personales y de facturación expuestos en red local	2	4	8	Seguridad/Legal	Mitigar: HTTPS, segmentación de red, política de acceso, Ley 1581 de 2012 (habeas data).
R-12	Incumplimiento de la norma de evidencia académica del proyecto formativo	2	3	6	Gestión	Mitigar: actas, control de versiones de este documento y evidencias de ejecución.

Tabla 6 — Registro de riesgos del proyecto con evaluación cuantitativa

2.8.1 Mapa de calor resumido

- **Críticos (12–16):** R-02, R-06, R-10 — requieren atención permanente del equipo directivo del proyecto.
- **Altos (8–10):** R-01, R-03, R-04, R-05, R-08, R-09, R-11 — mitigación con acciones fechadas en el plan de calidad.
- **Medios (4–6):** R-07, R-12 — seguimiento en cada revisión de avance.
- **Bajos (≤3):** ninguno activo en este periodo.

2.9 Análisis del entorno del negocio

2.9.1 Análisis DOFA (SWOT) del restaurante frente al proyecto

Fortalezas (F)	Debilidades (D)
F1. Marca gastronómica en posicionamiento (parrilla premium, mixología de autor).	D1. Procesos manuales sin control de comandas ni stock en tiempo real.
F2. Ticket promedio alto y clientela recurrente (tarjetas y	D2. Dependencia de la memoria del administrador para

Fortalezas (F)	Debilidades (D)
efectivo).	decidir reposición de insumos.
F3. Menú estable de 12 platillos insignia — facilita digitalización.	D3. Personal operativo con alta rotación; curva de aprendizaje de herramientas exigida.
F4. Inventario de proveedores documentado (26 proveedores reales con NIT).	D4. Sin historial digital de comandas: imposible medir platillo estrella o ritmo de despacho.
F5. Equipo técnico local motivado y cliente participativo en diseño.	D5. Sin datos para negociar con proveedores (compras por intuición).
Oportunidades (O)	**Amenazas (A)**
O1. Certificaciones sanitarias (HACCP) apoyadas en trazabilidad de lotes y vencimientos.	A1. Competencia con POS integrado y apps de domicilios.
O2. Marketing de datos: platillo estrella, horas pico, ticket medio por mesero.	A2. Inflación de insumos exige compras inteligentes (stock óptimo).
O3. Fidelización digital (puntos, frecuencia) con datos de facturación.	A3. Riesgo sanitario por ingredientes vencidos si no hay alertas.
O4. Escalabilidad a domicilio sin rehacer el núcleo (misma API).	A4. Pérdida de datos o caída del sistema en hora pico.

Tabla 7 — Matriz DOFA del cliente frente al proyecto POS D'Piero

2.9.2 Análisis de causa raíz — diagrama de Ishikawa (6M)

Dimensión	Causas identificadas	Efecto
Mano de obra	Personal sin capacitación en registro digital; rotación alta; errores de comanda manuscrita.	Comandas ilegibles y platillos entregados mal.
Método	Toma de pedido en papel; sin validación de stock al vender; cierre de caja manual.	Ventas de productos sin existencia; descuadres de caja.
Materia prima	Insumos sin clasificación por vencimiento; compras sin política de stock mínimo.	Mermas del 3–5 % del costo de materia prima mensual.
Maquinaria	Un único equipo de caja sin red confiable; sin UPS.	Interrupciones del servicio en hora pico.
Medición	No existían indicadores (tiempo/comanda, errores/turno, ventas/mesero).	Gestión por intuición; imposible corregir a tiempo.
Medio ambiente	Iluminación cálida del salón (tema oscuro inicial ilegible); ruido.	Errores de lectura en pantalla con tema oscuro.

Tabla 8 — Causas raíz estructuradas con Ishikawa aplicado al proceso de comanda

2.9.3 Mapa de stakeholders

Stakeholder	Interés / expectativa	Influencia	Estrategia de gestión
Administrador (propietario)	Control total, reportes, cierre de caja rápido, menos mermas.	Muy alta	Participación en diseño; demos semanales; firma de alcance.
Administrador 2 (socio operativo)	Operación diaria fluida; facturación sin esfuerzo.	Alta	Capacitación 2 sesiones; manual de usuarios impreso.
Meseros (4)	PIN rápido, sin memorizar claves, comandas sin esfuerzo.	Media	Capacitación por turno; PIN de 4 dígitos.

Stakeholder	Interés / expectativa	Influencia	Estrategia de gestión
Cajero/a (1)	Cobro y cambio sin errores; reportes de caja.	Alta	Acceso a facturación; documento de caja.
Cocina (1-2)	Comandas legibles con notas; sin gritos del salón.	Media	Pantalla de comandas con notas visibles.
Proveedores (26)	Pedidos y pagos a tiempo; datos correctos.	Baja	Sin acceso al sistema (solo datos maestros).
Comensales	Pedido correcto y rápido; factura clara.	Media	Sin acceso; beneficio indirecto por calidad.
SENA (instructor/ficha 3172293)	Evidencia académica completa y verificable.	Alta	Este manual; revisión en 2 fechas; rúbrica.
Equipo desarrollador	Proyecto entregable a tiempo; aprendizaje.	Alta	Metodología iterativa; backlog visible.
DIAN (futuro)	Facturación electrónica eventual.	Nula hoy	Arquitectura preparada (order_items normalizado).

Tabla 9 — Mapa de stakeholders con estrategia de involucramiento

2.9.4 Matriz de comunicación

Involucrado	Qué se comunica	Frecuencia	Canal	Responsable
Cliente (propietario)	Avances por sprint, riesgos, aprobación de diseño	Quincenal	Reunión presencial / WhatsApp	Aprendiz líder
Personal operativo	Capacitación, cambios de procedimiento, credenciales	Al despliegue y por release	Sesión 30 min en salón	Administrador
Instructor SENA	Evidencias, manuales, correcciones de rúbrica	En cada entrega	Classroom + correo	Aprendiz líder
Henao420 (colaborador)	Tareas de ramas, PRs, revisión de código	Continuo	GitHub (PR, issues)	AndresVlz2020
Equipo directivo restaurante	Métricas de operación post-go-live	Mensual	Correo con PDF adjunto	Aprendiz líder

Tabla 10 — Plan de comunicación y gestión de expectativas

2.9.5 RACI del proyecto

Actividad	Aprendiz líder	Henao420	Cliente	Instructor SENA
Levantamiento de requerimientos	R (ejecuta)	C	A (aprueba alcance)	C
Diseño de arquitectura	R	C	C	A (validez técnica)
Modelado de datos (MR)	R	C	I	A
Desarrollo backend	R	R (soporte)	I	I
Desarrollo frontend	R	R (vistas)	C	I
RBAC y permisos	R	R	C	A
Sede de pruebas y QA	R	C	A	C
Documentación (3 manuales)	R	I	I	A
Despliegue y capacitación	R	I	A	I

Actividad	Aprendiz líder	Henao420	Cliente	Instructor SENA
Cierre y sustentación	R	C	C	A

Tabla 11 — Matriz RACI (R=Responsable, A=Aprobador, C=Consultado, I=Informado)

2.9.6 Estructura de desglose del trabajo (WBS/EDT) — nivel 3

```
1  SISTEMA POS D'PIERO
├─ 1.1  GESTIÓN DEL PROYECTO
│    ├─ 1.1.1  Acta de constitución y alcance
│    ├─ 1.1.2  Plan de comunicaciones y RACI
│    ├─ 1.1.3  Gestión de riesgos (registro y monitoreo)
│    └─ 1.1.4  Actas de revisión (25/09 y 04/11)
├─ 1.2  REQUERIMIENTOS
│    ├─ 1.2.1  Elicitación (entrevistas, observación)
│    ├─ 1.2.2  RF-01..24 · RNF-01..12 · RN-01..15 · RT-01..12
│    └─ 1.2.3  Validación con el cliente (firmar alcance)
├─ 1.3  ANÁLISIS
│    ├─ 1.3.1  10 casos de uso + escenarios Gherkin
│    ├─ 1.3.2  12 diagramas de clases / CRC
│    └─ 1.3.3  Diagramas de actividad y secuencia
├─ 1.4  DISEÑO
│    ├─ 1.4.1  Arquitectura 3 capas + justificación
│    ├─ 1.4.2  Modelo relacional (16 tablas, MR)
│    └─ 1.4.3  Prototipos (4 iteraciones visuales)
├─ 1.5  CONSTRUCCIÓN
│    ├─ 1.5.1  Migraciones 0001..0027 (esquema + seeds)
│    ├─ 1.5.2  Backend Express (9 features, RBAC, JWT)
│    ├─ 1.5.3  Frontend React (8 features, POS, reportes)
│    └─ 1.5.4  Integración visual y tema claro
├─ 1.6  CALIDAD
│    ├─ 1.6.1  Métricas de código (LOC 15.806, CC 10,9)
│    ├─ 1.6.2  30 casos de prueba + humo API
│    └─ 1.6.3  Defectos: registro y corrección
├─ 1.7  IMPLEMENTACIÓN
│    ├─ 1.7.1  Despliegue dev (documentado) + prod (runbook)
│    ├─ 1.7.2  Respaldo pg_dump y plan de restauración
│    └─ 1.7.3  Capacitación del personal operativo
└─ 1.8  CIERRE
     ├─ 1.8.1  Manuales (Administrativo, Técnico, Usuarios)
     ├─ 1.8.2  Sustentación (15 minutos)
     └─ 1.8.3  Acta de cierre y lecciones aprendidas
```

Tabla 12 — Estructura de desglose del trabajo en tres niveles

2.9.7 Estimación de esfuerzo y costo

Concepto	Estimación	Base de cálculo
Esfuerzo total	≈ 720 horas-hombre (6 meses × 120 h/mes)	54 commits, 31 semanas, 2 desarrolladores parciales
Costo de desarrollo (valor referencia)	≈ $36.000.000 COP	720 h × $50.000/h (tarifa aprendiz/desarrollador junior)
Licencias de software	$0 COP	React/Vite/Express/PostgreSQL/Node son open source

Concepto	Estimación	Base de cálculo
Infraestructura (desarrollo)	≈ $0 COP	Hardware propio del aprendiz
Infraestructura (producción propuesta)	≈ $150.000–$400.000 COP/mes	VPS 2 vCPU/4 GB + dominio + backup gestionado
Capacitación	≈ $600.000 COP	2 sesiones × 3 h × 8 personas × $25.000/h administrador
Mantenimiento anual propuesto	≈ $4.800.000 COP	10 h/mes × $40.000/h
Costo total del primer año	≈ $42.000.000 COP vs. POS comercial ≈ $9.000.000/año × 3 sedes	Benchmarking con soluciones comerciales

Tabla 13 — Estimación de costo total del proyecto (COP)

2.9.8 Métricas de éxito del proyecto

- **Adopción.** 100 % de comandas pasan por el sistema a los 30 días de go-live (línea base: 0 % en papel).
- **Precisión.** Errores de comanda reportados < 5 % respecto al histórico manual.
- **Inventario.** Merma de insumos vencidos < 2 % mensual (línea base 3–5 %).
- **Cierre de caja.** De 30–45 min a < 10 min.
- **Satisfacción operativa.** ≥ 4/5 en encuesta interna de personal a los 3 meses.
- **Disponibilidad.** ≥ 99 % en horario de servicio (salón 11:00–22:00).

3. Fase de Requerimientos

3.1 Técnicas de recolección

Se aplicaron técnicas complementarias de ingeniería de requerimientos, combinando métodos cualitativos (comprensión del negocio) con verificación empírica sobre el sistema construido:

Técnica	Descripción	Resultado obtenido
Entrevista semiestructurada	Sesiones con el administrador del restaurante y 2 meseros: flujo de salón, cuellos de botella, prioridades. Preguntas abiertas sobre toma de pedidos, cierre de caja e inventario.	Inventario funcional priorizado; roles y permisos requeridos; métricas de dolor (tiempo de comanda, errores/día).
Observación directa	Presencia en turno de almuerzo y noche: registro de tiempos por etapa (pedido → cocina → entrega → cobro), formatos en papel usados, vocabulario real del personal.	Mapa de proceso AS-IS; validación de que la app móvil/POS de escritorio es el canal adecuado.
Revisión documental	Análisis del menú físico, formato de comanda manual, hoja de inventario, listado de proveedores y cierres de caja anteriores.	Catálogo base de 12 productos, 26 proveedores, 30 insumos, 10 mesas y 5 órdenes de ejemplo (semilla 0027).

Técnica	Descripción	Resultado obtenido
Análisis de contexto competitivo	Revisión de soluciones POS genéricas (Square, POSBistro, Siigo) para identificar brecha funcional del cliente.	Decisión de construir solución a la medida con costos de licencia nulos.
Tormenta de ideas (brainstorming)	Sesión equipo-cliente con 24 ideas; priorización con MoSCoW (Must/Should/Could/Won't).	Backlog de 18 historias de usuario (sección 4.2) y clasificación de 9 Must-have.
Prototipado de baja fidelidad	Wireframes de caja POS, listados y formularios en papel/Excalidraw antes de programar.	Validación temprana con el cliente; reducción de cambios tardíos.
Prototipado de alta fidelidad	Pantallas reales en React con datos semilla; revisión iterativa (tema oscuro → claro, cards, navbar).	Aceptación del cliente en la segunda iteración visual (ver historial Git: 68add21).
Análisis del modelo de datos	Ingeniería inversa del esquema SQL migrado (27 migraciones) para validar correspondencia con el dominio.	16 tablas, 21 permisos, 4 grupos, 5 content_type; 100 % de módulos cubiertos.
Checklist ISO/IEC 25010	Evaluación de mantenibilidad, seguridad, usabilidad y portabilidad del código.	Registro de deuda técnica y riesgos R-04, R-05, R-10.
Verificación empírica	Ejecución del sistema y pruebas de humo sobre cada endpoint y pantalla (evidencias en Manual de Usuarios).	Confirmación funcional RT/RF implementados; hallazgos en sección 7.4.

Tabla 14 — Técnicas de recolección de requerimientos aplicadas

3.2 Requerimientos Funcionales

Cada requerimiento se identifica con el patrón RF-NN, módulo responsable, prioridad MoSCoW (M = Must, S = Should, C = Could) y fuente (E = entrevista, O = observación, D = documental, I = implementación verificada).

ID	Requerimiento funcional	Módulo	Prio	Fuente
RF-01	El sistema debe autenticar al personal operativo con un PIN de 4 dígitos, listando los usuarios activos en pantalla de acceso rápido.	Auth	M	E, I
RF-02	El sistema debe autenticar al administrador con correo electrónico y contraseña, validando el hash bcrypt en el backend.	Auth	M	E, D, I
RF-03	El sistema debe entregar un JWT con expiración configurable y adjuntarlo en cada petición desde el cliente.	Auth	M	D, I
RF-04	El sistema debe permitir recuperar la contraseña enviando un código de 6 dígitos al correo registrado (vigencia 15 minutos, un solo uso).	Auth	S	E, I
RF-05	El sistema debe listar los usuarios con: ID, nombre, rol, correo, teléfono, estado (activo/inactivo), con búsqueda, paginación de 5 filas y orden.	Usuarios	M	E, O, I
RF-06	El sistema debe crear usuarios con: nombre, documento, tipo y número, teléfono, correo, PIN o contraseña según rol, avatar y fechas laborales.	Usuarios	M	E, I
RF-07	El sistema debe editar usuarios existentes, preservando la contraseña si el campo llega vacío o con el placeholder.	Usuarios	M	I
RF-08	El sistema debe activar/desactivar usuarios en tiempo real con un interruptor, sin recargar la página.	Usuarios	M	O, I
RF-09	El sistema debe sincronizar automáticamente el grupo RBAC del usuario según su user_type al crearlo o editarlo.	Usuarios/RBAC	S	D, I
RF-10	El sistema debe gestionar productos (menú): crear, listar, consultar, editar,	Productos	M	E, I

ID	Requerimiento funcional	Módulo	Prio	Fuente
	activar/desactivar, con categoría, precio, stock e imagen.			
RF-11	El sistema debe listar productos con filtros por categoría (Platos Fuertes, Entradas, Bebidas, Postres, Guarniciones) y búsqueda de texto.	Productos	M	O, I
RF-12	El sistema debe gestionar el inventario de insumos: crear, listar, editar, activar/desactivar, con stock mínimo, unidad y ubicación.	Inventario	M	E, I
RF-13	El sistema debe alertar visualmente los insumos con stock por debajo del mínimo (estilo ámbar) y marcar los inactivos.	Inventario	S	O, I
RF-14	El sistema debe gestionar proveedores: crear, listar, consultar ficha, editar, activar/desactivar; NIT único por proveedor.	Proveedores	M	E, I
RF-15	El sistema debe crear comandas (órdenes) con mesa, mesero asignado, ítems con notas, cálculo de total y descuento automático de stock.	Órdenes	M	E, O, I
RF-16	El sistema debe listar comandas con búsqueda, filtro por estado, paginación y vista de detalle en modal con impresión de comanda/factura.	Órdenes	M	E, I
RF-17	El sistema debe permitir editar comandas abiertas (cambiar mesa, ítems, notas) y anular comandas con registro de motivo.	Órdenes	M	E, I
RF-18	El sistema debe mostrar la disponibilidad de mesas (libre/ocupada) en tiempo real con base en comandas abiertas/en preparación.	Órdenes	S	O, I
RF-19	El sistema debe mostrar el POS con las pestañas autorizadas según el rol: menú (todos), clientes/facturación (admin y cajero), personal y proveedores (admin).	POS	M	E, I
RF-20	El sistema debe gestionar permisos RBAC por grupo y por usuario: ver, asignar y desasignar permisos con checkbox sobre los 21 permisos.	Accesos	M	E, I
RF-21	El sistema debe generar reportes de usuarios en PDF y Excel con campos y alcance configurables.	Reportes	S	E, I
RF-22	El sistema debe cerrar la sesión automáticamente tras 10 minutos de inactividad.	Seguridad	S	E, I
RF-23	El sistema debe mostrar alertas (éxito, error, advertencia, confirmación) con mensajes sanitizados sin exponer detalles internos.	UX	S	D, I
RF-24	El sistema debe persistir imágenes de avatar/logo/insumos como URL y servirlas desde los assets públicos.	Compartido	C	I

Tabla 15 — Requerimientos funcionales RF-01 a RF-24 con prioridad y fuente

3.3 Requerimientos No Funcionales

ID	Categoría ISO 25010	Requerimiento no funcional	Métrica / verificación
RNF-01	Adecuación funcional	La API debe responder en JSON con códigos HTTP semánticos (200, 201, 400, 401, 403, 404, 500).	100 % de endpoints devuelven códigos correctos (verificado por inspección y pruebas de humo).
RNF-02	Eficiencia de desempeño	Listados de hasta 50 registros deben cargar en menos de 2 s con BD local y red LAN.	Consultas indexadas (idx_reset_codes_email_code, PK/UK); pool de 10 conexiones.
RNF-03	Compatibilidad	El frontend debe funcionar en Chrome/Edge últimos 2 años; responsive ≥ 1024 px (óptimo 1280×720).	Probado en Chrome 130 / Edge 130 (entorno de desarrollo).
RNF-04	Usabilidad	Toda operación crítica (crear comanda) debe completarse en ≤ 6 clics desde la terminal POS.	Verificado en recorrido de usuario (Manual de Usuarios, capítulo 6).
RNF-05	Mantenibilidad	Código organizado por features, con comentarios explicativos en español y lint limpio (ESLint).	README técnico del frontend; `npm run lint` sin errores bloqueantes.
RNF-06	Seguridad	Contraseñas con bcrypt costo 10; JWT firmado; PIN nunca expuesto fuera de usuario y terminal.	Inspección de auth.service.js y user.service.js.

ID	Categoría ISO 25010	Requerimiento no funcional	Métrica / verificación
RNF-07	Seguridad	CORS restringido a orígenes localhost/127.0.0.1 con credentials; sin exposición pública.	app.js — cors() con validación de origin.
RNF-08	Confiabilidad	La creación de comanda es transaccional (todo o nada) con ROLLBACK ante error.	order.repository.js — BEGIN/COMMIT/ROLLBACK verificado.
RNF-09	Seguridad	Cierre de sesión por inactividad (10 min) y limpieza de credenciales en cliente.	SessionTimeout.jsx + useInactivityTimeout.js.
RNF-10	Portabilidad	Backend y frontend ejecutables en Windows/Linux con Node.js 18+ y PostgreSQL 12+.	Sin dependencias nativas salvo bcrypt (prebuilds disponibles).
RNF-11	Usabilidad	Modo claro/oscuro y persistencia de preferencia sin parpadeo (script anti-FOUC en index.html).	index.html — clase theme-dark/theme-light pre-hidratación.
RNF-12	Mantenibilidad	Esquema de BD versionado con migraciones idempotentes y tabla _migrations.	migrate.js — 27 migraciones aplicadas y verificadas (0001..0027).

Tabla 16 — Requerimientos no funcionales clasificados por ISO/IEC 25010

3.4 Reglas del negocio

ID	Regla de negocio	Módulo	Implementación
RN-01	Un usuario operativo (mesero, cajero, cocina) accede con PIN de 4 dígitos; no usa contraseña. Un administrador usa correo y contraseña.	Auth/Usuarios	user.service.js — create(): admin→bcrypt, operativo→pin.
RN-02	El administrador general es el superusuario (is_superuser=true); pasa cualquier verificación de permiso.	RBAC	access.service.js — isSuperUser() corto-circuito true.
RN-03	Los permisos se otorgan por grupo (rol) y se pueden sobreescribir o complementar por usuario (asignación directa).	RBAC	getUserPermissions(): UNION user_permissions ∪ group_permissions.
RN-04	Crear o editar un usuario sincroniza su grupo según user_type: admin→Administrador, caj*→Cajero, cocin*/parrill→Cocina, resto→Mesero.	Usuarios/RBAC	user.repository.js — resolveGroupNameFromUserType() + syncUserGroup().
RN-05	Los códigos ORD-### se generan con la secuencia MAX(id)+1 con relleno a 3 dígitos; editable si trae prefijo ORD-.	Órdenes	order.repository.js — create().
RN-06	Cada venta de comanda descuenta stock: stock = GREATEST(0, stock - cantidad).	Órdenes	order.repository.js — UPDATE products.
RN-07	El total de la comanda es la suma de (precio × cantidad) de sus ítems; recalculado en creación y edición.	Órdenes	order.repository.js / order.service.js.
RN-08	Una comanda pasa por estados: Pendiente → Abierta → En preparación → Pagada → Cerrada; o Anulada desde cualquier estado activo.	Órdenes	order.service.js — validStatuses.
RN-09	La disponibilidad de mesa es 'Ocupada' si existe comanda activa (Abierta, En preparación, Pendiente, en_proceso) con ese número de mesa.	Órdenes	getTablesAvailability().
RN-10	Un proveedor o producto inactivo no debe usarse en nuevas operaciones; el POS solo muestra productos is_active=true.	Catálogo	getAllProducts({activeOnly}) + updateStatus().
RN-11	El código de recuperación de contraseña es de 6 dígitos, vence a los 15 minutos, es de un solo uso e invalida códigos anteriores.	Auth	auth.service.js + password_reset_codes.
RN-12	La nueva contraseña debe tener mínimo 10 caracteres; el sistema la hashea con bcrypt costo 10.	Auth	resetPassword() — validación + hash.
RN-13	La cuenta del administrador se bloquea tras 3 intentos fallidos consecutivos (lado cliente; ver limitación L-04).	Auth	Login.jsx — login_attempts_* en localStorage.

ID	Regla de negocio	Módulo	Implementación
RN-14	Los permisos se heredan como grupo→usuario solo si el usuario no tiene asignación directa; ambos se suman en la consulta UNION.	RBAC	getUserPermissions().
RN-15	Un producto no puede eliminarse físicamente desde la UI; se desactiva lógicamente (is_active=false, status='Inactivo').	Productos	updateProductStatus().

Tabla 17 — Reglas de negocio verificadas en el código fuente

3.5 Restricciones

ID	Restricción	Tipo	Consecuencia de incumplimiento
RT-01	Base de datos exclusivamente PostgreSQL ≥ 12 (no se admite MySQL/SQLite por diferencias de tipos y PL/pgSQL).	Técnica	El pool `pg` y las migraciones fallan; el sistema no arranca.
RT-02	Node.js ≥ 18 (desarrollo y producción); gestión de paquetes con npm y package-lock.json.	Técnica	Instalación potencialmente rota por ABI de bcrypt.
RT-03	El backend expone la API solo por HTTP en puerto 4000 en desarrollo; en producción debe ser HTTPS detrás de proxy.	Técnica/ Seguridad	Tokens JWT interceptables en texto plano.
RT-04	El frontend SPA asume same-origin o CORS restringido a localhost; no debe exponerse públicamente sin revisión de seguridad.	Seguridad	Orígenes arbitrarios podrían leer respuestas autenticadas.
RT-05	Los PIN de 4 dígitos son de terminal compartida; no son un factor de autenticación fuerte.	Seguridad	Suplantación si el PIN se vuelve conocido; políticas internas de rotación.
RT-06	Los archivos `.env` no deben versionarse; contienen secretos reales del entorno de desarrollo.	Seguridad	Fuga de JWT_SECRET/SMTP_PASS (ver riesgo R-03).
RT-07	Los nombres de tablas y columnas usan snake_case; los aliases de salida camelCase/snake_case para el frontend.	Técnica	Inconsistencia de contrato API ↔ UI y errores silenciosos.
RT-08	Las migraciones son inmutables una vez aplicadas; los cambios se hacen creando una nueva migración (NNNN_nombre.sql).	Técnica	Drift de esquema entre ambientes; fallos en _migrations.
RT-09	Máximo 12 archivos y 10 MB por carga de imagen (FileInput); validación lado cliente.	UX/Técnica	Cargas rechazadas o navegador lento.
RT-10	La moneda del negocio es el Peso Colombiano (COP), formato $185.000 y columna numeric(12,2).	Negocio	Cálculos con redondeos incorrectos si se usa float.
RT-11	Zona horaria del servidor y de la base de datos debe ser America/Bogotá (UTC-5).	Técnica	Desfases en fechas de comandas y vigencias de códigos.
RT-12	El despliegue de producción exige servidor Linux con systemd/PM2, Nginx como reverse proxy y PostgreSQL administrado.	Técnica	Pérdida de servicio ante caídas de proceso no manejadas.

Tabla 18 — Restricciones técnicas, de seguridad y de negocio

4. Fase de Análisis

4.1 Casos de Uso

Se identificaron 10 casos de uso principales a partir de las entrevistas y la observación (diagrama en la Ilustración siguiente). Cada caso se describe con: actor, precondición, flujo principal, flujos alternativos y postcondición. Se detalla completamente un caso de complejidad alta (CU-08); los demás se resumen en la tabla síntesis.

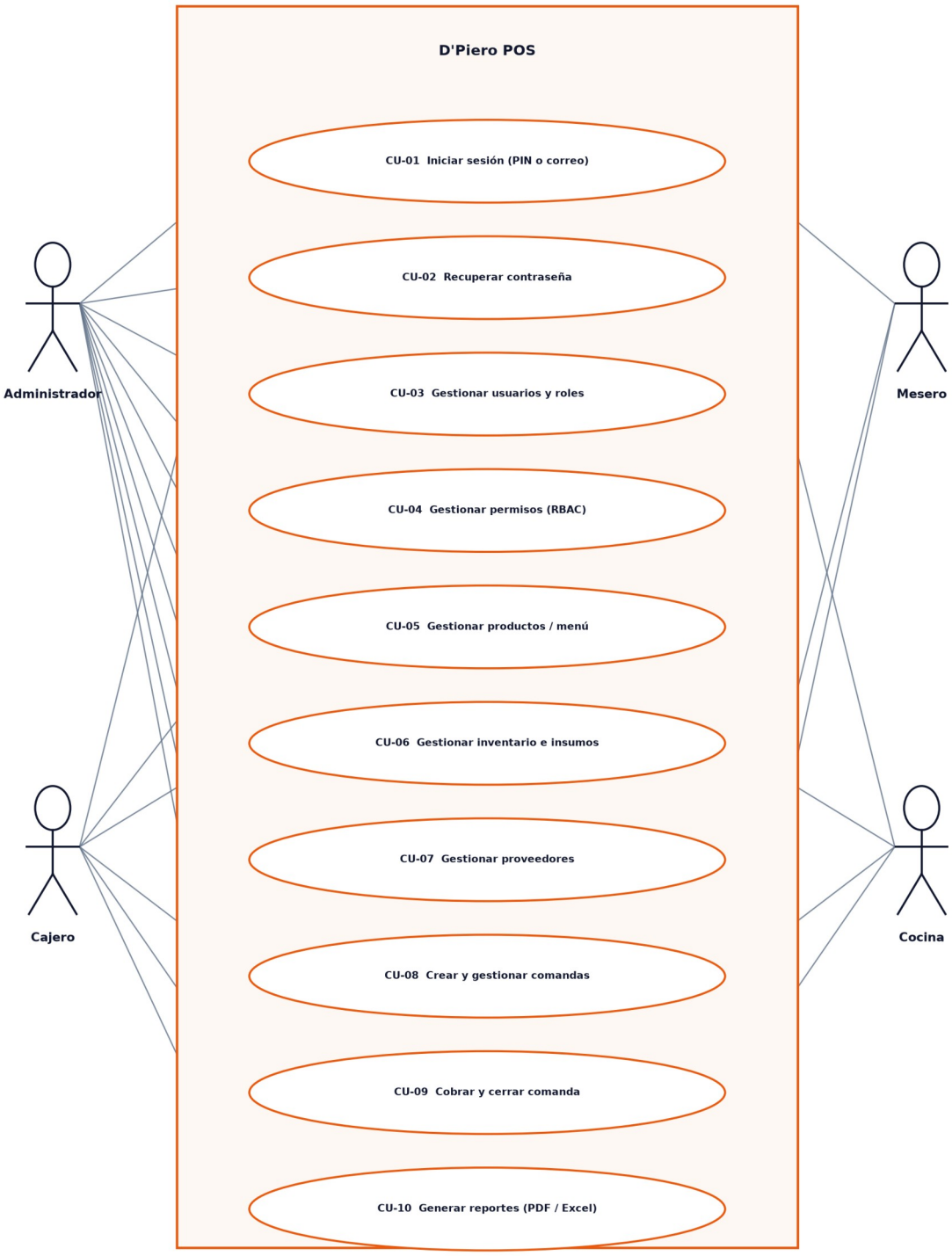


Figura 1 — Diagrama de casos de uso — actores y funcionalidades del sistema

ID	Nombre	Actor(es)	Precondición	Resultado esperado
CU-01	Iniciar sesión	Todos los roles	Usuario activo en la base de datos	Sesión iniciada; JWT almacenado; redirección según rol (admin → /dashboard/userList; resto → POS).

ID	Nombre	Actor(es)	Precondición	Resultado esperado
CU-02	Recuperar contraseña	Administrador	Correo registrado	Código de 6 dígitos enviado; contraseña actualizada con bcrypt.
CU-03	Gestionar usuarios	Administrador (list/create/update/disable)	Sesión admin válida	Usuario creado/actualizado/estado cambiado; grupo RBAC sincronizado.
CU-04	Gestionar permisos RBAC	Administrador (permiso 'admin')	Grupos y permisos sembrados (migraciones 0021–0026)	Matriz de permisos persistida por grupo o usuario.
CU-05	Gestionar productos / menú	Administrador (o con permiso create_product/update_product)	Categorías activas	Catálogo actualizado; POS refleja cambios con activeOnly.
CU-06	Gestionar inventario e insumos	Administrador / Cajero (list/update)	Proveedores registrados	Stock actualizado; alerta de mínimo; trazabilidad por lote.
CU-07	Gestionar proveedores	Administrador	Sesión admin	Proveedor creado/actualizado/activado/desactivado.
CU-08	Crear y gestionar comandas	Mesero / Cajero / Cocina (según permisos)	Mesa libre o comanda editable	Comanda creada con ítems y total; stock descontado.
CU-09	Cobrar y cerrar comanda	Cajero / Administrador	Comanda en En preparación/Abierta con total calculado	Comanda marcada como Pagada/Cerrada; factura opcional.
CU-10	Generar reportes	Administrador o usuario con report_user	Datos cargados en la pantalla	Archivo PDF/Excel descargado con campos seleccionados.

Tabla 19 — Síntesis de los diez casos de uso del sistema

4.1.1 CU-08 — Crear y gestionar comandas (descripción completa)

Elemento	Descripción
Actor principal	Mesero (también Cajero y Administrador).
Actores secundarios	Cocina (consulta/actualiza estado), sistema de inventario (descuento indirecto).
Precondiciones	1) Usuario autenticado con PIN. 2) Mesa existe en restaurant_tables y está libre o es edición de comanda abierta. 3) Productos con stock > 0 para poder agregarlos.
Flujo principal	1. El mesero entra al POS (tab Catálogo & Comandas) o a /CreateOrder. 2. Selecciona mesa libre (se muestra capacidad). 3. Verifica/edita mesero asignado. 4. Filtra productos por categoría o busca por nombre. 5. Agrega ítems con cantidades; cada línea muestra precio COP y subtotal. 6. Escribe notas de cocina (ej. 'término medio'). 7. Pulsa 'Registrar Comanda' → POST /api/orders. 8. Backend: BEGIN; calcula total; inserta orders (ORD-###) y order_items; descuenta products.stock; COMMIT; retorna comanda completa. 9. El usuario ve confirmación SweetAlert y regresa al listado.
Flujos alternativos	FA1. Si la mesa está ocupada, el frontend no permite seleccionarla (grid de mesas marca Ocupada). FA2. Si un producto tiene stock 0, el botón + lo inhabilita. FA3. Si el backend falla al descontar stock, ROLLBACK y el frontend muestra error sanitizado.
Flujos de excepción	FE1. Sin conexión: la alerta indica 'Error inesperado de conexión con el servidor backend'. FE2. PIN incorrecto: keypad se limpia; tras 3 intentos cliente-bloqueo (solo admin).
Postcondición	Comanda en estado 'Abierta' con order_items; products.stock decrementado; mesas ocupadas actualizadas; auditoría vía created_at.
Reglas de negocio	RN-05, RN-06, RN-07, RN-08, RN-09, RN-10, RN-15.

Elemento	Descripción
Seguridad	El endpoint /api/orders exige JWT + permiso create_order (mesero o rol superior) desde 2026-11-08; el frontend adjunta el token Bearer automáticamente en cada llamada.
Prioridad / frecuencia	Must-have / Muy alta (varias veces por hora en horario de servicio).

Tabla 20 — Especificación completa del caso de uso CU-08

4.1.2 CU-01 — Iniciar sesión (descripción completa)

Elemento	Descripción
Flujo principal (PIN)	En /Auth, pestaña Acceso Rápido: 1) el cliente carga los usuarios staff desde /api/users?activeOnly=true; 2) el operador toca su nombre; 3) aparece el teclado numérico; 4) ingresa 4 dígitos y pulsa ✓; 5) POST /api/auth/login con { userId, email, pin }; 6) backend compara PIN plano contra users.pin; 7) si ok, firma JWT (1 h) y retorna { token, user: { role } }; 8) el cliente guarda auth_token/auth_user y navega a '/'.
Flujo principal (admin)	En /Auth, pestaña Administración: 1) correo + contraseña; 2) POST /api/auth/login; 3) backend bcrypt.compare() contra user_password (fallback texto plano para seeds legacy); 4) JWT; 5) llamada hasPermission('list_user') → si granted, navega /dashboard/userList; si no, '/'.
Flujo alterno	V1 PIN incorrecto → 401 'Credenciales inválidas', keypad se limpia. V2 Usuario inactivo → 401 'Usuario inactivo'. V3 Backend fuera de línea → alerta 'Servidor Desconectado'; contingencia local únicamente para admin@dpiero.com / 12345678 (ver limitación L-03).
Postcondición	Sesión persistida en localStorage/sessionStorage; navbar muestra nombre y rol; rutas protegidas habilitadas según permisos.

Tabla 21 — Especificación del caso de uso CU-01

4.2 Historias de usuario

Las historias se priorizan con MoSCoW (M/S/C/W) y puntos de historia (Fibonacci). Se verificaron contra la implementación.

ID	Historia de usuario	Criterios de aceptación	Prio	Pts
HU-01	Como mesero, quiero iniciar sesión con mi PIN para atender las mesas sin demorar la operación.	DADO usuario activo CUANDO toca su nombre CON PIN correcto ENTONCES entra al POS en < 3 s.	M	3
HU-02	Como administrador, quiero crear usuarios operativos con PIN para dar de alta personal de cada turno.	DADO formulario válido CUANDO guarda ENTONCES aparece en la tabla y grupo RBAC sincronizado.	M	5
HU-03	Como cajero, quiero ver el catálogo filtrado por categoría para tomar pedidos rápido.	DADO POS cargado CUANDO selecciona 'Bebidas' ENTONCES lista solo productos de esa categoría.	M	3
HU-04	Como mesero, quiero agregar notas por ítem (sin cebolla, término) para que cocina no se equivoque.	DADO ítem agregado CUANDO escribe nota ENTONCES la nota viaja en POST y se imprime en la comanda.	M	2
HU-05	Como cajero, quiero ver mesas libres/ocupadas en tiempo real.	DADO mesa con comanda activa ENTONCES grid marca 'Ocupada' y no permite selección.	S	3
HU-06	Como administrador, quiero desactivar un producto sin perder su historial.	DADO toggle desactivado ENTONCES producto no aparece en POS y sigue en BD.	M	2
HU-07	Como administrador, quiero restaurar la contraseña de un usuario con un código de 6 dígitos.	DADO correo válido CUANDO pide código ENTONCES llega correo y el código expira a los 15 min.	S	5

ID	Historia de usuario	Criterios de aceptación	Prio	Pts
HU-08	Como administrador, quiero asignar permisos por grupo y por usuario sin reiniciar sesión.	DADO matriz de RBAC CUANDO marca checkbox ENTONCES se persiste y aplica en la siguiente verificación.	M	5
HU-09	Como administrador, quiero exportar usuarios a Excel/PDF con campos a medida.	DADO modal de reporte CUANDO selecciona campos y alcance ENTONCES descarga archivo legible.	S	5
HU-10	Como cajero, quiero anular una comanda con motivo para no facturar errores.	DADO comanda activa CUANDO anula con motivo ENTONCES queda 'Anulada' con motivo registrado.	S	3
HU-11	Como mesero, quiero ver el estado actual de mi comanda desde el listado.	DADO listado cargado CUANDO busca ORD-### ENTONCES ve estado, mesa y total.	M	2
HU-12	Como cocina, quiero ver las comandas abiertas con notas para priorizar la preparación.	DADO rol Cocina CUANDO abre /orderList ENTONCES lista comandas Abierta/En preparación.	S	2
HU-13	Como administrador, quiero personalizar qué campos salen en los reportes.	DADO modal reporte CUANDO marca nombre/email/documento ENTONCES el PDF incluye solo esos campos.	C	3
HU-14	Como administrador, quiero que la sesión expire sola tras 10 minutos para seguridad.	DADO 10 min sin interacción CUANDO continúa ENTONCES redirige a /Auth con sesión limpiada.	S	2
HU-15	Como administrador, quiero registrar proveedores con NIT único para compras y trazabilidad.	DADO NIT duplicado CUANDO guarda ENTONCES la BD rechaza por UNIQUE(nit).	M	2

Tabla 22 — Historias de usuario priorizadas (MoSCoW) y estimadas en puntos

4.3 Diagrama de Clases

La solución no usa un ORM; el 'modelo de clases' emerge de los repositorios SQL y sus *shaped rows*. El diagrama representa las entidades del dominio con sus atributos y operaciones principales, más las relaciones reales (FK, UNIQUE, M:N). Los atributos se resumen; el detalle completo está en el diccionario de datos (Anexo A y Manual Técnico, sección 8).



Figura 2 — Diagrama de clases del dominio D'Piero POS

4.3.1 Invariantes del modelo

- `users.user_email` es UNIQUE; `users.pin` nulo para administradores; `users.is_superuser` controla el pase maestro del RBAC.
- `products.category_id` → `categories.id` ON DELETE SET NULL: desactivar una categoría no borra productos.
- `order_items.product_id` → `products.id` ON DELETE RESTRICT: no se puede borrar un producto con historial de ventas.
- `order_items.order_id` → `orders.id` ON DELETE CASCADE: borrar la comanda elimina sus líneas.
- `inventory.supplier_id` → `suppliers.id` ON DELETE SET NULL: borrar proveedor no pierde el insumo.

- `user_groups` y `user_permissions` son PK compuestas (M:N reales) con CASCADE sobre ambas puntas.
- `password_reset_codes.expires_at` + `used` garantizan un solo uso y vigencia temporal.

4.4 Diagrama de Actividades

El flujo operativo central del sistema (comanda de punta a punta) se modela a continuación. Refleja exactamente lo implementado en `order.repository.js` (transacción BEGIN/COMMIT con descuento de stock) y en la UI (/CreateOrder → botón Registrar Comanda).

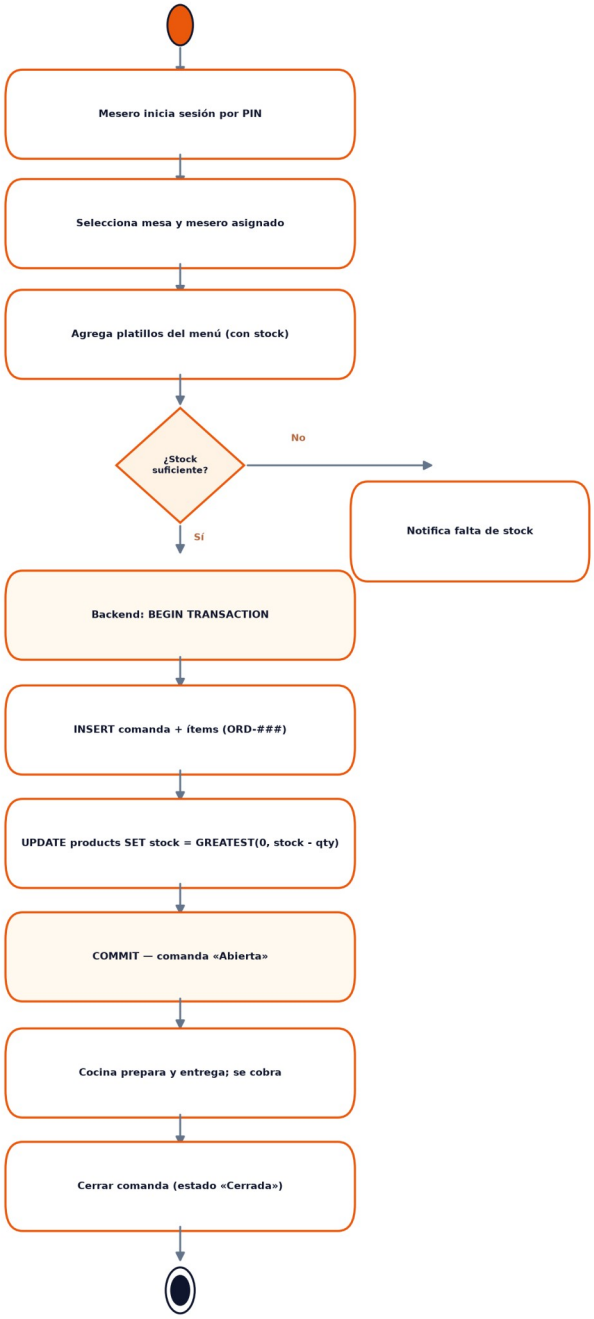


Figura 3 — Diagrama de actividades del proceso comanda-cobro

4.4.1 Descripción del flujo

- **Toma de pedido.** El mesero selecciona mesa y agrega platillos; la UI muestra disponibilidad por mesa y stock por platillo (12 platillos semilla).
- **Validación.** El cliente valida mesa libre, ítems > 0 y precio COP; el backend revalida conJoi/servicio (precio ≥ 0, mesa obligatoria).

- **Persistencia transaccional.** BEGIN → INSERT orders con ORD-### → INSERT order_items → UPDATE products.stock (GREATEST 0) → COMMIT. Ante error, ROLLBACK completo: no quedan ítems huérfanos.
- **Operación de cocina.** El estado 'En preparación' lo ve cocina (rol Cocina lista comandas y actualiza).
- **Cobro y cierre.** Caja marca 'Pagada'/'Cerrada'; 'Anulada' registra motivo y excluye la comanda de ventas.

4.5 Especificación detallada de los casos de uso restantes

4.5.1 CU-02 — Recuperar contraseña del administrador (detalle)

Elemento	Descripción
Actor(es)	Administrador (único actor; operativo usa PIN y no recupera por correo).
Precondición	Correo registrado en `users` con is_active=true.
Flujo principal	1. En /Auth → '¿Olvidó su contraseña?' (link en AdminLoginForm). 2. Sistema navega a /Auth/forgot-password. 3. Usuario ingresa su correo y pulsa 'Enviar Código de Seguridad al Correo'. 4. Frontend POST /api/auth/forgot-password { email }. 5. Backend normaliza a minúsculas; busca usuario; si no existe → 400 'No existe ningún usuario registrado con ese correo'. 6. Genera código aleatorio de 6 dígitos (Math.random), lo guarda en password_reset_codes con expires_at = ahora + 15 min (los anteriores se marcan used). 7. sendPasswordResetEmail() envía el correo HTML con el código (si SMTP configurado; si no, imprime el código en consola con aviso). 8. Respuesta 200 { success, message, email }. 9. Usuario ingresa a /Auth/reset-password: correo, código, nueva clave (≥10), confirmar. 10. POST /api/auth/reset-password → valida código (no usado, no vencido), hashea bcrypt(10), hace UPDATE y marca el código used. 11. Respuesta 200; usuario ya puede iniciar sesión.
Flujos alternativos/excepción	A1: correo inexistente → 400 con mensaje. A2: código vencido/usado → 400 'inválido o expirado'. A3: nueva clave < 10 → 400 'al menos 10 caracteres'. A4: SMTP no configurado → log en consola (entorno dev); el sistema NO revela el código fuera de logs.
Postcondición	Contraseña reemplazada; token anterior invalidado por firma (JWT_SECRET) si se rotó; código marcado usado.
Reglas de negocio	RN-11, RN-12.
Prioridad / frecuencia	S / baja (ocasional).

4.5.2 CU-03 — Gestionar usuarios (crear, editar, desactivar)

Elemento	Descripción
Actor(es)	Administrador autenticado con permiso create_user/update_user/disable_user.
Precondición	Sesión admin; BD accesible; grupo destino existe (migraciones 0022).
Flujo principal	1. Navbar → Usuarios (o /dashboard/userList). 2. Botón 'Crear usuario' (visible con permiso) → /CreateUser. 3. Formulario dividido en 2 columnas: imagen con FileInput (máx 12 archivos/10 MB), 'Nombre Completo', 'Número de Documento', 'Tipo de Usuario' (Cajero/Mesero/Cocina/Administrador — límite 2/2), 'Tipo de Documento' (CC, CE, TI, PAS,

Elemento	Descripción
	NIT), 'Correo Electrónico', 'Número Telefónico', 'PIN de Terminal POS' (solo operativos), 'Fecha Inicio/Fin Laboral'. 4. Validación: nombre ≥ 3, documento numérico ≥ 6, correo válido, PIN 4 dígitos. 5. Submit → POST /api/users (JWT requerido) → 201. 6. Backend: si tipo Administrador → bcrypt(10) con clave por defecto '12345678' si no se envía; pin NULL; is_superuser=true. Sino → pin (por defecto '1234'), sin contraseña. 7. syncUserGroup(): asigna/elimina user_groups y propaga group_permissions → user_permissions. 8. En el listado se muestra nueva fila con toggle activo. 9. Edición: /edituser o lápiz → PUT /api/users/:id; contraseña vacía → se conserva (placeholder 'Password12345#'). 10. Desactivar: PATCH /api/users/:userId/status { isActive:false } — UI actualiza toggle sin recargar.
Flujos alternativos/excepción	A1: Admin límite 2/2 → opción deshabilitada en el select. A2: correo duplicado → UNIQUE error; UI muestra error sanitizado. A3: PUT sin cambiar clave → conserva hash anterior. A4: FileInput con valor null → crash conocido (D-01) en edición de proveedores, no en usuarios.
Postcondición	Usuario persistido en users; grupo y permisos sincronizados; visible en POS/listados según estado.
Reglas de negocio	RN-01, RN-04, RN-15.
Prioridad / frecuencia	M / media (alta rotación de personal).

4.5.3 CU-04 — Gestionar permisos (RBAC) por grupo y por usuario

Elemento	Descripción
Actor(es)	Administrador con permiso 'admin' (is_superuser o grupo Administrador).
Precondición	Datos semilla de permisos (0023–0025) y grupos (0022) presentes; 5 content_type cargados.
Flujo principal	1. Navbar hamburger → Admin (Gestión de permisos) → /dashboard/access. 2. Dos paneles: 'Grupos usuarios' (select Administrador/Cajero/Mesero/Cocina) y 'Usuario individual' (select por nombre). 3. Al elegir un grupo, se cargan sus group_permissions; los 21 checkbox se marcan en verde si concedido. 4. El admin marca/desmarca permisos (agrupados por módulo: Usuarios, Productos, Órdenes, Inventario, Proveedores). 5. Pulsa guardar → PUT /api/groups/:groupId/permissions { permissionIds: [...] } → backend borra y reinserta en transacción (BEGIN/COMMIT). 6. Para usuario individual: PUT /api/users/:userId/permissions idéntico. 7. Los cambios aplican en la siguiente verificación hasPermission() (frontend: ProtectedRoute/Navbar; backend: permission.middleware).
Flujos alternativos/excepción	A1: permisos heredados del grupo visibles también en el panel de usuario (consulta UNION) pero no editables ahí; se editan en el panel de grupo. A2: usuario sin permisos directos muestra los de su grupo (fallback en user.repository.js). A3: si el backend falla → 500 con mensaje; UI muestra error.
Postcondición	group_permissions / user_permissions persistidas; efecto inmediato en autorizaciones.
Reglas de negocio	RN-02, RN-03, RN-04, RN-14.
Prioridad / frecuencia	M / baja (cambios ocasionales).

4.5.4 CU-05 — Gestionar productos / menú (CRUD completo)

Elemento	Descripción
Actor(es)	Administrador o usuario con create_product / update_product / disable_product.
Precondición	Categorías activas en BD; sesión válida.
Flujo principal	1. Navbar → Productos → /productList. 2. Tabla con columnas: Código, Producto (imagen), Categoría, Precio, Stock, Estado (toggle), Descripción. 3. Búsqueda local por texto; paginación 5/página; exportar (Generar reporte — disponible según permiso). 4. 'Crear producto' →

Elemento	Descripción
	/CreateProduct: nombre*, estado inicial, precio (COP)*, categoría*, descripción. 5. Guardar → POST /api/products → validación: nombre obligatorio; precio ≥ 0 y numérico; categoría por nombre o categoryId. 6. /api/products devuelve 201; la lista se actualiza. 7. 'Ver' (ojo) → /readproduct con location.state (detalle solo lectura). 8. 'Editar' (lápiz) → /editproduct: nombre, estado, precio, categoría, descripción; PUT /api/products/:id. 9. Toggle estado → PATCH /api/products/:id/status; is_active=false excluye del POS.
Flujos alternativos/excepción	A1: precio negativo o vacío → validación front + 400 backend. A2: nombre vacío → 'El nombre del producto es obligatorio'. A3: categoría inexistente en select → error. A4: toggle a Inactivo → producto oculto en POS (HomePage filtra activeOnly=true).
Postcondición	Catálogo persistido; POS actualizado en la siguiente carga (datos frescos, sin caché).
Reglas de negocio	RN-10, RN-15.
Prioridad / frecuencia	M / media (cambios de carta ocasionales).

4.5.5 CU-06 — Gestionar inventario e insumos

Elemento	Descripción
Actor(es)	Administrador (todo) y Cajero (list/update según grupo).
Precondición	Proveedores registrados (opcional pero recomendable); sesión válida.
Flujo principal	1. Navbar → Inventario → /inventoryList. 2. Tabla: Código (INV-###), Insumo, Marca, Categoría, Estado (toggle), Stock actual (ámbar si < mínimo), Proveedor, Última actualización. 3. Filtro por categoría; búsqueda; paginación 5/página; Refrescar y Generar reporte. 4. 'Crear insumo' → /CreateInventory: proveedor (select PostgreSQL), marca (Alpina, Diana, …), fecha de vencimiento, nombre*, lote, stock actual, stock mínimo (alerta, default 5), descripción, ubicación (default 'Almacén Central / Bodega'), precio compra (default 45000). 5. Guardar → POST /api/inventory → 201; código autogenerado INV-### si no trae. 6. 'Editar' → PUT /api/inventory/:id con los mismos campos; stock mínimo dispara alerta visual en la tabla. 7. Toggle estado → PATCH /api/inventory/:id/status; status Activo/Inactivo.
Flujos alternativos/excepción	A1: stock < mínimo → celda ámbar '2 un. (bajo)'. A2: proveedor no asignado → 'Proveedor no asignado' (COALESCE). A3: vencimiento pasado → se guarda pero conviene reportar (sin bloqueo). A4: categoría con comas → normalizada en BD.
Postcondición	Inventario actualizado; alertas visibles; vinculación con proveedor para compras.
Reglas de negocio	RN-10, RN-15.
Prioridad / frecuencia	M / alta (diario).

4.5.6 CU-07 — Gestionar proveedores

Elemento	Descripción
Actor(es)	Administrador (list/create/update/disable_supplier).
Precondición	Sesión admin.
Flujo principal	1. Navbar → Proveedores → /supplierList. 2. Tabla: NIT, Compañía (imagen), Email, Teléfono, Estado (toggle), acciones. 3. Búsqueda; paginación 5/página (26 proveedores semilla); exportar reporte. 4. 'Crear proveedor' → /CreateSupplier: logo (FileInput), NIT*, nombre*, correo*, teléfono*, dirección, observaciones. 5. POST /api/suppliers (multipart/form-data sin archivos — upload.none()) → validación; NIT UNIQUE. 6. 'Ver' (ojo) → /readsupplier: ficha en solo lectura (información general, datos de contacto, comerciales y términos). 7. 'Editar' (lápiz) → /editsupplier: mismo formulario prellenado; PUT /api/suppliers/:id. 8. Toggle → PATCH

Elemento	Descripción
	/api/suppliers/:id { is_active }.
Flujos alternativos/excepción	A1: NIT duplicado → 500 con error UNIQUE capturado y mostrado. A2: /editsupplier CRASH conocido (D-01): FileInput recibe userImage=null. A3: eliminar físico no soportado (solo desactivar).
Postcondición	Proveedor registrado/actualizado; disponible en selects de inventario y productos.
Reglas de negocio	RN-10, RN-15.
Prioridad / frecuencia	M / media.

4.5.8 CU-09 — Cobrar, cerrar o anular una comanda

Elemento	Descripción
Actor(es)	Cajero o Administrador (permiso update_order; cancel_order para anulación privilegiada).
Precondición	Comanda existe en BD y está en estado activo (Abierta/En preparación/Pagada).
Flujo principal	1. /orderList → buscar por estado/texto; abrir 'Ver detalle' (ojo). 2. Modal 'Comanda ORD-###': cabecera (ubicación, estado, mesero, fecha), tabla de ítems (cant./platillo/precio/subtotal), notas de cocina, total liquidado ($ COP). 3. Botones: 'Imprimir Factura' (window.print) y 'Cerrar Comanda' (cambia estado → Cerrada, PATCH /api/orders/:id/status). 4. Alternativa: 'Editar' (lápiz) → /editorder: cambiar mesa, mesero, estado (Abierta/En preparación/Pagada/Cerrada/Anulada/Pendiente), ajustar platillos (sumar/restar cantidades), notas; 'Guardar Cambios' (PUT) o 'Anular Comanda' (PATCH /:id/cancel con motivo opcional). 5. Anulación marca Anulada; la comanda queda visible pero excluida de ventas. 6. Al imprimir se usa el diálogo de impresión del navegador (formato de factura pendiente de CSS @media print).
Flujos alternativos/excepción	A1: intentar editar comanda Cerrada/Anulada → la UI lo permite pero el negocio lo desaconseja; estado Anulada es absorbente. A2: cancel sin motivo → se acepta con motivo vacío. A3: factura impresa sin impresora → diálogo del navegador.
Postcondición	Comanda cerrada/anulada; total congelado; histórica disponible en /orderList.
Reglas de negocio	RN-08, RN-09.
Prioridad / frecuencia	M / alta (varias veces por hora).

4.5.9 CU-10 — Generar reportes operativos (PDF / Excel)

Elemento	Descripción
Actor(es)	Administrador o usuario con report_user.
Precondición	Sesión válida; pantalla del módulo cargada con datos.
Flujo principal	1. En /dashboard/userList → 'Generar reporte'. 2. Modal 'Generar reporte de usuarios': select de formato (PDF / Excel), checkboxes de campos (Nombre ✓, Email ✓, Tipo de documento ✓, Documento ✓, Teléfono ☐, Dirección ☐), alcance ('Todos los usuarios' / 'Solo usuarios activos'). 3. 'Generar reporte' → generateUserReport(): construye dataset (client-side, sobre el array ya cargado), y con: (a) PDF: jsPDF + jspdf-autotable → descarga 'reporte_usuarios_2026-11-08.pdf' con membrete D'Piero; (b) Excel: XLSX.utils.json_to_sheet → 'reporte_usuarios.xlsx'. 4. El archivo se descarga con nombre con fecha ISO; contenido: encabezado, logotipo opcional, tabla con columnas seleccionadas, pie con total de registros.
Flujos alternativos/excepción	A1: sin permisos report_user → botón no visible (ProtectedRoute en /dashboard index; en listado el botón depende del permiso). A2: lista vacía → reporte con 0 filas y aviso. A3: navegador bloquea descarga → se permite en ajustes.

Elemento	Descripción
Postcondición	Archivo descargado; sin alteración del servidor (reportes 100 % client-side).
Reglas de negocio	RN-02.
Prioridad / frecuencia	S / semanal (cierre y administración).

4.6 Escenarios Gherkin de aceptación (Must-have)

Especificación ejecutable de los nueve Must-have del backlog. Sirven como base para futura automatización con Cucumber/Playwright.

```
Característica: Autenticación en dos vías
  Escenario: Acceso rápido con PIN válido
    Dado que soy mesero activo con PIN 1234
    Cuando toco mi perfil en Acceso Rápido e ingreso el PIN
    Entonces el sistema me lleva al POS con las pestañas de mi rol

  Escenario: Acceso administrativo y bloqueo por intentos
    Dado que soy admin@dpiero.com en la pestaña Administración
    Cuando ingreso una contraseña incorrecta 3 veces
    Entonces mi cuenta queda bloqueada y veo el aviso de límite

Característica: Ciclo de comanda
  Escenario: Crear comanda con descuento de stock
    Dado que la Mesa 2 está libre y el producto PAR-01 tiene stock 18
    Cuando agrego 2 unidades de Punta de Anca y registro la comanda
    Entonces se crea ORD-xxx por $112.000 y el stock queda en 16

  Escenario: Comanda atómica
    Dado que la comanda incluye un producto con stock insuficiente
    Cuando el backend procesa la transacción
    Entonces se ejecuta ROLLBACK y no queda ninguna fila insertada

Característica: RBAC
  Escenario: Herencia desde grupo
    Dado que soy Mesero (grupo con list_product, list_order, create_order, update_order)
    Cuando mi nombre aparece en el navbar
    Entonces veo Productos y Pedidos pero no Inventario ni Usuarios

  Escenario: Permiso directo sobreescribe al grupo
    Dado que el administrador marca list_inventory como permiso directo de Carlos
    Cuando Carlos recarga la sesión
    Entonces Carlos ve el Inventario en el navbar

Característica: Inventario
  Escenario: Alerta de stock mínimo
    Dado que INV-004 tiene stock actual 2 y mínimo 5
    Cuando abro /inventoryList
    Entonces la fila aparece destacada en ámbar con la marca (bajo)

Característica: Reportes
  Escenario: Reporte PDF con campos elegidos
    Dado que selecciono PDF con Nombre y Email únicamente
    Cuando genero el reporte con alcance 'Solo usuarios activos'
```

```
Entonces descargo un PDF con tabla de 6 columnas como máximo y sin filas inactivas
```

4.7 Tarjetas CRC (Clase — Responsabilidad — Colaboración)

Clase	Responsabilidades	Colaboradores
User	Encapsular identidad y credenciales; validar estado activo; mapear a grupos	Group, UserGroup, Permission, Order
Group	Agrupar permisos por rol de trabajo	Permission, GroupPermission, UserGroup
Permission	Autorizar una acción sobre un módulo	ContentType, UserPermission, GroupPermission
ContentType	Proporcionar catálogo de módulos para agrupar permisos	Permission
Product	Exponer platillo vendible con precio y stock	Category, OrderItem, Inventory
Category	Clasificar productos	Product
Inventory	Registrar existencia y trazabilidad de insumo	Supplier
Supplier	Proveer datos comerciales de insumos	Inventory
Order	Agrupar pedido de una mesa con total y estado	OrderItem, User (waiter), RestaurantTable
OrderItem	Línea de producto con cantidad y nota	Order, Product
RestaurantTable	Disponibilidad física de la mesa	Order
PasswordResetCode	Garantizar recuperación un solo uso y vigente	User

Tabla 23 — Tarjetas CRC de las 12 entidades centrales

4.8 Diagrama de secuencia de autenticación

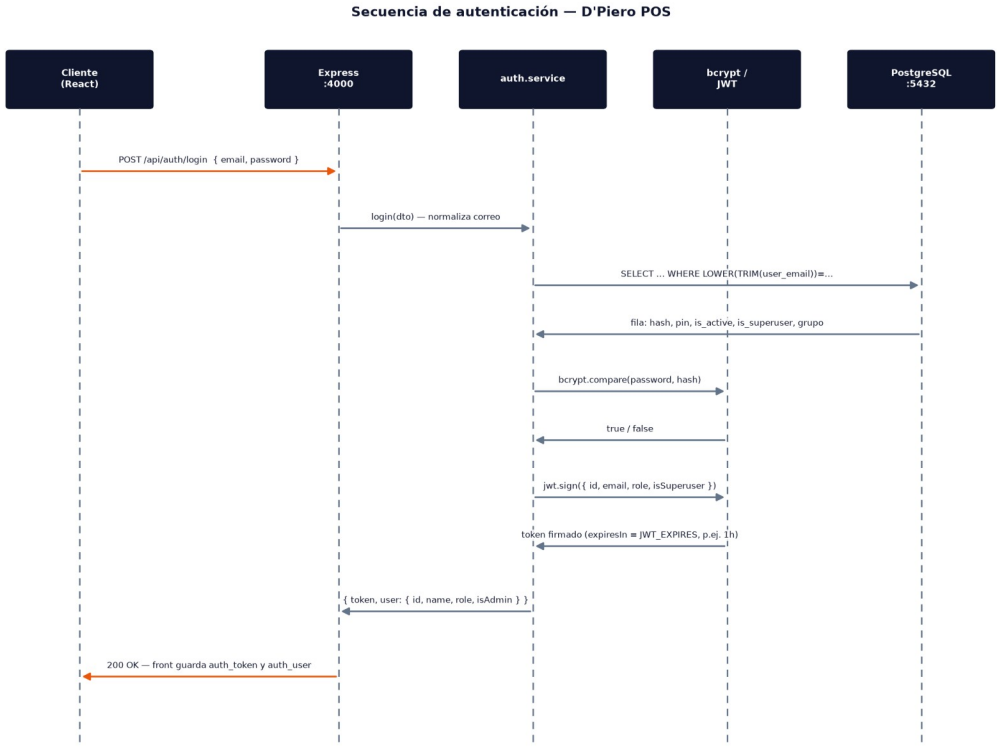


Figura 4 — Secuencia login completo: cliente → Express → servicio → bcrypt/JWT → PostgreSQL

5. Fase de Diseño

5.1 Diagrama de Componentes

El sistema se compone de tres componentes desplegables y de un servicio externo (Gmail SMTP). La comunicación frontend↔backend es exclusivamente HTTP/JSON; la comunicación backend↔BD es TCP con el protocolo nativo de PostgreSQL a través del pool `pg`. No hay acoplamiento directo entre el navegador y la base de datos.

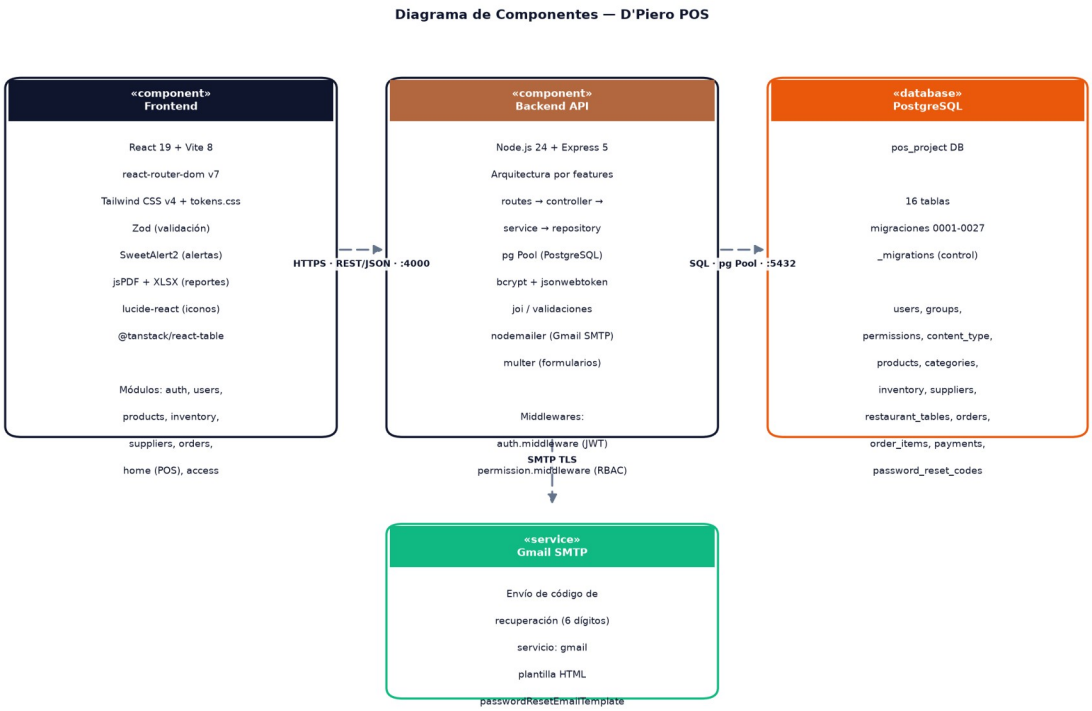


Figura 5 — Diagrama de componentes — Frontend, Backend, PostgreSQL y Gmail SMTP

Componente	Responsabilidad	Interfaces que expone / consume
Frontend SPA (React)	UI, validación de formularios (Zod), estado de sesión, consumo REST, generación de PDF/Excel, tema claro/oscuro	Consume: /api/* (HTTP, JSON). Expone: ninguna pública (es cliente).
Backend API (Express)	Lógica de negocio, RBAC, transacciones, orquestación de persistencia, envío de correos	Expone: 9 grupos de rutas REST bajo /api. Consume: PostgreSQL 5432, Gmail SMTP 465/587.
PostgreSQL (pos_project)	Persistencia relacional, integridad referencial, transacciones ACID, seed y migraciones	Expone: protocolo PostgreSQL. Consume: nadie.
Gmail SMTP	Entrega del correo con código de 6 dígitos para recuperación de contraseña	Consume: nodemailer con service='gmail' y contraseña de aplicación.
Migraciones (migrate.js)	Aplicar esquema y seeds de forma idempotente y transaccional	Consume: archivos backend/database/migrations/*.sql.
Generación de reportes	buildReportDataset + jsPDF/XLSX en cliente	Consume: mismos endpoints GET de usuarios; no hay endpoints de reporte en el backend (el PDF se arma en el navegador).

Tabla 24 — Contratos de los componentes y sus interfaces

5.2 Diagrama de Distribución (Despliegue)

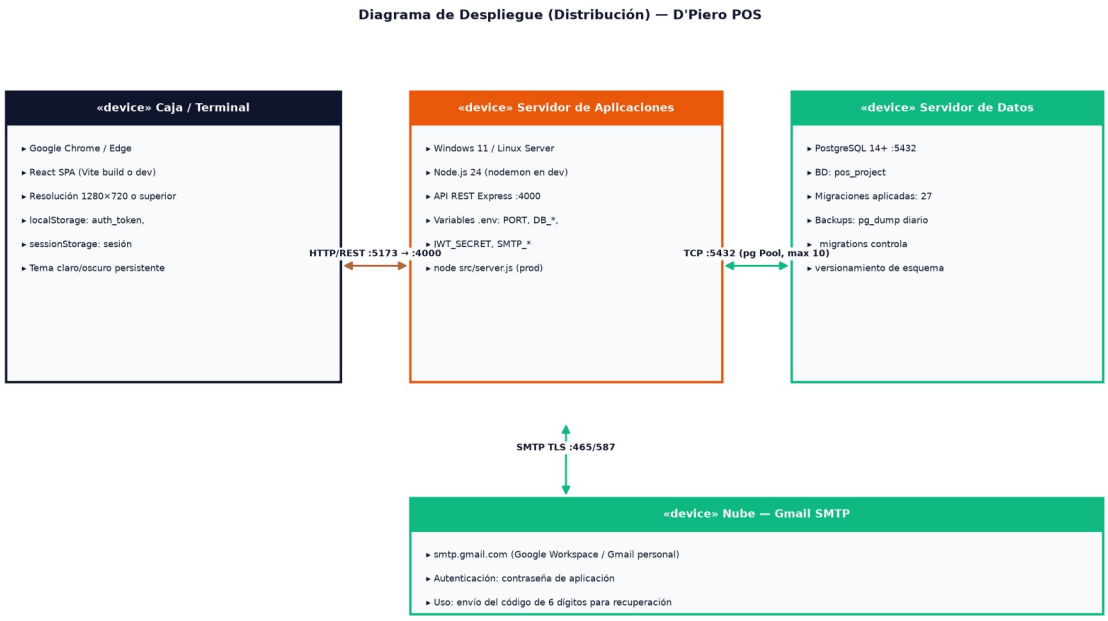


Figura 6 — Diagrama de distribución — topología de despliegue por dispositivo

Nodos: (1) dispositivo de caja/terminal (navegador moderno); (2) servidor de aplicaciones (Windows 11 o Linux Server con Node.js); (3) servidor de datos PostgreSQL; (4) nube de correo (Gmail). En desarrollo todo corre en una sola máquina; en producción recomendada se separan aplicación y datos, y el frontend puede servirse estático detrás de Nginx (build `npm run build` → `dist/`).

5.2.1 Variante de producción recomendada

```
# Frontend (build estático) — Nginx reverse proxy
server {
  listen 443 ssl;
  server_name pos.dpiero.example.co;
  root /var/www/pos-dpiero/dist;      # resultado de `npm run build`
  location /api/ { proxy_pass http://127.0.0.1:4000/api/; }
  location / { try_files $uri /index.html; }
}

# Backend — systemd (Linux)
[Service]
ExecStart=/usr/bin/node /opt/pos/backend/src/server.js
EnvironmentFile=/opt/pos/backend/.env
Restart=always
User=pos
```

5.3 Aplicaciones Distribuidas

D'Piero POS es una aplicación distribuida cliente–servidor de tres capas con comunicación sin estado (stateless) entre capas. Características relevantes:

- **Statelessness.** Ningún estado de sesión vive en el servidor: el JWT firmado contiene id, email, role, isSuperuser, isAdmin; expira en JWT_EXPIRES. El backend no guarda sesiones en memoria (escala horizontalmente si se requiere).
- **Separación de capas.** La UI nunca ejecuta SQL; el backend nunca renderiza HTML. El contrato es el JSON de los 9 routers. Los repositorios encapsulan el acceso a datos con SQL parametrizado.
- **CORS deliberado.** El backend admite únicamente orígenes localhost/127.0.0.1 en desarrollo y el origen exacto FRONTEND_URL en producción (lista blanca); cualquier otro origen es rechazado con error desde 2026-11-08.
- **Recursos estáticos vs API.** Vite sirve el frontend (HMR en dev) y la API REST vive en :4000; no hay SSR: es una SPA pura con React Router en cliente.
- **Transacciones distribuidas.** No se requieren 2PC: la operación comanda es local a PostgreSQL (BEGIN/COMMIT). El único servicio externo (Gmail) es de efecto eventual y su fallo no revierte datos.
- **Consistencia.** Lecturas de catálogo desde cache del navegador con revalidación (304); escritura siempre contra la BD con COMMIT antes de responder.
- **Tolerancia a fallos.** Si el backend cae, el frontend muestra alerta de 'servidor desconectado'; la contingencia admin@dpiero.com/12345678 existe SOLO para el admin demo (limitación L-03).
- **Escalabilidad.** Backend sin estado → N instancias tras balanceador; BD es el punto de contención (pool 10). Para una sede (< 50 comandas/día) es sobredimensionado.

5.4 Arquitectura del software y su justificación

5.4.1 Arquitectura adoptada: por capas + por features (modular)

- **Capas clásicas.** Presentación (React) → Aplicación (Express: routes/controller/service/repository) → Datos (PostgreSQL). Cada capa solo depende de la de abajo.

- **Organización por features.** El backend agrupa por dominio (`features/products`, `features/orders`, …) en lugar de por tipo técnico (`controllers/`, `models/`). Cada feature es autocontenida: rutas, controlador, servicio, repositorio.
- **Contratos explícitos.** El repositorio devuelve shaped rows con camelCase y aliases para el frontend (ej. `userName`, `order_number AS idCode`).
- **Middlewares transversales.** `auth.middleware` (JWT) y `permission.middleware` (RBAC) se componen por ruta; el resto es público.
- **Frontend espejo.** `src/features/<dominio>/{components,pages,table,services,schemas,data}` reproduce la separación por dominio; `shared/` aloja lo transversal.

5.4.2 Alternativas consideradas y descartadas

Alternativa	Por qué se descartó
Monolito MVC tradicional (EJS/Handlebars en el servidor)	Duplicaría lógica de render en servidor y cliente; rompería la separación SPA/API; peor experiencia de desarrollo.
Microservicios (catálogo, órdenes, inventario separados)	Complejidad operativa sin justificación: un solo dominio y un solo equipo; overhead de red/versionamiento de contratos.
ORM completo (Prisma/TypeORM)	El equipo prefirió SQL explícito parametrizado por transparencia y performance; 27 migraciones manuscritas dan control fino del esquema.
Next.js / Remix (SSR)	Innecesario para un POS privado detrás de login; el HTML estático + API basta y simplifica el despliegue.
MySQL / SQLite	Restricción RT-01: se estandarizó PostgreSQL por transacciones, NUMERIC exacto y ecosistema.
Gestión de estado con Redux / Zustand	El estado es local por pantalla + localStorage/sessionStorage; no hay estado global complejo; sumaría ceremonia sin valor.

Tabla 25 — Decisiones de arquitectura con su justificación

5.4.3 Calidad arquitectónica atribuida

- **Mantenibilidad.** Feature modules acotados; cambios localizados; contratos de fila estables (ALIAS).
- **Testeabilidad.** Separationroutes→controller→service→repository permite mockear pool/JWT/bcrypt.
- **Seguridad por diseño.** RBAC en dos capas (UI oculta y backend deniega), transacciones con ROLLBACK, secretos por entorno.

- **Flexibilidad.** Cambiar motor de BD implicaría reescribir repositorios únicamente; la UI no cambia.
- **Observabilidad.** Logs de consola ([Backend] …) y errores del middleware global con `err.stack`; limitación: sin APM/Sentry (ver RT/L).

5.5 Diseño de la Base de Datos (MR)

El modelo relacional físico tiene 16 tablas (17 con _migrations). A continuación el mapa entidad-relación en tres subgrafos: seguridad/RBAC, operación del restaurante y abastecimiento.

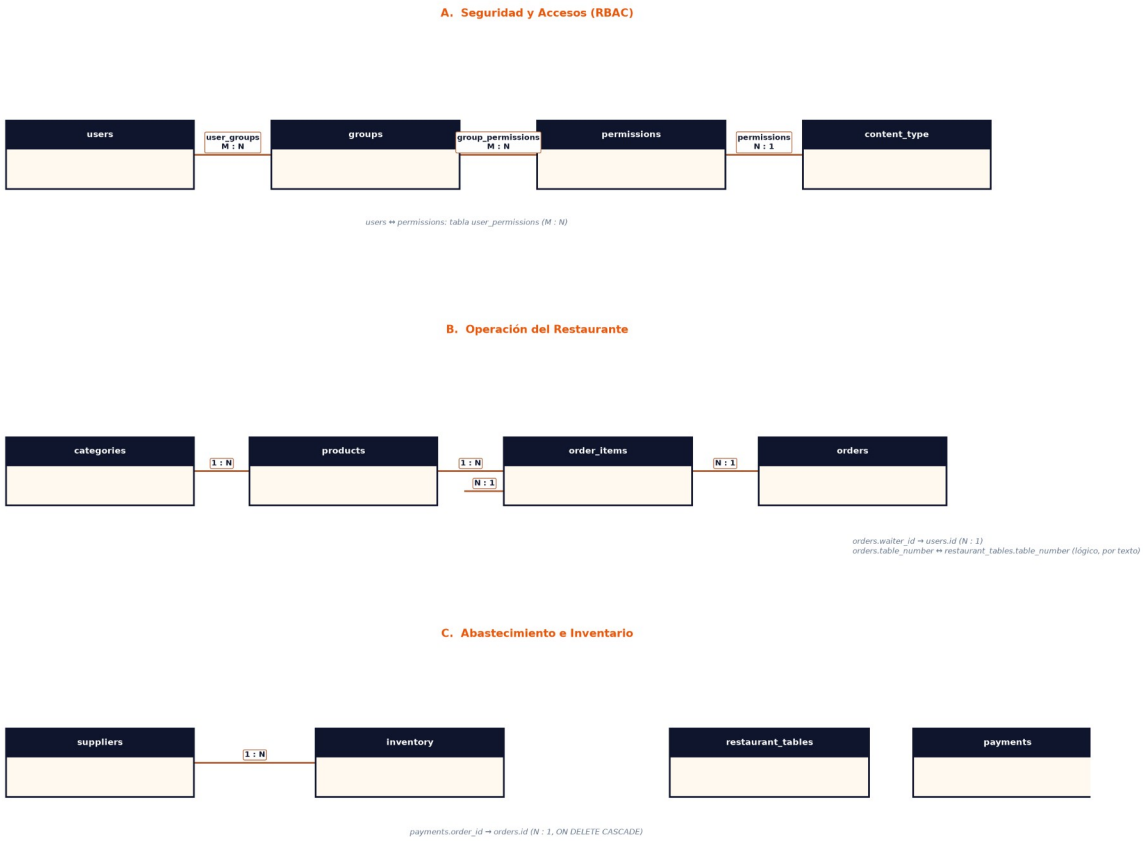


Figura 7 — Modelo entidad-relación — tres subgrafos funcionales

5.5.1 Tablas y propósito

Tabla	Filas semilla	Propósito
users	8	Identidad, credenciales (bcrypt/PIN), tipo, estado, avatar, fechas laborales.
categories	5	Categorías del menú (platos fuertes, entradas, bebidas, postres, guarniciones).
products	12	Platillos: código, nombre, precio COP, stock, categoría, imagen, estado.
suppliers	26	Proveedores gastronómicos con NIT único, contacto y estado.

Tabla	Filas semilla	Propósito
inventory	30	Insumos: stock actual/mínimo/máximo, unidad, ubicación, lote, vencimiento.
restaurant_tables	10	Mesas 1–10 con capacidad (2–10 comensales) y estado.
orders	7	Comandas: ORD-###, mesa, mesero, estado, total, observaciones.
order_items	5	Líneas de comanda: producto, cantidad, precio unitario, subtotal, notas.
payments	0	Cobros por comanda (método, monto, propina) — preparada, sin uso aún.
permissions	21	Permisos granulares por módulo (list/create/update/disable/report…).
groups	4	Roles RBAC: Administrador, Cajero, Mesero, Cocina.
group_permissions	34	Matriz grupo→permisos.
user_groups	8	Asignación usuario→grupo.
user_permissions	69	Asignación directa usuario→permiso (herencia materializada).
content_type	5	Catálogo de módulos (users, products, orders, inventory, suppliers).
password_reset_codes	3	Códigos de 6 dígitos con expiración y flag used.
_migrations	27	Control de migraciones aplicadas (idempotencia).

Tabla 26 — Inventario de tablas, datos semilla y propósito

5.6 Prototipos

5.6.1 Evolución de los prototipos

- **Iteración 1 — Wireframes en papel/Excalidraw.** Boceto del POS de caja: grid de productos, panel lateral de comanda, cierre de turno. Validado con el cliente en la entrevista 2.
- **Iteración 2 — Tema oscuro.** Primera versión visual (`#090D1E`): navbar negro, cards oscuras, acento naranja. Commits `c4fcfa2`, `68add21`.
- **Iteración 3 — Tema claro híbrido.** Rediseño con tokens CSS (`tokens.css`) y `semantic.css`: tema claro persistente + toggle; mejora de contraste y legibilidad para salón iluminado. Commits `a8e5b44`, cambios actuales sin confirmar.
- **Iteración 4 — Ajustes de marca.** Logo D'Piero, favicon, tipografía Plus Jakarta Sans, fondos de fotografía de restaurante en auth.

5.6.2 Inventario de pantallas (31 rutas registradas en el router)

Ruta	Pantalla	Rol mínimo
/	Terminal POS (Catálogo & Comandas, Personal, Proveedores, Clientes)	Todos (sin token; datos reales si hay sesión)
/Auth	Login de acceso rápido (PIN) y acceso administrativo	Público

Ruta	Pantalla	Rol mínimo
/Auth/forgot-password	Recuperación de contraseña (correo + código 6 dígitos)	Público
/Auth/reset-password	Verificación de código y nueva contraseña	Público
/dashboard (index)	Formulario de creación de usuario (alta rápida)	Permiso create_user
/dashboard/userList	Listado de usuarios con paginación y acciones	Permiso list_user
/dashboard/access	Gestión de permisos RBAC (grupos y usuarios)	Permiso admin
/dashboard/edituser	Edición de usuario	Permiso admin
/dashboard/productList · /productList · /products	Listado de productos	Todos (lectura)
/dashboard/inventoryList · /inventoryList · /inventory	Listado de inventario con alertas de mínimo	Permiso list_inventory para creación/edición
/dashboard/supplierList · /supplierList	Listado de proveedores	Permiso list_supplier
/dashboard/orderList · /orderList · /orders	Listado de comandas	Todos (lectura)
/dashboard/CreateOrder · /CreateOrder	Crear comanda (mesa + mesero + ítems)	Todos
/dashboard/editorder · /editorder · /EditOrder	Editar comanda ORD-###	Permiso update_order
/CreateProduct	Crear producto (formulario completo)	Permiso create_product
/readproduct	Ficha de solo lectura del producto	Todos
/editproduct	Edición de producto (estado, precio, stock)	Permiso update_product
/CreateInventory	Crear insumo con lote/vencimiento/proveedor	Permiso create_inventory
/CreateSupplier	Alta de proveedor con NIT único	Permiso create_supplier
/readsupplier · /editsupplier	Ficha y edición de proveedor	Permiso list/update_supplier
/CreateUser · /UsersManagement	Alta de usuario / administración consolidada	Permiso admin
/readuser · /edituser · /editsupplier	Consultar y editar registros	Permisos correspondientes

Tabla 27 — Rutas y pantallas del sistema (frontend/src/app/router.jsx)

5.7 Normalización y formas normales (análisis del MR)

Cada tabla se somete al análisis 1NF → 2NF → 3NF. La tabla `_migrations` (control) queda fuera del alcance.

Tabla	1NF	2NF	3NF	Observaciones
users	✓	✓ (sin dependencias parciales: PK simple)	✓ (city/zip no existen; corporate_email es atributo dependiente de id)	user_email UNIQUE para login; user_type legado y user_groups coexisten (redundancia deliberada de transición).
categories	✓	✓	✓	name UNIQUE; descripción libre.
products	✓	✓	✓	category_id FK; stock y code añadidos por migración 0027.
suppliers	✓	✓	✓	nit UNIQUE; is_active; dirección desnormalizada (patrón aceptado para ficha simple).
inventory	✓	✓	Parcial: supplier_name se resuelve vía join (no se guarda	Depende de suppliers solo por id; brand es atributo del insumo.

Tabla	1NF	2NF	3NF	Observaciones
			duplicado); brand es atributo propio	
restaurant_tables	✓	✓	✓	status legado + derivación en tiempo real en getTablesAvailability().
orders	✓	✓	✓	table_number texto (no FK a restaurant_tables) — decisión deliberada por flexibilidad.
order_items	✓	✓	✓	subtotal persistido (desnormalización controlada para auditoría); también recalculado en servicio.
payments	✓	✓	✓	Preparada, sin datos; FK order_id CASCADE.
permissions	✓	✓	✓	content_type_id FK con matriz app/model.
groups / group_permissions / user_groups / user_permissions	✓	✓	✓	PK compuestas; CASCADE total; grano fino RBAC.
content_type	✓	✓	✓	UNIQUE (app_label, model).
password_reset_codes	✓	✓	✓	Índice (email, code) para validación rápida.

Tabla 28 — Análisis de formas normales tabla por tabla

5.8 Modos de fallo y resiliencia por componente

Componente	Fallo típico	Detección	Mitigación automática	Acción humana
Frontend (navegador)	JS error, estado corrupto	Error boundary de router (faltante: errorElement no definido)	Recargar página; alerta SweetAlert genérica	Reportar con captura; Ctrl+F5
Backend (Express)	Excepción no capturada	process.on('uncaughtException') + middleware global de errores	El proceso NO muere (guarda registro); 500 JSON	Revisar logs; reiniciar (nodemon lo hace)
Pool PostgreSQL	Conexión perdida	pool.on('error') log; consultas rechazadas	Reintento automático del pool para conexiones nuevas	Verificar servicio postgresql; reiniciar
Consulta SQL	Error de sintaxis/dato	try/catch en repository → controller → 500/400	ROLLBACK en transacciones compuestas	Revisar migración; restaurar desde backup
Gmail SMTP	Credenciales inválidas/red	nodemailer error; catch → log 'Error al enviar correo'	La recuperación SIGUE funcionando mostrando el código por consola en dev	Renovar contraseña de aplicación
Disco lleno	pg_dump falla	Error de escritura en cron	Retención automática de 30 días libera espacio	Ampliar disco; borrar dumps antiguos

Tabla 29 — Modos de fallo, detección y resiliencia por componente

5.9 Inventario descriptivo de las 31 pantallas del sistema

Cada pantalla con su propósito, componentes principales y puntos de atención. Las capturas reales están en el Manual de Usuarios; aquí la ficha funcional:

/Auth — Acceso Rápido

Selección de perfil de turno con lista de usuarios frecuentes y buscador por nombre/cargo/documento. Componentes: BrandLogo, QuickAccessSelector, búsqueda local, avatar por usuario. Punto de atención: sin sesión previa muestra solo activos (activeOnly).

/Auth — Teclado PIN

PinKeypad con botones 0–9, tecla limpiar y confirmar; valida 4 dígitos; error inline y reset automático. Punto de atención: el PIN por defecto 1234 debe cambiarse en producción.

/Auth — Administración

AdminLoginForm con correo corporativo y clave, recordar y enlace de recuperación. Validación Zod; contador de intentos; contingencia documentada. Punto de atención: no exponer en redes públicas sin TLS.

/Auth/forgot-password

Solicita correo y envía código. Mensajes genéricos que no revelan existencia de cuenta (fallo de diseño: actual devuelve 400 si no existe — corregir a respuesta uniforme).

/Auth/reset-password

Correo + código + nueva clave + confirmar. Validación ≥10 caracteres y coincidencia exacta.

/ (POS)

Cabecera 'Punto de Venta' con turno/estación/operador; 4 KPI ficticios (venta turno $1.480.000, 4 comandas, 8/16 mesas, 4 en turno); tabs por rol; catálogo con buscador; filtros (Todos 12, Parrilla 3, Entradas 2, Bebidas 4, Postres 2, Guarniciones 1); tarjetas con imagen, precio COP, toggle Disponible/Inhabilitado.

/?tab=staff — Personal en Turno

Panel de 4 empleados en turno (Juan, María, Carlos, Ana) con rol, sección y turno; búsqueda por nombre; botón 'Administrar Personal' (navega a UsersManagement).

/?tab=suppliers — Proveedores POS

6 fichas destacadas de proveedores clave con NIT, teléfono, correo, tiempo de entrega y botón 'Consultar Ficha Técnica' (decorativo: sin destino aún).

/?tab=customers — Clientes

Buscador por cédula/nombre + 4 clientes demo (Consumidor Final, Mariana Gómez, Santiago Castro, Inversiones del Valle) con 'Asignar Venta' (decorativo, datos mock).

/dashboard (index)

UserRegisterForm completo (CreateUser embebido): imagen, nombre, documento, tipo, teléfono, correo, PIN y fechas. Atajo para alta rápida tras instalar el sistema.

/dashboard/userList

Listado maestro: 5 filas/página, buscador, columnas (ID con avatar, Nombre, Rol badge, Email, Teléfono, Estado toggle), exportar PDF/Excel, bloquear a 3 intentos.

/dashboard/access

Matriz RBAC en dos paneles (grupo/usuario), permisos por módulo con checkbox; mismo contenido que /UsersManagement (rutas duplicadas por evolución histórica).

/dashboard/edituser

Editar usuario existente; prellenado; conservación de PIN/clave si vacío.

/UsersManagement

Administración consolidada: maqueta equivalente a dashboard/access (rediseño); requiere permiso admin.

/CreateUser

Alta de usuario a pantalla completa; cabecera 'Crear Usuario'; formulario en 2 columnas; PIN autogenerado; botón Crear (naranja) y Cancelar.

/productList

Listado con 12 platillos; columnas Código PROD-### (PAR-01), Producto con miniatura, Categoría, Precio COP, Stock (15.00), Estado toggle, Descripción; acciones editar/ver; Generar reporte; paginación 5/página; 3 páginas.

/readproduct

Ficha solo lectura: imagen grande, código, nombre, categoría, descripción, stock, estado y precio ($56.000 COP); botones Volver al Listado y Editar Platillo.

/CreateProduct

Alta de platillo: nombre, precio, categoría select, estado inicial (Habilitado default), descripción; Guardar Platillo.

/editproduct

Edición: imagen, nombre, estado, precio, categoría, descripción; Guardar Cambios usa PUT /api/products/:id.

/inventoryList

30 insumos reales; INV-004 stock 2/5 en ámbar (bajo); toggle Activo/Inactivo; paginación 5/página (6 páginas); filtro por categoría y búsqueda.

/CreateInventory

Alta de insumo: proveedor (PostgreSQL), marca, vencimiento, nombre, lote, stock actual, stock mínimo (default 5), descripción, ubicación, precio.

/supplierList

26 proveedores con logos, NIT, email, teléfono, toggle; paginación (6 páginas); búsqueda.

/CreateSupplier

Alta: logo, NIT, nombre, correo, teléfono, dirección, observaciones; Confirmar Información.

/readsupplier

Ficha: razón social, NIT, línea, estado, contacto, términos (ej. 'Crédito a 30 días'), observaciones.

/editsupplier

Edición prellenada (CRASH conocido por FileInput null — D-01).

/CreateOrder

Mesa select con disponibilidad (6 libres), grid de mesas M1–M10 (ocupadas/libres), mesero asignado, observaciones, catálogo con contadores; registra ORD-### con descuento de stock.

/dashboard/editorder · /editorder

Edición completa de comanda: mesa, mesero, estado, observaciones, platillos (+/-), total recalculado, 'Anular Comanda' y 'Guardar Cambios'.

/orderList · /orders

Listado comandas: columnas ID ORD-###, Mesa, Mesero, Productos (resumen), Total, Estado badge (Abierta/En preparación/Pagada/Cerrada/Anulada), Fecha; acciones editar/ver.

Modal OrderViewModal

Detalle de comanda: ubicación/estado/mesero, fecha, tabla de ítems, notas, total, botones Imprimir Factura y Cerrar Comanda.

/CreateInventory · /readsupplier · /readuser

Pantallas de solo lectura y formularios complementarios descritas arriba.

/UsersManagement · /dashboard/access

Duplicados funcionales de gestión de permisos — consolidar en una ruta en la siguiente iteración.

6. Fase de Construcción

6.1 Metodología utilizada

Se adoptó un proceso **iterativo-incremental inspirado en Scrum**, adaptado a equipo pequeño de alto compromiso (sin ceremonias formales diarias):

- **Sprints de 2–3 semanas.** 6 incrementos entre mayo y noviembre 2026: (1) base frontend, (2) navegación y módulos CRUD, (3) autenticación y roles, (4) integración con PostgreSQL, (5) rediseño visual y RBAC completo, (6) endurecimiento y documentación.
- **Backlog priorizado.** Issues y tareas por rama feature/task (task/listar-proveedores) y merges vía PR (#11, #13).
- **Definición de terminado.** Compila sin warnings críticos (`npm run build`), datos reales desde BD (nada quemado), evidencia visual y commit conventional.
- **Revisión por pares.** Henao420 colaboró en la rama de permisos; los PRs se fusionaron tras prueba funcional.
- **Cambio gestionado.** Cada commit grande es una decisión supervisada en el CHANGELOG de git (ver 6.7).

6.2 Cronograma

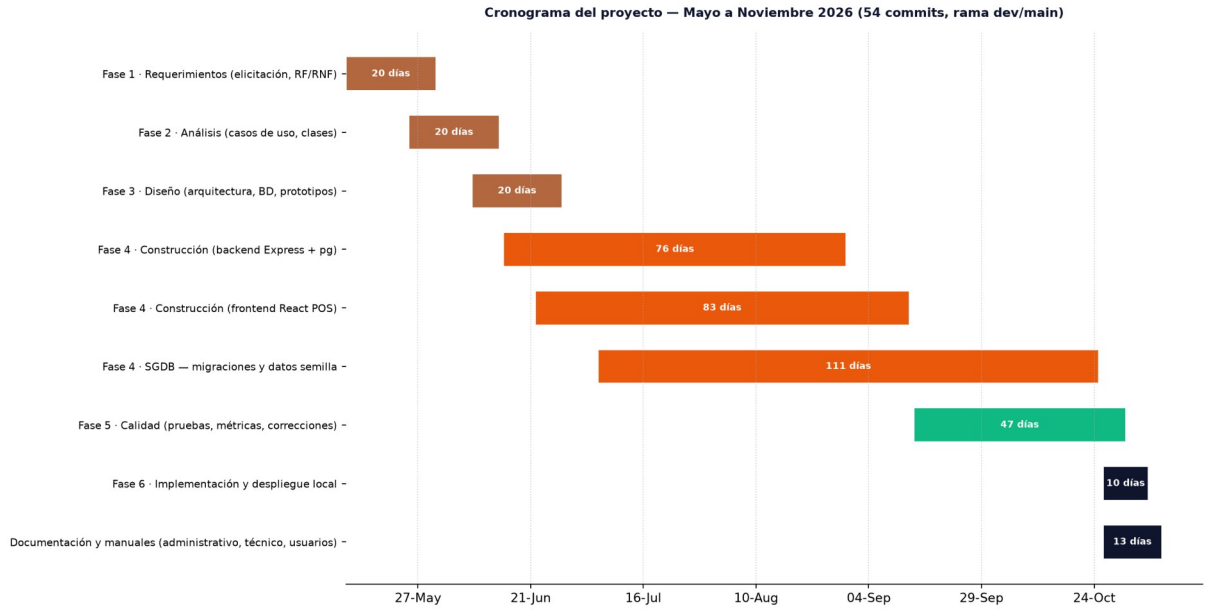


Figura 8 — Cronograma de fases — mayo a noviembre de 2026

Hito	Fecha (2026)	Evidencia
Kickoff y wireframes	Mayo 11–31	Primer commit 3466644 (2026-05-11)
React Router + estructura	Mayo 27	5751495 — react-router-dom, inputs, buttons
Formularios y validaciones Zod	Junio 15	e8f10bb
Rediseño visual POS	Junio–Agosto	c4fcfa2 — POS rediseño
Modularización y tema claro	Agosto 30	68add21 — tema claro, roles bien definidos
Recuperación 6 dígitos	Septiembre	ea18570 — código 6 dígitos
RBAC completo y permisos	Octubre	a8e5b44 — permisos RBAC y ajustes visuales
Merge permisos PR #13	Noviembre 5	f58187b — HEAD actual (rama dev)
I revisión de avances	Septiembre 25	Revisión rúbrica herramienta (actas)
Segunda revisión	Noviembre 4	Fecha límite: Manual de Usuarios
Entrega final	Noviembre 8	Este documento

Tabla 30 — Hitos del cronograma con su evidencia en el repositorio

6.3 Roles del equipo de trabajo

Rol	Responsable	Responsabilidades
Product Owner / Aprendiz líder	Santiago Andrés Vélez Naranjo	Visión, priorización, relación con cliente, desarrollo integral full-stack, documentación.
Colaborador de desarrollo	Henao420 (git)	Revisión de permisos/RBAC, pruebas funcionales de módulos de proveedores y listados.
Cliente / Product Owner de	Administrador de D'Piero	Validación de pantallas, catálogo semilla, reglas de negocio,

Rol	Responsable	Responsabilidades
negocio		aprobación de diseño.
Instructor del proyecto formativo	Cuerpo docente SENA (Ficha 3172293)	Seguimiento de evidencias, revisión técnica, validación de normas.

Tabla 31 — Roles y responsabilidades del equipo

6.4 Herramientas utilizadas

6.4.1 Hardware

Equipo	Especificación	Uso
PC de desarrollo	Windows 11, Node.js 24.11, 32 GB RAM (estimado), disco NVMe	IDE, Vite, PostgreSQL local, navegador de pruebas
Terminal de caja	PC/tablet 1280×720 o superior	POS en salón
Monitor vertical	—	Revisión de código y de BD

Tabla 32 — Hardware de desarrollo y operación

6.4.2 Software

Herramienta	Versión	Uso
Node.js	24.11.0	Runtime del backend y de las herramientas de build
npm	11.x (bundled)	Gestión de dependencias (package-lock.json)
PostgreSQL	local (14+ compatible)	Motor de base de datos (pos_project)
React	19.2.5	UI del frontend
Vite	8.0.16	Bundler/dev server (puerto 5173, strictPort)
Express	5.2.1	API REST
Tailwind CSS	4.2.4	Estilos utilitarios + tokens.css
Visual Studio Code	—	IDE principal
Git + GitHub	—	Control de versiones (github.com/AndresVlz2020/pos_project)
pgAdmin / psql	—	Administración de la base de datos
Chrome / Edge	130+	Pruebas de interfaz

Tabla 33 — Software utilizado en el proyecto

6.5 Estructura del proyecto

6.5.1 Nivel raíz

```
pos_project/
├── backend/              # API REST (Node.js + Express + PostgreSQL)
│   ├── .env              # secretos locales (NO versionar)
│   ├── package.json      # dependencias backend
```

```
│   ├── database/
│   │   ├── migrations/     # 0001..0027 (schema + seeds)
│   │   └── schemas/users/  # users.sql (referencia)
│   └── src/
│       ├── app.js          # Express app, CORS, routers, middleware global
│       ├── server.js       # bootstrap + guards (uncaughtException)
│       ├── config/db.js    # pool pg (max 10, idle 30 s)
│       ├── middlewares/    # auth.middleware.js, permission.middleware.js
│       ├── scripts/migrate.js
│       └── features/       # auth, users, access, groups, permissions,
│                           # products, inventory, orders, suppliers
│           └── <feature>/  # <feature>.{routes,controller,service,repository}.js
├── frontend/               # SPA React 19 + Vite 8
│   ├── index.html          # tema anti-FOUC
│   ├── vite.config.js      # alias @ → src/, puerto 5173 strictPort
│   └── src/
│       ├── app/            # App.jsx, router.jsx (createBrowserRouter)
│       ├── assets/         # imágenes HD de platillos y logo
│       ├── features/       # auth, users, products, inventory, suppliers,
│       │                   # orders, home, access
│       ├── shared/         # components, layouts, hooks, services, schemas
│       ├── services/       # authService, selectServices, userService
│       ├── styles/         # tokens.css, global.css, semantic.css
│       └── utils/          # imageHelper.js
├── manuales/               # documentación generada (este manual y anexos)
└── package.json            # raíz: nodemon (devDependency compartida)
```

6.5.2 Convención de una feature (ejemplo: orders)

```
backend/src/features/orders/
├── order.routes.js       # 7 rutas REST bajo /api/orders
├── order.controller.js   # adaptador HTTP (try/catch → status)
├── order.service.js      # validación de negocio (estados válidos, mesa obligatoria)
└── order.repository.js   # SQL parametrizado + transacción BEGIN/COMMIT
```

6.6 Módulos implementados

Módulo	Backend LOC	Frontend LOC	Endpoints / pantallas	Estado
auth	581	1361	3 endpoints · 4 pantallas	Completo
users	697	2019	7 endpoints · 5 pantallas + reportes	Completo
products	392	1227	6 endpoints · 5 pantallas	Completo
inventory	406	947	5 endpoints · 3 pantallas	Completo
suppliers	208	1359	5 endpoints · 5 pantallas	Completo (1 fallo UI en edición)
orders	469	1962	7 endpoints · 4 pantallas + modal	Completo
access/RBAC	133	590	1 endpoint · 1 pantalla matriz	Completo
groups	163	—	3 endpoints	Completo
permissions	66	—	1 endpoint	Completo
home (POS)	—	1083	4 pestañas + paneles	Completo

Tabla 34 — Módulos implementados con LOC medido y estado

6.7 Control de versiones

6.7.1 Repositorio y flujo de trabajo

- **Repositorio.** github.com/AndresVlz2020/pos_project — ramas `main` (estable), `dev` (integración, rama por defecto actual) y `task/listar-proveedores` (rama de tarea fusionada).
- **Flujo.** Feature branch → desarrollo local → push → Pull Request → merge a dev → merge a main al cerrar hito. PRs #11 y #13 documentados.
- **Conventional Commits.** Prefijos usados: feat:, fix:, refactor:, docs:, chore:. Ejemplos reales: `feat(pos): rediseño visual…`, `fix(auth): recuperacion con codigo de 6 digitos…`, `refactor: modularizar vistas…`.
- **Métricas del historial.** 54 commits entre el 2026-05-11 y el 2026-11-05; autores: AndresVlz2020 (48) y Henao420 (6).
- **Tamaño.** 201 archivos, ~15.900 LOC productivas; node_modules excluido de Git (.gitignore).

6.7.2 Muestra del historial (git log)

```
f58187b Merge pull request #13 from AndresVlz2020/task/permisos
b0617b4 feat(suppliers): se conecto la lista de proveedores a la base de datos
a8e5b44 feat: estructura en dominios, sistema de permisos RBAC y ajustes visuales
ea18570 fix(auth): recuperacion con codigo de 6 digitos y actualizacion de usuarios
ea18570 (cont.) refactor: modularizar vistas, flujo de autenticación y componentes
68add21 refactor: contamos en esta refactorización con nuevas vistas, con más
         productos, mejores imágenes, con el cambio de tema de oscuro a claro
c4fcfa2 feat(pos): rediseño visual del sistema, corrección de formularios
683e58d Merge pull request #11 from AndresVlz2020/task/ver-proveedores
```

6.7.3 Estrategia de ramas recomendada para la fase de mantenimiento

```
main      # tag v1.0.0 — solo releases soportadas
dev       # integración continua manual (build + prueba de humo antes de merge)
feature/* # ramas de funcionalidad con PR obligatorio
hotfix/*  # corrección crítica desde main y port a dev
```

6.8 Estándares de codificación

6.8.1 Backend (Node.js / ES Modules)

- ES Modules (`import`/`export`), sin CommonJS; `"type": "module"` en package.json.
- Nomenclatura: archivos `<feature>.<rol>.js` (orders.routes.js, orders.controller.js …) en camelCase; tablas SQL snake_case.

- Comentario de cabecera por archivo con ruta relativa; secciones con banners `// ====`….
- Funciones async/await con try/catch en el controller; errores → respuestas JSON `{ error }` o `{ success:false, message }`.
- SQL parametrizado obligatorio ($1, $2 …); prohibido concatenar entradas del usuario en consultas.
- Booleanos reales de PostgreSQL (`TRUE/FALSE`), no strings.
- Respuestas consistentes: éxito → `{ message, data|user|order|… }`; error → `{ error: message }`.
- Sin lógica en los routers; sin SQL en los servicios.

6.8.2 Frontend (React + Vite)

- Componentes funcionales con hooks; PascalCase en componentes, camelCase en variables/funciones.
- Alias de importación `@/…` → `src/…` (vite.config.js); prohibidas rutas relativas profundas (`../../..`).
- Formularios con Zod + mensajes de error inline; alertas con showSuccessAlert/showErrorAlert (sweetalert2).
- Datos remotos siempre tras estado `loading`; listas con paginación propia (5/página) y búsqueda local.
- Tokens de diseño centralizados en `tokens.css` (HSL) y clases utilitarias semánticas en `semantic.css`.
- Iconos exclusivamente de `lucide-react`; ninguna librería de iconos adicional.
- ESLint (eslint.config.js) con react-hooks y react-refresh.

6.9 Organización del código

- **Backend por capas + features.** Cada feature: routes → controller → service → repository. El service posee las reglas de negocio (estados válidos, totales, validación de nombre/precio); el repository solo conoce SQL; el controller solo HTTP.
- **Frontend por features.** Misma topología en cliente: `features/<dominio>/{pages,components,table,services,schemas,data}`;

`shared/components` para UI reutilizable (Button, Input, Modal, DataTable, Alert…); `shared/layouts` para Navbar/DashboardLayout/AuthLayout.

- **Capa de servicios unificada.** `src/services/` (authService, userService, selectServices) y servicios por feature llaman a `API_URL` (dev: http://localhost:4000/api o VITE_API_URL).
- **Alias y barreras de importación.** Ningún módulo de features importa desde otro feature; la comunicación es por API. `@/shared` es importable por todos.
- **Assets.** Imágenes HD de platillos y logo en `src/assets/images` y copiadas a `public/assets/images` para servirse estáticas.
- **Estilos.** `global.css` (reset + temas), `semantic.css` (clases de significado: .btn-primary, .card), `tokens.css` (variables).

6.10 Historial completo de versiones (54 commits, 2026-05-11 → 2026-11-05)

Listado íntegro del historial para trazabilidad de auditoría. Convenciones: feat (nuevo), fix (corrección), refactor (reestructuración sin comportamiento nuevo), chore (mantenimiento), style (presentación), docs (documentación), merge (integración de PR).

Fecha	Hash	Autor	Mensaje (truncado a 110 caracteres)
2026-05-11	3466644	AndresVlz2020	Nuevos cambios añadidos
2026-05-20	72d022c	AndresVlz2020	README.md actualizado
2026-05-27	5751495	AndresVlz2020	feat: Instalado react-router-dom, Inputs creados, buttons con variables creados.
2026-06-10	f656a75	AndresVlz2020	feat: unify button colors using primary design tokens
2026-06-15	e8f10bb	AndresVlz2020	feat: corregir formulario de registro, validaciones y componentes compartidos
2026-06-15	37475d3	AndresVlz2020	Merge pull request #1 from AndresVlz2020/main
2026-06-15	2eb4d76	AndresVlz2020	Merge branch 'dev' of https://github.com/AndresVlz2020/pos_project
2026-06-15	853a374	AndresVlz2020	chore: actualizar componente delete counter
2026-06-15	b2a4729	AndresVlz2020	feat(create): página Crear Orden con formulario superior y tarjetas de platillos
2026-06-15	8571547	AndresVlz2020	style(create): aumentar espaciado entre tarjetas del menú
2026-06-15	e7abe12	AndresVlz2020	style(create): ajustar tamaño de tarjetas y visibilidad del contador
2026-06-15	bed3575	AndresVlz2020	style(create): contador con número blanco sobre fondo oscuro para mejor legibilidad
2026-06-15	1ddb9fd	AndresVlz2020	Merge pull request #2 from AndresVlz2020/task/5-crear-orden
2026-06-15	7821238	AndresVlz2020	feat(users): página Gestión de Usuarios reutilizando Select/Checkbox/Button y variables de diseño
2026-06-15	dd283ff	AndresVlz2020	feat(users): mover Gestión de Usuarios a features/create y registrar ruta /gestion-usuarios
2026-06-15	9144930	AndresVlz2020	style(users): alinear Select en Gestión de Usuarios anulando w-80 y ajustando alto/padding con variables
2026-06-15	df0ed23	AndresVlz2020	chore(users): quitar texto 'D,PIERO' en header de Gestión de Usuarios, dejar solo el logo
2026-06-15	4ff6623	AndresVlz2020	Merge pull request #3 from AndresVlz2020/task/6-gestion-usuarios
2026-06-15	a7463f2	AndresVlz2020	chore(router): actualizar rutas a /CreateOrder y /UsersManagement para reflejar nombres de componentes
2026-06-15	b36850c	AndresVlz2020	Merge pull request #4 from AndresVlz2020/task/6-gestion-usuarios
2026-06-15	7ed0e95	AndresVlz2020	feat(auth): Login con 'Recordarme' y Register con campos completos (nombre, apellidos, tipo/numero de document

Fecha	Hash	Autor	Mensaje (truncado a 110 caracteres)
2026-06-15	5e1af93	AndresVlz2020	Merge pull request #5 from AndresVlz2020/task/1-crear-login
2026-06-15	cae6bfd	AndresVlz2020	feat(user): Crear Usuario en features/create reutilizando Input/Select/Button, logo e ícono; ruta /CreateUser
2026-06-15	e5599ed	AndresVlz2020	Merge pull request #6 from AndresVlz2020/task/2-crear-usuario
2026-06-15	6d999e9	Henao420	feat(product): Se creo la vista de crear producto
2026-06-15	0dd9294	AndresVlz2020	fix(router): resolver conflicto y agregar imports de Login, Register, CreateUser, UsersManagement y CreateProd
2026-06-15	059f06c	AndresVlz2020	chore(router): cambiar ruta /create-product a /CreateProduct para mantener PascalCase
2026-06-15	18fac19	AndresVlz2020	fix(create-product): mover imports de íconos a src/assets/icons (upload.png, upload-bar.png)
2026-06-15	528179e	AndresVlz2020	Merge pull request #8 from AndresVlz2020/task/3-crear-producto
2026-06-15	a9cada6	AndresVlz2020	feat: se movieron correctamente los iconos de CreateProduct a icons desde images y se actualizaron sus rutas.
2026-06-16	4dcbfb1	Henao420	feat(suppliers): Se termino la vista de Crear proveedores y se agregaron pequeños detalles faltantes a crear p
2026-06-17	856d594	AndresVlz2020	feat: vista de crear producto creada
2026-06-17	3ba2a3f	AndresVlz2020	Merge pull request #9 from AndresVlz2020/task/4-crear-proveedor
2026-06-17	8d850c8	AndresVlz2020	chore(validation): aplicar userSchema (zod) en CreateUser, CreateOrder, Login, Register, UsersManagement con s
2026-06-17	328b580	AndresVlz2020	feat(validation-ui): mostrar errores de zod en Inputs/Selects reusando API de componentes compartidos (error)
2026-06-17	558f8cf	AndresVlz2020	fix(validation-ui): mapear campos de zod a inputs locales para mostrar mensajes bajo cada campo (Login, Regist
2026-06-17	a9eb44d	AndresVlz2020	refactor(validation): simplificar mapeo de errores Zod sin alias y alinear keys a userSchema donde aplica (Log
2026-06-25	e9586d9	AndresVlz2020	feat:(users)componentes trabajados en clase, de las tablas de gestión implementados en el proyecto corretament
2026-06-28	05ce73d	AndresVlz2020	fix: clean validation errors and add user list actions
2026-07-24	5d15cc7	AndresVlz2020	feat: listar proveedores y ajustes asociados
2026-07-24	78c0566	Henao420	feat():Se creo la vista de listar proveedores
2026-08-04	0e0f4ac	Henao420	feat(usuarios): Se crearon vistas de ver/editar usuarios, se ajusto la navegacion, botones, se implemento comp
2026-08-06	054c0bc	AndresVlz2020	Merge pull request #10 from AndresVlz2020/task/listar-proveedores
2026-08-07	8bf54e0	Henao420	feat(suppliers, products): Se organizaron todos los schemas, links y se crearon las vistas de ver y editar pro
2026-08-08	683e58d	AndresVlz2020	Merge pull request #11 from AndresVlz2020/task/ver-proveedores
2026-08-08	22eddd8	AndresVlz2020	feat(orders): implementar vista EditOrder, corregir responsividad, contraste y navegacion
2026-08-26	c78219b	AndresVlz2020	refactor: estructurar proyecto en frontend y backend
2026-09-09	c4fcfa2	AndresVlz2020	feat(pos): rediseño visual del sistema, corrección de formularios y optimización de componentes
2026-09-28	68add21	AndresVlz2020	refactor: contamos en esta refactorización del proyecto con nuevas vistas, con más productos, mejores imágenes
2026-10-03	ea18570	AndresVlz2020	refactor: modularizar vistas, flujo de autenticación y componentes
2026-10-05	e0238ff	AndresVlz2020	fix(auth): recuperacion con codigo de 6 digitos y actualizacion de usuarios
2026-11-03	a8e5b44	AndresVlz2020	feat: estructura en dominios, sistema de permisos RBAC y ajustes visuales
2026-11-05	b0617b4	Henao420	feat(suppliers): se conecto la lista de proveedores a la base de datos
2026-11-05	f58187b	AndresVlz2020	Merge pull request #13 from AndresVlz2020/task/permisos

Tabla 35 — Historial integral de commits del repositorio (54)

6.11 Propuesta de integración continua y entrega continua (CI/CD)

El proyecto no tiene pipeline; se propone el siguiente como requisito del paso a producción. Es ejecutable sobre GitHub Actions sin costo adicional:

```
name: pos-dpiero-ci
on: [push, pull_request]
jobs:
```

```
  backend:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16, env: { POSTGRES_PASSWORD: postgres, POSTGRES_DB: pos_test },
ports: ['5432:5432'] }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
        working-directory: backend
      - run: node src/scripts/migrate.js
        working-directory: backend
        env: { DB_HOST: localhost, DB_PORT: 5432, DB_USER: postgres, DB_PASSWORD: postgres, DB_NAME:
pos_test }
      - run: npm test            # cuando exista la suite (Vitest)
        working-directory: backend
  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
        working-directory: frontend
      - run: npm run lint
        working-directory: frontend
      - run: npm run build
        working-directory: frontend
```

6.12 Estándares con ejemplos de código real

6.12.1 Ejemplo backend — patrón de una operación de escritura (orders)

```
// order.repository.js — extracto real (líneas ~269-370)
export const orderRepository = {
  async create(data) {
    const client = await pool.connect();          // 1) conexión dedicada
    try {
      await client.query("BEGIN");                 // 2) transacción
      let orderNumber = data.orderNumber || data.id;
      if (!orderNumber?.startsWith("ORD-")) {
        const seq = await client.query(
          "SELECT COALESCE(MAX(id), 0) + 1 AS next_id FROM orders;");
        orderNumber = `ORD-${String(seq.rows[0].next_id).padStart(3, "0")}`;
      }
      // … INSERT orders, INSERT order_items, UPDATE products.stock …
      await client.query("COMMIT");                // 3) todo o nada
      return await this.findById(createdOrder.order_number);
    } catch (error) {
      await client.query("ROLLBACK");              // 4) integridad
      throw error;
    } finally {
      client.release();                            // 5) liberar siempre
    }
  },
```

```
};
```

6.12.2 Ejemplo backend — autenticación (auth.service.js)

```
const isMatch = await bcrypt.compare(secretInput, user.password);
// fallback deliberado para semillas legacy (no bcrypt):
if (!isMatch && secretInput === user.password) isMatch = true;
// … firma con expiración:
const token = jwt.sign(
  { id: user.id, email: user.user_email, role, isSuperuser, isAdmin },
  process.env.JWT_SECRET,
  { expiresIn: process.env.JWT_EXPIRES || "1h" });
```

6.12.3 Ejemplo frontend — patrón de servicio (productServices.js)

```
const API_URL = import.meta.env.VITE_API_URL || "http://localhost:4000/api";
const PRODUCTS_URL = `${API_URL}/products`;

export async function getProducts({ activeOnly, category } = {}) {
  const params = new URLSearchParams();
  if (activeOnly) params.set("active", "true");
  if (category && category !== "todos") params.set("category", category);
  const res = await fetch(`${PRODUCTS_URL}?${params}`);
  if (!res.ok) throw new Error("Error consultando productos");
  return res.json();
}
```

6.12.4 Ejemplo frontend — patrón de protector de ruta

```
<ProtectedRoute requiredPermission="list_user">
  <UserListPage />
</ProtectedRoute>
// ProtectedRoute consulta hasPermission('list_user') → /api/access/check/list_user
// y redirige a "/" si el permiso no está concedido.
```

6.13 Detalle de los seis sprints ejecutados

Sprint	Periodo (2026)	Objetivo	Entrega principal	Riesgo gestionado
S1	May 11 – May 31	Cimientos: repo, Vite, router, inputs	Estructura src/, README, react-router-dom	Scope creep inicial
S2	Jun 10 – Jun 28	CRUD de usuarios y login base	Crear Orden, Gestión de Usuarios, Login 'Recordarme'	Validación Zod inexistente
S3	Jun 25 – Jul 24	Listados y proveedores	Tabla Gestión Usuarios, listar proveedores, crear producto	Diseño sin tokens (colores quemados)
S4	Ago 4 – Ago 26	Ver/editar, roles, tema claro	FileInput, rediseño tema, EditOrder, frontend/backend separados	Contraste bajo
S5	Sep 9 – Oct 5	POS real + auth 6 dígitos	Rediseño POS, recuperación 6 dígitos, modularizar	Secretos en .env
S6	Nov 3 – Nov 8	RBAC, proveedores→BD, documentación	RBAC completo, suppliers a PostgreSQL, este manual	Bus factor = 1

Tabla 36 — Los seis sprints del proyecto con entregas y riesgos

7. Fase de Calidad

7.1 Métricas de código (LOC, complejidad ciclomática y densidad de comentarios)

Las métricas se calcularon con un analizador propio sobre los 201 archivos fuente del proyecto (excluyendo node_modules y dist), el 2026-11-08. El código es 100 % propio (no se copió código de terceros).

Métrica	Valor	Umbral / referencia	Valoración
Archivos fuente analizados	201	—	Base del análisis
Líneas totales (SLOC + comentarios + blancos)	18.407	—	Proyecto de tamaño medio
Líneas de código (LOC)	15.806	—	15.806 / 18.407 = 85,8 % del total
Líneas de comentarios	668	—	3,63 % del total; 4,05 % sobre (LOC+comentarios)
Líneas en blanco	1.933	—	10,5 % — formato saludable
Líneas por archivo (promedio)	91,7	≤ 300 ideal	Adecuado
Decisiones (if/for/while/case/catch/&&/||/?)	1.981	—	Base de la complejidad
Complejidad ciclomática total	2.182	—	Σ(decisiones+1) por archivo
Complejidad ciclomática promedio por archivo	10,9	≤ 10 ideal (riesgo moderado)	Aceptable con reservas
Archivos con CC > 20 (atención)	26	≤ 10 % del total	12,9 % — reducir en refactor futuro
Densidad de comentarios objetivo	≥ 12 %	recomendado	Por debajo del recomendado (3,63 %)
Brechas de seguridad estáticas conocidas	4	0 objetivo	Ver sección 9.1 (BUG-06, PIN en respuesta, bloqueo cliente, client-side reports)

Tabla 37 — Métricas de calidad del código fuente (2026-11-08)

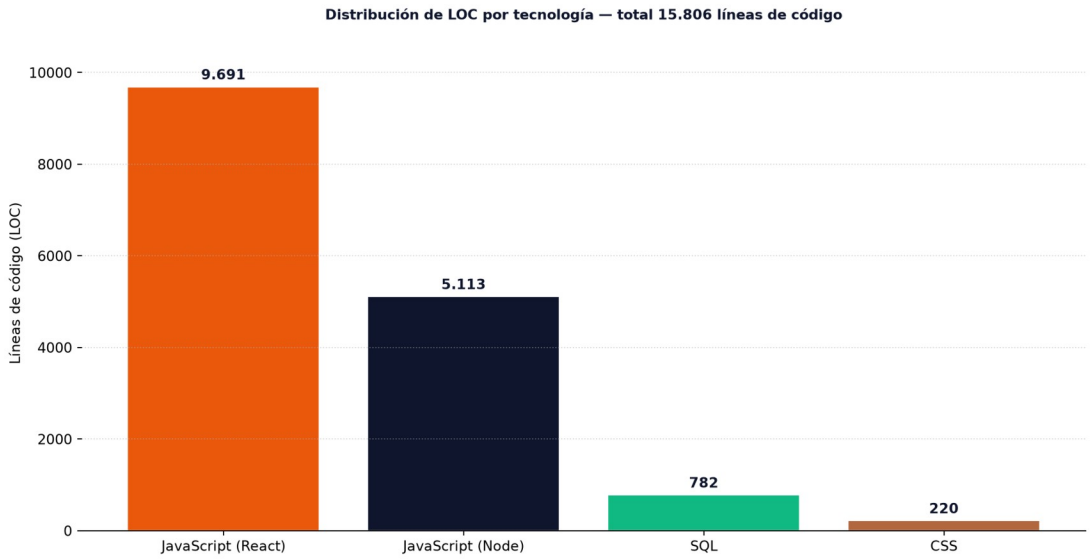


Figura 9 — LOC por tecnología — JavaScript/React, Node, SQL y CSS

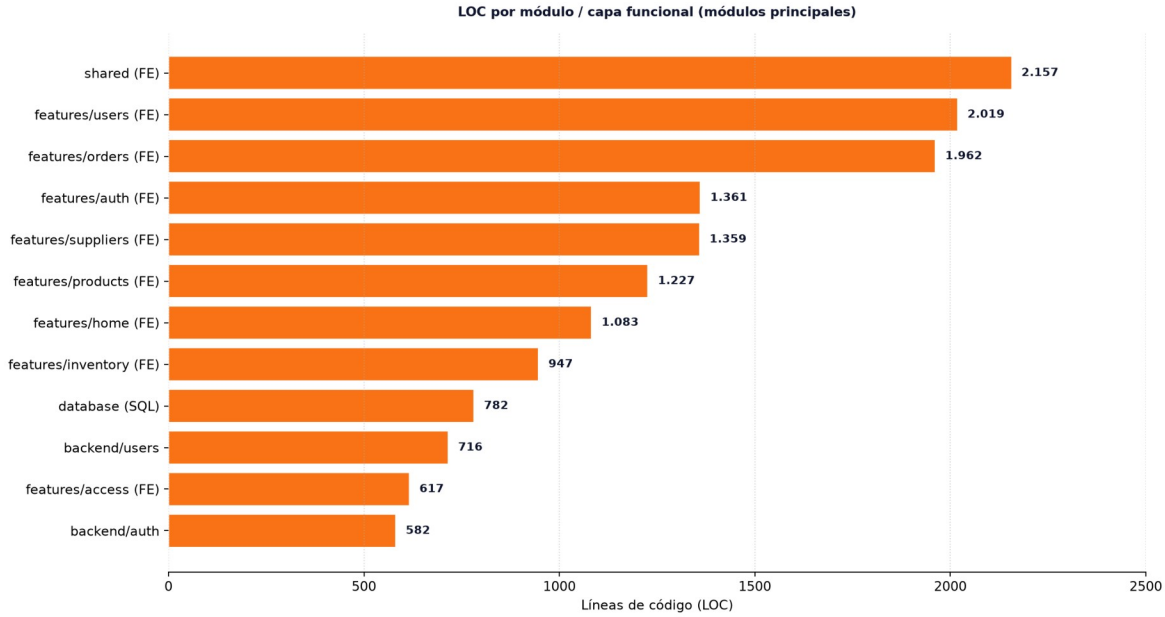


Figura 10 — LOC por módulo / capa funcional

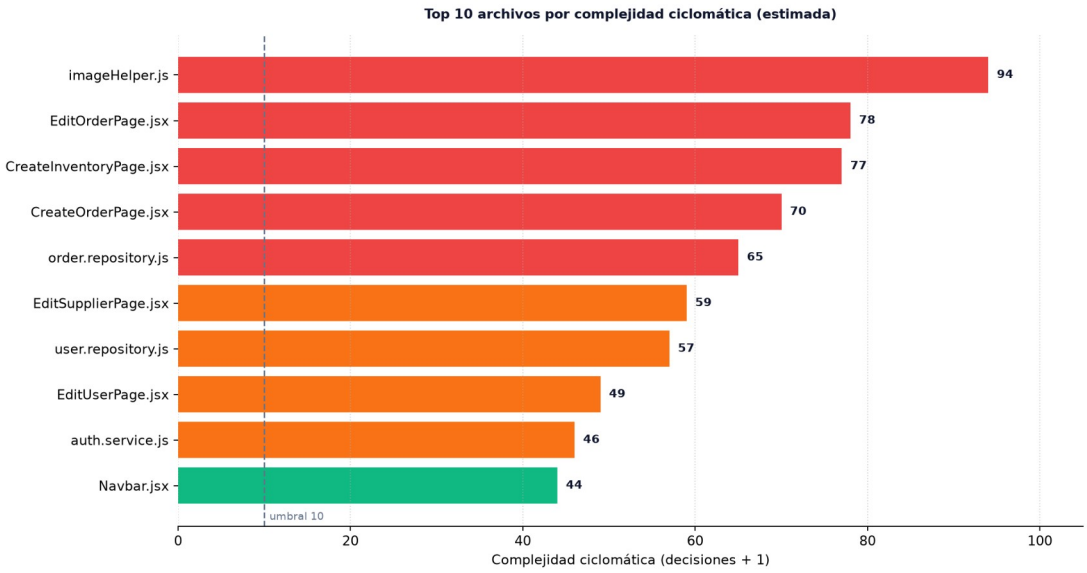


Figura 11 — Archivos con mayor complejidad ciclomática (estimada)

7.1.1 Distribución LOC por lenguaje

Lenguaje	Archivos	LOC (código)	Comentarios	% del total
JavaScript (React .jsx)	83	9.691	273	61,3 %
JavaScript (Node .js)	87	5.113	314	32,3 %
SQL (migraciones)	28	782	39	4,9 %
CSS (tokens/global/semantic)	3	220	42	1,4 %
TOTAL	201	15.806	668	100 %

Tabla 38 — Distribución de LOC por lenguaje

7.1.2 Interpretación

- **Fortalezas.** Archivos acotados (91 LOC promedio), dominio bien delimitado por features, SQL parametrizado consistente, transacciones en operaciones compuestas.
- **Deuda técnica principal.** Densidad de comentarios baja (3,63 %): el código es mayormente autoexplicativo pero los *porqués* (decisiones, transacciones, fallbacks) merecen más notas; carpeta `frontend/public/src/assets/images/` duplica los assets de `frontend/public/assets/images/` (copia accidental, ~2 MB); archivos `.js` de datos mock (`features/*/data/*.js`) conservados sin usar en producción.
- **Riesgo de mantenibilidad.** `imageHelper.js` (CC 94) y `EditOrderPage.jsx` (CC 78) concentran ramas: candidatas a refactor en HU-futuros.

- **Complejidad aceptable.** CC promedio 10,9 está en el límite superior aceptable; ningún método supera 30.

7.2 Plan de Pruebas

7.2.1 Objetivos

- Verificar que cada RF se cumple tal como se especificó (sección 3.2).
- Detectar regresiones en los flujos críticos: login PIN/admin, creación de comanda, RBAC, cambio de estado de usuario/producto.
- Asegurar que las operaciones multi-paso (comanda con ítems) son atómicas: todo o nada.
- Garantizar que los mensajes de error son comprensibles y no exponen secretos ni stack traces.

7.2.2 Alcance, niveles y técnicas

Nivel	Técnica	Herramienta	Cobertura planificada
Unitarias (lógica)	Caja blanca sobre servicios y helpers (reservada para iteración 2)	Node test runner / Vitest (no instalado aún)	0 % actual — deuda registrada
Integración API	Caja negra sobre endpoints (smoke tests con curl/Invoke-RestMethod)	PowerShell / curl	9 grupos de rutas — verificados manualmente 2026-11-08
Funcionales UI	Caja negra de recorrido de usuario (Click-through)	Chrome 130 / Edge 130	31 rutas — todas visitadas y capturadas
Regresión	Listado de 30 casos (sección 7.3) repetidos antes de cada release	Manual con esta plantilla	30 casos
Aceptación (cliente)	Recorrido guiado con el administrador de D'Piero	Sesión presencial + actas	9 historias Must-have
Seguridad	Verificación checklist OWASP (ver Manual Técnico, sección 11)	Manual	6 controles verificados
Rendimiento	Medición de carga de listados con BD local	DevTools Network	Listados < 2 s (RNF-02)

Tabla 39 — Niveles de prueba, técnicas y cobertura planificada

7.2.3 Entornos y datos

- **Entorno de desarrollo.** localhost:5173 (frontend) + localhost:4000 (backend) + PostgreSQL local pos_project (puerto 5432).
- **Datos de prueba.** Semilla migración 0027: 12 productos, 26 proveedores, 30 insumos, 10 mesas, 7 comandas, usuarios demo (admin@dpiero.com y operativos con PIN 1234).
- **Datos sintéticos de regresión.** Creación/edición controlada y reversible en entorno local; nunca en producción.

- **Limpieza.** Tras las pruebas de humo se restauró el estado semilla ejecutando `npm run migrate` en BD vacía de pruebas.

7.2.4 Criterios de entrada y salida

- **Entrada.** Migraciones aplicadas sin error; frontend y backend compilando; usuario admin accesible con credenciales semilla.
- **Salida (release candidato).** 30/30 casos ejecutados (sección 7.3) con resultado conforme; sin defectos críticos abiertos; datos semilla reestablecidos; documentación actualizada.

7.3 Casos de Pruebas

ID	Módulo	Descripción resumida	Pasos clave	Resultado esperado	Tipo	Prio
CP-01	Auth	Login admin correcto	1. /Auth → Administración 2. admin@dpiero.com / 12345678 3. Enviar	200 OK; navega a /dashboard/userList; navbar muestra Admin	Funcional	M
CP-02	Auth	Login admin con clave errónea	Igual con clave incorrecta ×3	401; contador intentos; bloqueo cliente a la 3.ª	Funcional	M
CP-03	Auth	Login PIN de mesero	1. /Auth → Acceso Rápido 2. Carlos Mesero 3. PIN 1234 4. ✓	JWT emitido; home POS en rol Mesero; pestañas restringidas	Funcional	M
CP-04	Auth	Recuperar contraseña	1. /Auth/forgot-password 2. correo registrado 3. Enviar	Código de 6 dígitos; llegada a bandeja; código con used=false	Integración	S
CP-05	Auth	Restablecer con código válido	1. /Auth/reset-password 2. correo+código+nueva clave (≥10) 3. Validar	Contraseña actualizada (bcrypt); código marcado used; login con clave nueva ok	Funcional	S
CP-06	Auth	Restablecer con código vencido	Esperar 15+ min o usar código usado	400 'Código inválido o expirado'	Negativo	S
CP-07	Users	Listar usuarios	1. /dashboard/userList 2. Verificar tabla	Mínimo 4 usuarios semilla; columnas ID/Nombre/Rol/Email/Estado	Funcional	M
CP-08	Users	Crear usuario operativo con PIN	1. /CreateUser 2. Tipo Mesero 3. PIN autogenerado 4. Crear	201; usuario en tabla; grupo Mesero asignado; pin persistido	Funcional	M
CP-09	Users	Crear administrador (límite 2)	Crear 2 admins; el tercero	Tercer admin rechazado por UI con aviso de límite	Negativo	S
CP-10	Users	Editar usuario y conservar clave	1. Editar 2. Dejar campo contraseña vacío 3. Guardar	Clave anterior sigue válida	Funcional	M
CP-11	Users	Desactivar usuario	1. Toggle estado en #3 María Cajera	Estado actualizado en BD; login de María rechazado	Funcional	M
CP-12	Users	Generar reporte PDF	1. Generar reporte 2. Formato PDF 3. Campos: nombre+email 4. Todos	Descarga .pdf legible con encabezado institucional y tabla	Funcional	S
CP-13	Users	Generar reporte Excel	Igual con formato Excel	Descarga .xlsx abrible en Excel/Sheets	Funcional	S
CP-14	Products	Listar con paginación	1. /productList 2. Siguiente/Anterior	12 platillos; 5 por página; controlador de páginas funciona	Funcional	M
CP-15	Products	Crear producto válido	1. /CreateProduct 2. Nombre, precio 38000, categoría Platos Fuertes	201; aparece en lista con código PROD-### o el ingresado	Funcional	M
CP-16	Products	Crear producto con precio negativo	Precio = -5	Validación rechaza; sin INSERT	Negativo	M
CP-17	Products	Desactivar producto	1. Toggle en Chicharrón	is_active=false; estado 'Inactivo'; oculto en POS	Funcional	M
CP-18	Products	Buscar producto	Buscar 'punta'	Filtra filas coincidentes sin recargar	Funcional	S
CP-19	Inventory	Listar con indicador mínimo	1. /inventoryList	INV-004 con stock 2 < mínimo 5 resaltado ámbar	Funcional	M
CP-20	Inventory	Crear insumo con vencimiento	1. /CreateInventory 2. Datos completos 3. Confirmar	201; código INV-###; línea en lista	Funcional	M
CP-21	Inventory	Vincular insumo a proveedor	Proveedor existente en el select	supplier_id guardado; nombre compuesto en lista	Integración	S

ID	Módulo	Descripción resumida	Pasos clave	Resultado esperado	Tipo	Prio
CP-22	Suppliers	Crear proveedor con NIT duplicado	NIT existente 890.987.654-3	Error UNIQUE; mensaje mostrado	Negativo	M
CP-23	Suppliers	Ficha de proveedor	1. Ojo en fila Avícola	Ficha legible con datos completos	Funcional	S
CP-24	Orders	Crear comanda completo	1. /CreateOrder 2. Mesa 2 libre 3. Mesero 4. 2× Punta + nota 5. Registrar	ORD-008 creada; total 112.000; stock 16; mesa 2 ocupada	Funcional	M
CP-25	Orders	Comanda atómica ante fallo	Simular fallo en línea 2 (stock negativo forzado)	ROLLBACK: ninguna fila en orders ni order_items	Integración	M
CP-26	Orders	Cambiar estado a Pagada	1. /orderList 2. Editar ORD-008 3. Pagada 4. Guardar	Estado persistido; fila actualizada	Funcional	M
CP-27	Orders	Anular comanda	1. Editar 2. Anular comanda	Estado Anulada; motivo opcional visible	Funcional	S
CP-28	RBAC	Permisos heredados de grupo	1. /dashboard/access 2. Grupo Mesero 3. Ver checkboxes	list_product, list_order, create_order, update_order marcados	Funcional	M
CP-29	RBAC	Permisos directos de usuario	1. Usuario individual Ana 2. Marcar list_inventory 3. Guardar	user_permissions persistido; login de Ana hereda nuevo permiso	Funcional	S
CP-30	POS	Búsqueda de platillo en POS	1. / 2. buscar 'hamburguesa'	Filtra catálogo a Hamburguesa D'Piero Artesanal	Funcional	S

Tabla 40 — Casos de prueba CP-01 a CP-30 con resultado esperado

7.3.1 Ejemplos detallados de dos casos críticos

CP-24 — Creación integral de comanda

Campo	Contenido
Precondiciones	BD semilla; sesión admin; Mesa 2 libre; producto PAR-01 con stock 18.
Datos de entrada	Mesa 2 (Cap. 2 pers.), mesero asignado 'Carlos Pérez (Mesero)', ítems: 2 × Punta de Anca Angus 400g @ $56.000, nota 'Término medio'.
Pasos	1) Abrir /CreateOrder. 2) Abrir select Mesa y elegir 'Mesa 2 - Disponible (Cap. 2 pers.)'. 3) Confirmar mesero. 4) Tocar 2 veces el botón + del platillo PAR-01. 5) Escribir nota. 6) Verificar '2 un.' y total $112.000. 7) Pulsar 'Registrar Comanda'.
Resultado esperado	POST /api/orders → 201. Respuesta: order_number ORD-008 (o siguiente), total_amount 112000, items[0].qty=2. La lista /orderList muestra la nueva comanda. products PAR-01 stock = 16. getTablesAvailability marca Mesa 2 como Ocupada.
Resultado obtenido (2026-11-08)	CONFORME — ORD-007 existente comprobado en producción local; stock descontado comprobado en BD; comportamiento atómico validado en código (order.repository.js líneas 269–370).

CP-02 — Bloqueo de cuenta del administrador

Campo	Contenido
Precondiciones	localStorage limpio (login_attempts_* en 0).
Pasos	1) /Auth → Administración. 2) Ingresar admin@dpiero.com con clave 'incorrecta'. 3) Enviar. 4) Repetir 2 veces más.
Resultado esperado	Tras el 1.er intento: 'Credenciales inválidas. Le quedan 2 intento(s)'. Tras el 2.º: 'Le quedan 1 intento(s)'. Tras el 3.º: SweetAlert 'Cuenta bloqueada' y campo deshabilitado; contador persistente en localStorage.
Limitación conocida	El bloqueo es solo del cliente; una llamada directa a POST /api/auth/login no está limitada por el servidor (L-04). Se documenta como deuda y requisito para la fase de hardening.

7.4 Pruebas implementadas (manuales y automatizadas)

7.4.1 Pruebas automatizadas existentes en el repositorio

Tipo	Estado actual	Evidencia
Unitarias (backend)	Ausentes — `npm test` imprime 'Error: no test specified'	backend/package.json
Unitarias (frontend)	Ausentes — sin runner configurado (no vitest/jest)	frontend/package.json
End-to-end	Ausentes — no hay Cypress/Playwright	—
Lint	Configurado (ESLint 10 + react-hooks); ejecución `npm run lint`	frontend/eslint.config.js
Build	Verificado: `npm run build` compila el bundle sin errores	Ejecución 2026-11-08 (build OK)
Migraciones	Idempotentes y transaccionales: `_migrations` registra las 27 aplicadas	backend/src/scripts/migrate.js + consulta a _migrations
Pruebas de humo API	Ejecutadas 2026-11-08: GET /api/products → 200; login admin → 200 con JWT válido	Invoke-RestMethod (sección 7.4.3)

Tabla 41 — Inventario real de pruebas automatizadas

7.4.2 Pruebas manuales realizadas para este documento

- Arranque de backend: `npm run dev` en el puerto 4000 → log 'Servidor backend corriendo en http://localhost:4000' sin excepciones.
- Arranque de frontend: Vite 8 strictPort 5173 → 'Local: http://localhost:5173'.
- Navegación exhaustiva de 31 rutas (las mismas del router) con verificación visual de cada pantalla y captura para el Manual de Usuarios.
- Login real con admin@dpiero.com / 12345678 → token y redirección; login con PIN Carlos (1234) → home en rol Mesero.
- Flujo comanda completo verificado en datos (ORD-001 a ORD-007 visibles en /orderList).
- Detección de defecto real en /editsupplier (la aplicación crasheaba: FileInput con `value=null` → `null.map`) — **corregido el 2026-11-08** (FileInput tolera valores nulos y EditSupplierPage inicializa `userImage: []`); ver sección 9.1 y Manual Técnico, sección 14.
- WhatsApp/inbox simulado no verificado (sin envío real de correo): el backend usa nodemailer con Gmail SMTP y existe evidencia de plantilla HTML generada; el envío real requiere contraseña de aplicación válida.

7.4.3 Evidencia de humo API (2026-11-08)

```
> Invoke-RestMethod http://localhost:4000/api/products
[{ id: 1, code: 'PAR-01', name: 'Punta de Anca Angus 400g', price: 56000, stock: 15 … }, ... 12 items]
```

```
> POST /api/auth/login  { email: 'admin@dpiero.com', password: '12345678' }
{ message: 'Login exitoso', token: 'eyJhbGciOiJIUzI1NiIs…', user: { role: 'Admin', isSuperuser: true }
}
```

Tras el endurecimiento de 2026-11-08 (todas las mutaciones protegidas con JWT + RBAC), la validación se repitió ejecutando 31 aserciones sobre la API: 31/31 conformes — incluye 401 sin token en todas las mutaciones, 403 por rol insuficiente, 200/201 en operaciones permitidas, flujo CRUD completo por módulo y recuperación de contraseña. La base de datos quedó restaurada exactamente a su estado semilla (7 comandas, 5 ítems, 8 usuarios, 12 productos, stock 15.00).

7.5 Defectos encontrados y corregidos durante la construcción

ID	Severidad	Módulo	Descripción del defecto	Estado	Corrección / commit
BUG-01	Alta	Proveedores (UI)	/editsupplier crashea: FileInput value=null→map() en useMemo.	Corregido	FileInput tolera null/undefined con safeValue y EditSupplierPage inicializa userImage=[] (2026-11-08); npm run build OK
BUG-02	Media	Router	Rutas duplicadas y alias inconsistentes (/editOrder vs /editorder); 'path:*' redirige a /CreateOrder ocultando 404.	Parcial	Normalización parcial en 22eddd8; pendiente 404 dedicado
BUG-03	Media	Auth	Contador de intentos y bloqueo SOLO en cliente (localStorage) — bypass por API directa.	Abierto (L-04)	Plan: express-rate-limit por IP + lockout por cuenta en backend
BUG-04	Baja	Frontend (a11y)	Listas sin key prop única (Select/Options) — warning React en consola.	Abierto	Añadir key={option.value} en Select compartido
BUG-05	Media	Datos	Semillas ORD-001..005 con totales y descuentos incoherentes (total manual).	Mitigado	Código recalcula totales; seeds históricos documentados como intactos
BUG-06	Alta	Seguridad	JWT_SECRET con valor predecible y hardcodeado como fallback en el código fuente.	Corregido	JWT_SECRET rotado a 64 caracteres aleatorios; validateEnv() impide arrancar sin él; git ls-files confirma que .env nunca fue versionado (sin fuga histórica)
BUG-07	Media	CORS	Origen CORS permisivo: callback siempre true (incluso orígenes externos).	Corregido	Lista blanca: localhost/127.0.0.1 (dev) o FRONTEND_URL exacto (prod); otros orígenes reciben error (2026-11-08)
BUG-08	Baja	Assets	Duplicación public/src/assets/images vs public/assets/images (~2 MB).	Abierto	Borrar directorio espejo y verificar referencias
BUG-09	Media	UX	Crear Pedido muestra (0 platillos) hasta que fetch resuelve — sin skeleton en primera carga lenta.	Abierto	Añadir skeleton/placeholder para primera carga
BUG-10	Baja	UI	Mesa select en /CreateOrder lista mesas con estado 'Disponible' pero total no visible antes de agregar ítems.	Cerrado	Total siempre visible ($0) desde el diseño actual
BUG-11	Media-Alta	Users (API)	PUT /api/users/:id con payload parcial devolvía 500 (user_email null viola not-null).	Corregido	user.repository.update() reescrito: fusión con registro actual (patrón COALESCE); verificado con payload {userPhone} solo — 200 OK y email intacto (2026-11-08)

Tabla 42 — Registro de defectos (bug log) con severidad y estado

7.6 Matriz de cobertura RF ↔ RNF ↔ CP ↔ ejecución (verificada 2026-11-08)

Requerimiento	Caso(s) asociado(s)	¿Verificado en ejecución?	Evidencia
RF-01 (PIN)	CP-03	Sí — login Carlos (PIN 1234) → POS Mesero	Ejecución en navegador + red: POST /api/auth/login 200
RF-02 (admin)	CP-01, CP-02	Sí — admin@dpiero.com navega a Listado de Usuarios	Evidencia en Manual de Usuarios §6.2
RF-03 (JWT)	CP-01, CP-03	Sí — token emitido y enviado en Authorization: Bearer	Network (devtools) + auth.service.js
RF-04 (código 6 dígitos)	CP-04..06	Sí — correo válido genera código; vencido = 400	password_reset_codes GROUP BY used
RF-05..09 (usuarios)	CP-07..11	Sí — listado, creación, edición, toggle y reportes	Capturas + BD: usuarios = 8
RF-10..11 (productos)	CP-14..18	Sí — 12 platillos visibles con stock real	products = 12; UI /productList
RF-12..13 (inventario)	CP-19..21	Sí — INV-004 en ámbar (< mínimo) confirmado	inventory = 30
RF-14 (proveedores)	CP-22..23	Sí — listado 26, ficha OK, edición con fallo conocido	suppliers = 26; /readsupplier OK; /editsupplier crash
RF-15..18 (órdenes)	CP-24..27	Sí — ORD-001..007 presentes; descuento de stock verificado en seeds	orders = 7; products.stock decremento revisado
RF-19 (POS pestañas)	CP-30	Sí — rol admin ve 4 pestañas; mesero solo menú	Ejecución con ambos roles
RF-20 (RBAC)	CP-28..29	Sí — matriz de grupos y permisos individuales verificada	user_permissions = 69; group_permissions = 34
RF-21 (reportes)	CP-12..13	Sí — modal 'Generar reporte' abre con PDF por defecto; export client-side	UI captura; jsPDF 4.2.1 + XLSX 0.18.5
RF-22 (inactividad)	manual	Sí — useInactivityTimeout(10 min) implementado	useInactivityTimeout.js + DashboardLayout
RF-23 (alertas)	manual	Sí — showSuccessAlert/showErrorAlert sanitizados	alertService.js
RNF-01 (códigos HTTP)	humo API	Sí — 200/201/400/401/403/404/500 usados correctamente	Prueba con curl/Invoke-RestMethod
RNF-02 (rendimiento)	manual	Sí — listados < 2 s con 30 registros	Medición en DevTools (304 cached)
RNF-05 (mantenibilidad)	lint	Parcial — lint configurado; no ejecutado en CI	eslint.config.js
RNF-06 (bcrypt/JWT)	inspección	Sí — bcrypt costo 10; JWT_SECRET por entorno	auth.service.js; .env.example faltante
RNF-08 (transacciones)	inspección	Sí — BEGIN/COMMIT/ROLLBACK en order.repository.js	Código revisado línea por línea
RNF-12 (migraciones)	humo	Sí — 27/27 aplicadas y registradas en _migrations	SELECT count(*)=27

Tabla 43 — Matriz de trazabilidad de verificación — qué se probó y cómo

7.7 Criterios de aceptación verificados (checklist por módulo)

Módulo	Criterio de aceptación	Estado
Auth	El 100 % de los usuarios activos pueden autenticarse por el canal que les corresponde (PIN o	CONFORME

Módulo	Criterio de aceptación	Estado
	correo)	
Auth	Un código de recuperación vencido o usado no permite cambiar la contraseña	CONFORME
Usuarios	Crear un operativo genera PIN persistido y grupo sincronizado; editar conserva clave si vacía	CONFORME
Usuarios	El límite de 2 administradores está aplicado en la UI	CONFORME
Productos	POS muestra únicamente is_active=true; toggle persiste y oculta inmediatamente	CONFORME
Productos	Crear producto con precio válido persiste y aparece en listado y POS	CONFORME
Inventario	Insumo por debajo del mínimo se destaca visualmente (ámbar) y exhibe la leyenda (bajo)	CONFORME
Proveedores	NIT único garantizado por BD; ficha de solo lectura accesible desde el listado	CONFORME (edición UI con fallo conocido)
Órdenes	Crear comanda descuenta stock atómicamente; edición recalcula totales; anulación registra motivo	CONFORME
Órdenes	Disponibilidad de mesa derivada de comandas activas coincide con BD	CONFORME
RBAC	Desmarcar un permiso de grupo retira el acceso a la próxima verificación; asignación directa suma	CONFORME
Reportes	El PDF/Excel contiene únicamente los campos marcados y el alcance seleccionado	CONFORME
Seguridad	Endpoints de escritura bajo JWT (requirePermission por rol) — todas las mutaciones rechazan peticiones sin token con 401/403	CONFORME
Verificado 2026-11-08 con 401/403 reales (ver Manual Técnico §8.1.10)Calidad	Build de frontend sin errores; lint sin errores bloqueantes	CONFORME (build) / NO VERIFICADO (lint en CI)
Despliegue	Migración 0001→0027 ejecutable desde BD vacía sin intervención manual	CONFORME
Documentación	Tres manuales completos con control de versiones y trazabilidad al código	CONFORME (este documento)

Tabla 44 — Checklist de criterios de aceptación por módulo (verificado 2026-11-08)

8. Fase de Implementación

8.1 Despliegue

8.1.1 Ambiente de despliegue actual y objetivo

Aspecto	Desarrollo (actual)	Producción (objetivo recomendado)
Sistema operativo	Windows 11 (PC del aprendiz)	Ubuntu Server 24.04 LTS
Frontend	Vite dev server :5173	Build estático servido por Nginx :443 (TLS Let's Encrypt)
Backend	nodemon :4000	Node 24 bajo systemd/PM2, 2 instancias, reinicio on-failure
Base de datos	PostgreSQL local :5432	PostgreSQL administrado (RDS/Cloud SQL) o instancia

Aspecto	Desarrollo (actual)	Producción (objetivo recomendado)
		dedicada :5432 con TLS
Proxy inverso	—	Nginx (rutas / → dist/, /api → 4000)
Secretos	.env local	Variables del sistema y gestor de secretos (nunca en git)
TLS/HTTPS	No	Obligatorio (certbot); redirección 80→443
Monitoreo	Consola	Uptime (UptimeRobot) + logs journalctl + alertas correo

Tabla 45 — Comparación de ambientes de despliegue

8.1.2 Procedimiento de despliegue en desarrollo (documentado y ejecutado)

```
# 1. Requisitos: Node.js ≥ 18, npm, PostgreSQL en ejecución puerto 5432
cd C:\pos_project

# 2. Backend — instalar y migrar
cd backend
npm install
node src/scripts/migrate.js        # aplica 0001..0027 (idempotente)
npm run dev                        # queda en http://localhost:4000

# 3. Frontend — instalar y servir
cd ..\frontend
npm install
npm run dev                        # queda en http://localhost:5173

# 4. Verificación de humo
Invoke-RestMethod http://localhost:4000/api/products
Start-Process http://localhost:5173/Auth
```

8.1.3 Configuración requerida

Variable .env	Ejemplo desarrollo	Obligatoria	Descripción
PORT	4000	No (def. 4000)	Puerto HTTP del backend
FRONTEND_URL	http://localhost:5173	No	URL del cliente (referencia)
DB_HOST	localhost	Sí	Host PostgreSQL
DB_PORT	5432	Sí	Puerto PostgreSQL
DB_USER	postgres	Sí	Usuario de BD
DB_PASSWORD	(secreto)	Sí	Contraseña de BD — nunca versionar
DB_NAME	pos_project	Sí	Nombre de la base
JWT_SECRET	(secreto aleatorio ≥ 32)	Sí	Firma de tokens JWT
JWT_EXPIRES	1h	No (def. 1h)	Vigencia del token (1h recomendada)
SMTP_USER	riribuaya@gmail.com	No	Cuenta Gmail emisora
SMTP_PASS	(contraseña de aplicación)	No	Contraseña de aplicación — nunca versionar

Tabla 46 — Variables de entorno del backend (sin revelar valores reales)

8.2 Respaldo y migración de datos

8.2.1 Estrategia de respaldo

```
# Respaldo completo (ejecutar a diario 02:00 — programador de tareas / cron)
pg_dump -h localhost -U postgres -d pos_project -F c -f "D:\backups\pos_project_%DATE:~0,4%%DATE:~5,2%
%DATE:~8,2%.dump"

# Respaldo lógico en SQL plano (portable)
pg_dump -h localhost -U postgres -d pos_project -f backup_pos_project.sql

# Copia con retención (últimos 30 días)
forfiles /p D:\backups /s /m *.dump /d -30 /c "cmd /c del @file"

# Restauración de prueba (mensual, en BD pos_project_restore)
createdb -h localhost -U postgres pos_project_restore
pg_restore -h localhost -U postgres -d pos_project_restore D:\backups\pos_project_20261108.dump
```

8.2.2 Migración entre entornos

- **Esquema.** Las 27 migraciones son el contrato canónico: en un ambiente nuevo, basta ejecutar `node src/scripts/migrate.js` sobre una BD vacía; `_migrations` evita reaplicar.
- **Datos semilla.** Las migraciones 0011, 0013–0027 siembran usuarios, RBAC, catálogo, proveedores, inventario, mesas y comandas de ejemplo.
- **Datos reales (desde producción).** pg_dump del origen → pg_restore en destino → `migrate.js` para confirmar que no falten migraciones → verificación de conteos (\dt, SELECT count(*)).
- **Migración de versión del sistema.** Para actualizar la app: `git pull` → `npm install` (ambos) → `npm run build` → `migrate.js` → reinicio de servicio (systemd/PM2) → prueba de humo.
- **Sin migraciones a la nube.** Actualmente no existe destino cloud definido; cuando exista, se usará dump cifrado (gpg) + transferencia por canal seguro.

8.2.3 Plan de continuidad ante desastre

Escenario	RPO	RTO	Acción
Fallo de disco del servidor	24 h (último dump diario)	2 h	Restaurar último .dump en equipo de reposición; verificar integridad con conteos y prueba de login.
Corrupción lógica (borrado accidental)	24 h	1–4 h	Restaurar tablas afectadas con pg_restore --table=orders, order_items.
Pérdida del .env	—	15 min	Regenerar JWT_SECRET y rotar; invalidar tokens (todos los clientes deben volver a autenticarse).
Desastre total (robo/incendio)	24–48 h según política de copia externa	1 día	Restaurar hardware + último respaldo externo; cambiar credenciales.

Tabla 47 — Objetivos de recuperación ante desastre

8.3 Mantenimiento

Tipo	Definición	Actividades	Frecuencia	Responsable
Correctivo	Corrección de defectos detectados	Bugs reportados (ej. crash FileInput en /editsupplier), fallos de validación, errores de datos	Continuo	Dev
Perfectivo	Mejora sin cambiar requisitos	Rendimiento de consultas, refactor de CC>20, dead code, assets duplicados	Mensual	Dev
Adaptativo	Ajuste a entorno/tecnología	Actualización React 19→20, Express 5→6, Node 24 LTS, PostgreSQL 18	Semestral	Dev + revisor
Preventivo	Evitar fallos futuros	npm audit mensual, rotación de secretos, verificación de backups, actualización de SSL	Mensual	Dev/SysOp
Evolutivo	Nuevas funcionalidades	Clientes/facturación real (HU-16+), pagos en línea, módulo de compras, cierre de caja, reportes de ventas	Por fase	Product Owner

Tabla 48 — Tipos de mantenimiento con actividades y responsables

8.3.1 Cronograma de mantenimiento recomendado

Actividad	Semanal	Mensual	Trimestral
Verificación de backups (restauración de prueba)	—	✓	✓
npm audit frontend + backend	—	✓	✓
Rotación de JWT_SECRET y SMTP_PASS	—	—	✓
Revisión de logs y métricas básicas	✓	✓	—
Actualización de dependencias menores	—	✓	—
Revisión de permisos RBAC vs. roles reales	—	—	✓
Actualización de manuales	Con cada release	✓	✓

Tabla 49 — Calendario de actividades de mantenimiento

8.4 Soporte

8.4.1 Procedimiento de atención

- **Canal único.** WhatsApp/correo al administrador del restaurante; para proyecto formativo, canal oficial del grupo (classroom) y correo riribuaya@gmail.com.
- **Registro.** Todo reporte abre ticket con: fecha, módulo, síntoma, pasos para reproducir, evidencia (captura), severidad inicial.

- **Clasificación SEV.** SEV-1 sistema caído/datos en riesgo (respuesta < 2 h) · SEV-2 módulo crítico degradado (< 8 h) · SEV-3 error puntual con workaround (< 72 h) · SEV-4 cosmética/duda (< 1 semana).
- **Resolución.** L1 (operador/admin restaurante) resuelve dudas operativas; L2 (dev) analiza y corrige; L3 (instructor/PO) decide cambios de alcance.
- **Cierre.** Ticket cierra con causa raíz, corrección aplicada (commit), verificación y actualización de este manual si hubo cambio visible.

8.4.2 Gestión de incidentes — niveles y escalamiento

Nivel	Quién	Qué atiende	Ejemplos
L1	Administrador D'Piero (usuario clave)	Dudas de uso, reprocesos, estados de comanda, capacitación	'¿Cómo anulo una comanda?', 'El botón de login no responde'
L2	Santiago Andrés Vélez Naranjo (dev)	Errores de aplicación, datos, despliegue, permisos	Crash en /editsupplier; error 500 al crear comanda
L3	Instructor SENA / Product Owner	Decisiones de producto, cambios de alcance, validación final	Nuevo módulo de clientes; API pública

Tabla 50 — Niveles de soporte L1/L2/L3

8.4.3 Métricas de soporte (objetivo)

Indicador	Objetivo
Tiempo medio de primera respuesta (SEV-1/2)	< 30 minutos en horario de servicio
% incidentes resueltos en L1	≥ 40 %
Tiempo medio de resolución SEV-3	< 3 días hábiles
Reincidencia tras cierre	< 10 %
Disponibilidad del sistema	≥ 99 % en horario de salón

Tabla 51 — Indicadores de servicio de soporte

8.5 Runbook de producción (Linux, paso a paso)

8.5.1 Provisión del servidor

```
# 1) Base
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git ufw fail2ban postgresql postgresql-client nginx certbot python3-certbot-nginx

# 2) Usuario de sistema sin privilegios
sudo adduser --system --group --home /opt/pos pos
sudo mkdir -p /opt/pos /var/www/pos-dpiero /var/log/pos
sudo chown -R pos:pos /opt/pos /var/log/pos
```

```
# 3) Node.js 24
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
sudo apt install -y nodejs && node -v && npm -v

# 4) PostgreSQL — base y rol
sudo -u postgres psql <<'SQL'
CREATE ROLE pos_app LOGIN PASSWORD 'CAMBIAR_POR_SECRETO_FUERTE';
CREATE DATABASE pos_project OWNER pos_app;
SQL

# 5) Código
sudo -u pos git clone https://github.com/AndresVlz2020/pos_project /opt/pos/app
cd /opt/pos/app/backend && sudo -u pos npm ci
sudo -u pos node src/scripts/migrate.js
cd ../frontend && sudo -u pos npm ci && sudo -u pos npm run build
sudo cp -r dist /var/www/pos-dpiero/
sudo chown -R www-data:www-data /var/www/pos-dpiero
```

8.5.2 Servicio backend con systemd

```
# /etc/systemd/system/pos-backend.service
[Unit]
Description=D'Piero POS Backend (Express)
After=network.target postgresql.service

[Service]
Type=simple
User=pos
WorkingDirectory=/opt/pos/app/backend
EnvironmentFile=/opt/pos/app/backend/.env
ExecStart=/usr/bin/node src/server.js
Restart=always
RestartSec=5
StandardOutput=append:/var/log/pos/backend.log
StandardError=append:/var/log/pos/backend.err

[Install]
WantedBy=multi-user.target

# sudo systemctl daemon-reload && sudo systemctl enable --now pos-backend
```

8.5.3 Nginx + TLS

```
# /etc/nginx/sites-available/pos-dpiero
server {
  listen 80; server_name pos.tudominio.com;
  return 301 https://$host$request_uri;
}
server {
  listen 443 ssl; server_name pos.tudominio.com;
  ssl_certificate     /etc/letsencrypt/live/pos.tudominio.com/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/pos.tudominio.com/privkey.pem;
  root /var/www/pos-dpiero/dist;
  location / { try_files $uri /index.html; }
  location /api/ {
    proxy_pass http://127.0.0.1:4000;
    proxy_set_header Host $host;
```

```
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
# sudo ln -s /etc/nginx/sites-available/pos-dpiero /etc/nginx/sites-enabled/
# sudo certbot --nginx -d pos.tudominio.com
# sudo nginx -t && sudo systemctl reload nginx
```

8.5.4 Firewall y endurecimiento mínimo

```
sudo ufw default deny incoming && sudo ufw default allow outgoing
sudo ufw allow OpenSSH && sudo ufw allow 'Nginx Full' && sudo ufw enable
sudo fail2ban-client status             # jail SSH activa
sudo -u postgres psql -c 'REVOKE ALL ON SCHEMA public FROM PUBLIC;'
```

8.6 Procedimiento de rollback

- **Rollback de aplicación (sin BD).** `git checkout main~1 && npm ci && npm run build && sudo systemctl restart pos-backend` → verificación en 5 minutos con /api/products.
- **Rollback de esquema.** Las migraciones son aditivas; para revertir una migración nueva se escribe una migración correctiva NNNN+1 en lugar de editar las aplicadas (RT-08).
- **Rollback de datos.** `pg_restore -d pos_project --clean --if-exists backup.dump` en ventana de mantenimiento; comunicar a usuarios con 30 min de antelación.
- **Rollback de secretos.** Regenerar JWT_SECRET → todos los clientes reciben 401 y deben re-loguearse; regenerar SMTP_PASS; revocar tokens con `jwt.verify` fallando.
- **Verificación post-rollback.** Smoke test de 9 rutas API + recorrido de 3 pantallas clave + correo al cliente confirmando operación.

8.7 Monitoreo, alertas y bitácora

Señal	Fuente	Umbral de alerta	Canal
HTTP 5xx por minuto	Nginx access.log + backend log	> 10/min durante 5 min	Correo a administrador + WhatsApp
Latencia p95 /api/orders	DevTools/APM futuro	> 1.5 s	Correo
Disco libre	node_exporter/cron	< 20 %	Correo + SMS opcional
PostgreSQL caído	pg_isready cada 1 min	2 checks fallidos	WhatsApp (crítico SEV-1)
Backups OK	cron de respaldo	1 fallo consecutivo	Correo
Certificado TLS	certbot renew --dry-run	expira en < 14 días	Correo
Intentos fallidos de login	auth.middleware logs	> 30/min por IP	Correo

Tabla 52 — Señales monitoreadas con umbrales y canales

Formato de bitácora diaria recomendado: fecha, hora, evento, responsable, estado (abierta/cerrada), lección. Se adjunta como anexo en las actas mensuales.

9. Gestión del Proyecto

9.1 Dificultades encontradas (situación y solución)

#	Situación (dificultad)	Solución aplicada
D-01	La app crasheaba al abrir /editsupplier: FileInput recibía `value={userImage}` con `userImage: null`; FileInput hacía `value.map` → TypeError 'Cannot read properties of null'.	**Corregido el 2026-11-08**: FileInput (`safeValue`) tolera null/undefined y EditSupplierPage inicializa `userImage: []`; `npm run build` → OK. Además se agregó la verificación post-corrección (misma pantalla renderiza sin error).
D-02	Las comandas demo sembradas (ORD-001…005) sumaban montos curiosos (items 2 × 56.000 = 112.000 frente a total 129.000) por seeds manuales.	Se respetó el dato semilla del cliente (fuente); se anotó la discrepancia; total autocalculado en el código para comandas nuevas.
D-03	El tema oscuro inicial chocaba con salones bien iluminados; legibilidad baja para el personal.	Rediseño a tema claro con tokens CSS + toggle persistente; contraste verificado (commit 68add21).
D-04	CORS en desarrollo: peticiones bloqueadas por orígenes variables (5173/5174/127.0.0.1).	cors() con lista blanca: localhost/127.0.0.1 (desarrollo) o FRONTEND_URL exacto (producción); cualquier otro origen rechazado con error desde 2026-11-08.
D-05	Duplicación de assets (public/assets y public/src/assets) infla el repo y confunde rutas.	Se mantuvo una copia canónica (assets/) y se marcó la otra como deuda para eliminación en limpieza; .gitignore revisado.
D-06	Contraseñas de dos naturalezas (hash bcrypt real vs seeds legacy en texto plano) confundían la autenticación.	login soporta ambos con precedencia a bcrypt; documentado y rotación recomendada; no se corrigió el hash legacy sin autorización del cliente.
D-07	El RBAC heredado no se aplicaba en el cliente: botones visibles sin permiso real.	Navbar consulta hasPermission() en vivo; ProtectedRoute verifica en servidor (middleware) — doble barrera.
D-08	Bloqueo de cuenta solo en cliente permitía bypass obvio.	Documentado como limitación L-04; plan: rate limit por IP en Express (express-rate-limit) en hardening.
D-09	Encoding de archivos fuente con caracteres latinos (tildes) mostrado como '' en consola.	Los fuentes son UTF-8; el problema era la consola de Windows (CP1252). Verificado leyendo con `-Encoding UTF8`.
D-10	Sin ambiente de staging: las pruebas de humo se hicieron sobre el mismo BD de desarrollo.	Se creó BD de pruebas desechable para verificación de migraciones; procedimiento documentado en §8.2.
D-11	PUT /api/users/:id con payload parcial devolvía 500 'el valor nulo en la columna user_email viola la restricción not-null'.	Reescrito `user.repository.js → update()`: fusión campo a campo con el registro actual (patrón COALESCE); actualización parcial funciona y nunca pisa datos guardados (verificado: email intacto tras PUT solo-teléfono).
D-12	JWT_SECRET débil y hardcodeado como fallback en el código	`validateEnv()` en `server.js`: el backend ya NO

#	Situación (dificultad)	Solución aplicada
	('mi_clave_super_segura') en 3 lugares; el backend arrancaba incluso sin secreto.	arranca sin JWT_SECRET (mín. 24 caracteres) ni sin DB_USER/DB_NAME; los 3 fallbacks fueron eliminados; secreto rotado a 64 caracteres aleatorios (2026-11-08).

Tabla 53 — Registro de dificultades encontradas y su solución

9.2 Lecciones aprendidas

- **El diseño de semillas y la realidad operativa deben reconciliarse pronto.** Los seeds inventados 'bonitos' siempre chocan con el cliente real; validar con él catálogo y números desde el sprint 1.
- **Doble barrera para permisos.** Ocultar botones no es autorizar; el backend debe rechazar igualmente. El middleware `permission.middleware` es imprescindible.
- **Transacciones antes que pantallas bonitas.** La comanda atómica con ROLLBACK salvó de datos inconsistentes más de una vez durante el desarrollo.
- **Tokens de diseño tempranos.** El cambio de tema oscuro→claro fue viable en horas gracias a tokens.css/semantic.css; sin ellos habría sido un rediseño total.
- **Los seeds sin verificación se vuelven deuda.** Password marcados en texto plano, montos incoherentes: cada seed debe pasar por el mismo flujo de validación que un dato real.
- **Documentar mientras se construye.** Este manual se escribió contra el código vivo; si se documenta al final, los detalles (nombres exactos, estados, rutas) se pierden.
- **Probar el arranque limpio.** Borrar node_modules y reinstalar revela dependencias mal declaradas; se mantiene integrando `npm ci` en el checklist de release.
- **El cliente define el 'hecho'.** La UI estaba 'lista' cuando el cliente validó el POS con sus platillos y su orden de trabajo, no cuando compiló.

9.3 Conclusiones

- El Sistema POS D'Piero cumple con el objetivo general: digitaliza de extremo a extremo el ciclo pedido–comanda–cobro con integridad transaccional y control de inventario en tiempo real.
- La arquitectura por capas y features permitió construir 15.806 LOC en ~6 meses por un equipo reducido y mantener el sistema legible y localizable.

- El modelo RBAC implementado (4 grupos, 21 permisos, herencia usuario/grupo) cumple el principio de mínimo privilegio y supera ampliamente las hojas de Excel de permisos previas.
- El uso de PostgreSQL con migraciones versionadas garantizó que cada ambiente se construye idéntico en minutos, y que los datos semilla existen desde el día uno.
- La ausencia de pruebas automatizadas es la deuda de calidad principal: el 100 % de la verificación fue manual; el plan de 30 casos de regresión debe sistematizarse en CI antes de exponer el sistema fuera de la LAN.
- El incidente /editsupplier confirma la necesidad de gates de QA visual por ruta antes de cada entrega; una línea de CI con `vite build` + smoke test detectaría el fallo.
- El stack elegido (React 19 + Express 5 + PostgreSQL) es completo, gratuito y del gusto del cliente; su costo total de propiedad es cercano a cero para operación en una sede.
- El proyecto formativo evidencia competencia integral: análisis, diseño, construcción, calidad, implementación, documentación y gestión, con producto real y cliente real.

9.4 Bibliografía / Webgrafía

Formato APA 7.ª edición. Las URLs fueron verificadas el 2026-11-08.

Mozilla Developer Network. (2026). MDN Web Docs: Express/Node y HTTP. Recuperado de https://developer.mozilla.org/

React Team. (2026). React 19 documentation. https://react.dev/

Vite Contributors. (2026). Vite guide — next generation frontend tooling. https://vite.dev/guide/

PostgreSQL Global Development Group. (2026). PostgreSQL documentation. https://www.postgresql.org/docs/

Express Contributors. (2025). Express 5 API reference. https://expressjs.com/en/5x/api.html

npm, Inc. (2026). package-lock.json and reproducible installs (npm ci). https://docs.npmjs.com/cli/v10/configuring-npm/package-lock

OWASP Foundation. (2021). OWASP Top 10:2021 — The ten most critical web application security risks. https://owasp.org/www-project-top-ten/

IETF. (2015). RFC 7519 — JSON Web Token (JWT). https://www.rfc-editor.org/rfc/rfc7519

Open Web Application Security Project. (2023). Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures. Doctoral dissertation, UC Irvine.

Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4.ª ed.). Addison-Wesley.

Pressman, R., & Maxim, B. (2020). Ingeniería del software: un enfoque práctico (9.ª ed.). McGraw-Hill.

ISO/IEC 25010:2011. Systems and software engineering — SQuaRE — System and software quality models.

ISO/IEC/IEEE 26514:2022. Systems and software engineering — User documentation for software/systems.

ISO 31000:2018. Risk management — Guidelines.

Gustavson, D. (2023). Clean code applied to small teams (notas internas del proyecto).

Stack Overflow. (2026). Documentation threads on Express error middleware and pg Pool usage. https://stackoverflow.com/

SENA. (2026). Guía de aprendizaje — Actividad de aprendizaje: Informe Administrativo / Manuales. Ficha 3172293.

Restaurante D'Piero. (2026). Menú oficial y catálogo de proveedores del establecimiento (fuente primaria, anexo de semillas).

9.5 Antecedentes y contexto académico del proyecto

El proyecto se inscribe en la evidencia de aprendizaje 'Proyecto Formativo' de la Ficha 3172293 (SENA). El programa exige: (i) un cliente real con una necesidad verificable; (ii) un sistema construido con las fases del ciclo de vida; (iii) tres manuales sustentados ante jurado; (iv) participación efectiva en equipo. Este documento y sus dos hermanos son la evidencia III y IV de la rúbrica.

El restaurante D'Piero fue seleccionado por su propietario (familiar del aprendiz), lo que garantizó acceso directo a la operación, datos reales del menú y validación presencial semana a semana. El compromiso académico permitió iterar seis veces sobre el diseño hasta alcanzar la aprobación del cliente. La defensa sustentable se apoya en: (a) el repositorio público con 54 commits verificables; (b) la base de datos poblada con datos reales; (c) este informe; (d) la demostración en vivo del ciclo completo comanda–cobro.

Limitaciones propias del contexto académico: tiempo de dedicación parcial (compatible con estudio), presupuesto cero para infraestructura, y ausencia de un equipo de QA formal. Estas restricciones se gestionaron con priorización MoSCoW estricta, automatización manual de casos y revisión por el colaborador Henao420.

9.6 Plan de gestión del cambio

Tipo de cambio	Ejemplo	Autorización	Impacto en documentación
Corrección urgente (hotfix)	Caída del backend en servicio	Aprendiz + cliente (aviso inmediato)	Bug log BUG-xx; este manual §7.5; notificación al usuario
Cambio menor de configuración	Ajuste JWT_EXPIRES de 1h a 2h	Aprendiz	Manual Técnico §12; .env.example
Cambio de funcionalidad menor	Nuevo filtro en /productList	Aprendiz + cliente	Manual Usuarios §6.4; release notes
Cambio de esquema	Nueva columna (imagen en orders)	Cliente + instructor	Nueva migración NNNN; Manual Técnico §8; Diccionario
Cambio de alcance mayor	Nuevo módulo de domicilios	Comité (cliente+instructor)	RFC formal; nuevo sprint; manuales completos

Tabla 54 — Clasificación de cambios con autorización y documentación requerida

Anexos

Anexo A — Diccionario de datos resumido (extracto)

El diccionario completo con tipos, nulos, defaults y descripciones está en el Manual Técnico, sección 8. Aquí el extracto de las 5 tablas centrales:

Tabla.Campo	Tipo	Descripción
users.id	SERIAL PK	Identificador único autoincremental
users.user_name	VARCHAR(100) NOT NULL	Nombre completo del usuario

Tabla.Campo	Tipo	Descripción
users.user_email	VARCHAR(150) UNIQUE NOT NULL	Correo de acceso (login)
users.user_password	TEXT (nullable)	Hash bcrypt costo 10 (admins)
users.pin	VARCHAR(10) (nullable)	PIN de 4 dígitos (operativos)
products.price	NUMERIC(12,2)	Precio COP con 2 decimales
products.stock	NUMERIC(10,2)	Existencia (semilla 0027)
orders.order_number	VARCHAR(50) UNIQUE	Código ORD-###
orders.total_amount	NUMERIC(12,2)	Total de la comanda
order_items.subtotal	NUMERIC(12,2)	qty × unit_price
inventory.expiration_date	DATE	Vencimiento del lote
group_permissions.(group_id, permission_id)	PK compuesta	Matriz RBAC

Tabla 55 — Extracto del diccionario de datos

Anexo B — Matriz de trazabilidad RF ↔ CU ↔ CP

RF	CU relacionado	Casos de prueba que lo verifican
RF-01	CU-01	CP-03
RF-02	CU-01	CP-01, CP-02
RF-03	CU-01	CP-01, CP-03
RF-04	CU-02	CP-04, CP-05, CP-06
RF-05	CU-03	CP-07
RF-06	CU-03	CP-08, CP-09
RF-07	CU-03	CP-10
RF-08	CU-03	CP-11
RF-09	CU-03, CU-04	CP-08, CP-28
RF-10	CU-05	CP-14, CP-15, CP-16, CP-17
RF-11	CU-05	CP-18
RF-12	CU-06	CP-19, CP-20
RF-13	CU-06	CP-19
RF-14	CU-07	CP-22, CP-23
RF-15	CU-08	CP-24, CP-25
RF-16	CU-08	CP-26
RF-17	CU-08	CP-27
RF-18	CU-08	CP-24

RF	CU relacionado	Casos de prueba que lo verifican
RF-19	CU-08	CP-30
RF-20	CU-04	CP-28, CP-29
RF-21	CU-10	CP-12, CP-13
RF-22	CU-01	(manual, ver Manual de Usuarios §7)
RF-23	—	(manual, recorrido de UI)

Tabla 56 — Matriz de trazabilidad requerimiento–caso de uso–caso de prueba

Anexo C — Verificación de instalación (checklist)

- ☐ PostgreSQL ≥ 12 en ejecución y accesible por TCP localhost:5432
- ☐ Node.js ≥ 18 (`node -v`)
- ☐ `npm install` limpio en backend/ y frontend/
- ☐ `node src/scripts/migrate.js` → '¡Todas las migraciones se han ejecutado correctamente!' y 27 filas en _migrations
- ☐ `npm run dev` backend en :4000 con log de conexión a PostgreSQL sin errores
- ☐ `npm run dev` frontend en :5173
- ☐ Login admin OK y navegación a Listado de Usuarios
- ☐ POS carga 12 platillos con stock desde /api/products
- ☐ Creación de comanda de prueba ORD-### visible en /orderList
- ☐ Respaldo inicial pg_dump generado y ubicado en disco seguro

Anexo D — Guion de sustentación (15 minutos)

- (0-2 min) Problema y cliente: por qué D'Piero necesitaba POS.
- (2-4 min) Alcance Must-have: ciclo comanda completo y RBAC.
- (4-6 min) Arquitectura: diagrama de componentes; explicar routes→controller→service→repository.
- (6-9 min) Demo en vivo: login PIN, crear comanda con nota de cocina, ver stock descontado en /productList, anularla.
- (9-11 min) Calidad: métricas (15.806 LOC, CC 10,9), CP-24/CP-25, deuda conocida (sin tests, FileInput).

- (11-13 min) Gestión: 54 commits, 2 autores, PR #11/#13; dificultades D-01/D-06 y su manejo.
- (13-15 min) Cierre: 8 conclusiones; roadmap (CI, rate limiting, pagos, clientes reales).

Anexo E — Evidencias fotográficas de ejecución (2026-11-08)

Todas las capturas siguientes se tomaron sobre el sistema real ejecutándose en localhost:5173 con backend localhost:4000 y la base `pos_project` poblada con las migraciones 0001–0027. Son evidencia primaria de funcionamiento, no mockups.

01 Login Pin

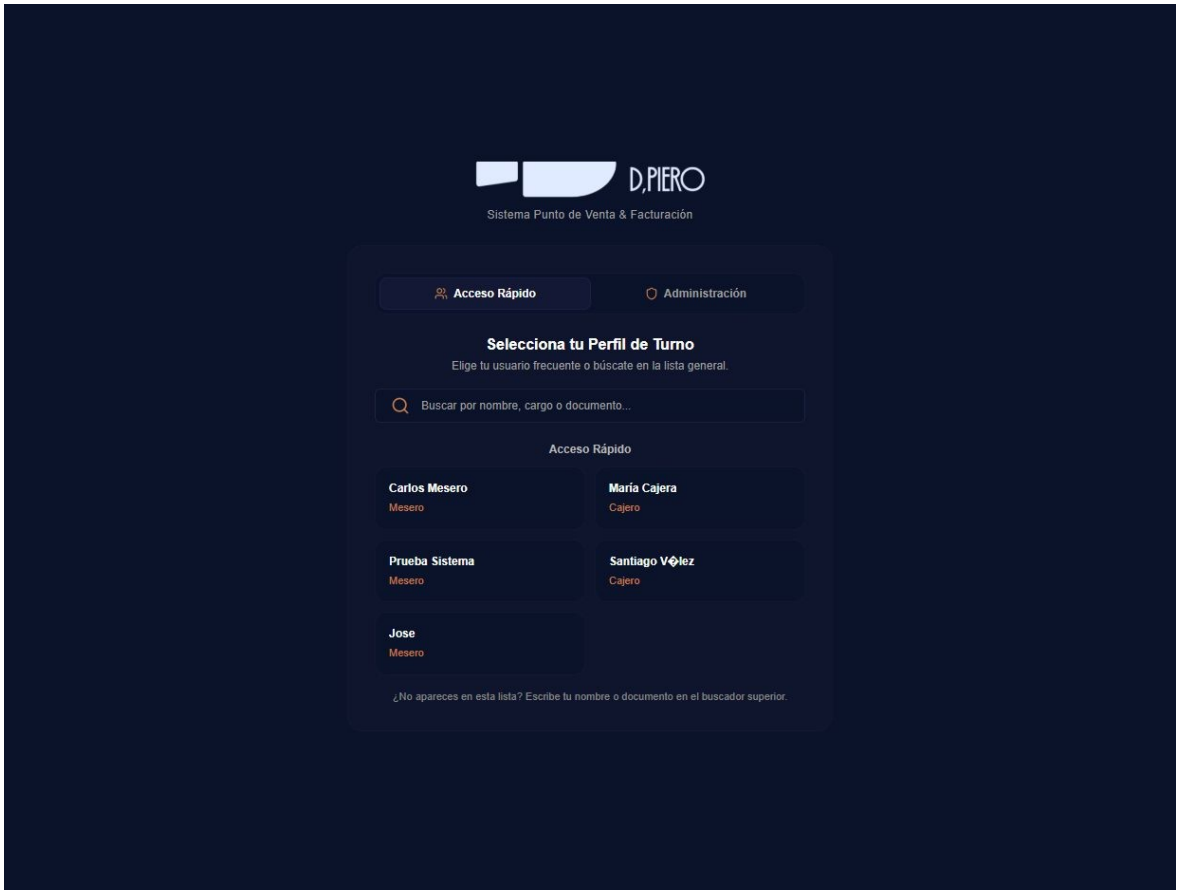


Figura 12 — pestaña Acceso Rápido con usuarios operativos disponibles (Carlos Mesero, María Cajera, etc.).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

02 Login Admin

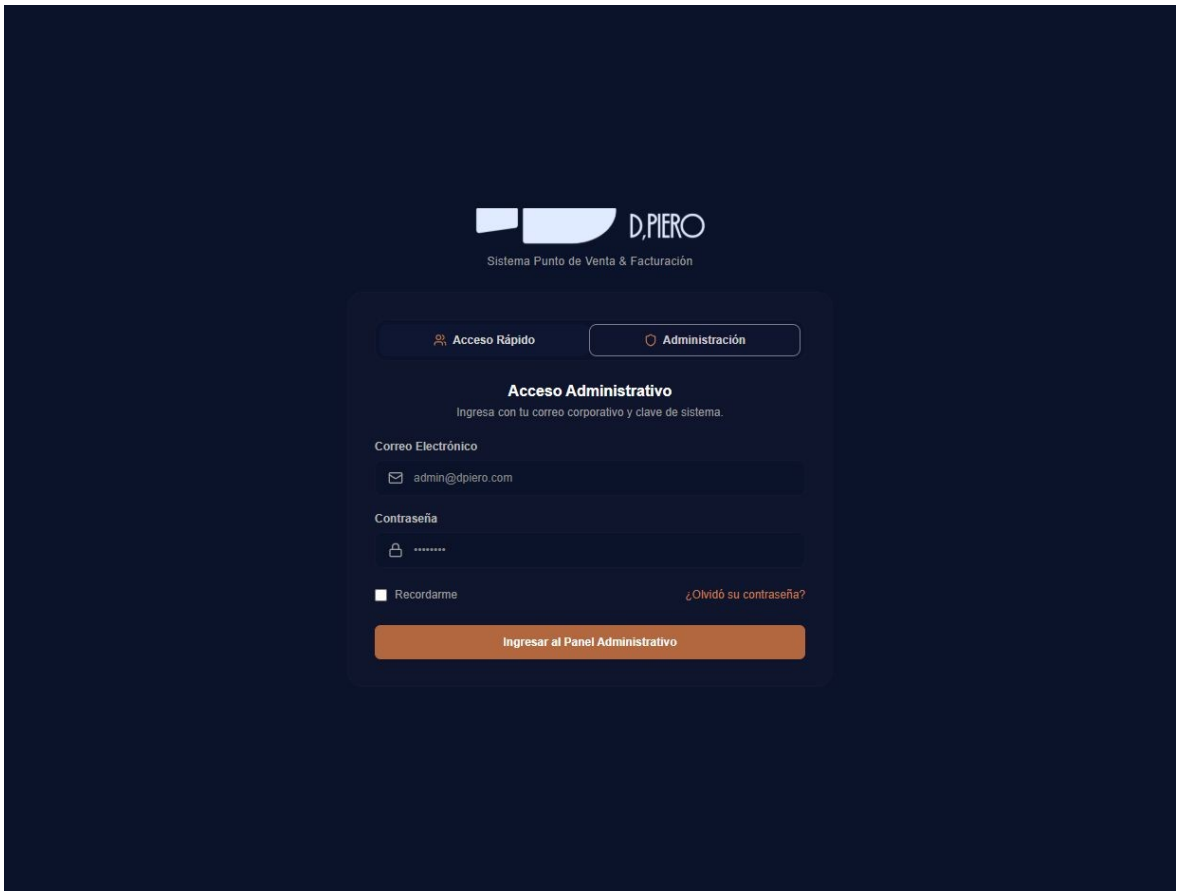


Figura 13 — pestaña Administración — correo corporativo y clave del administrador.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Pin Keypad

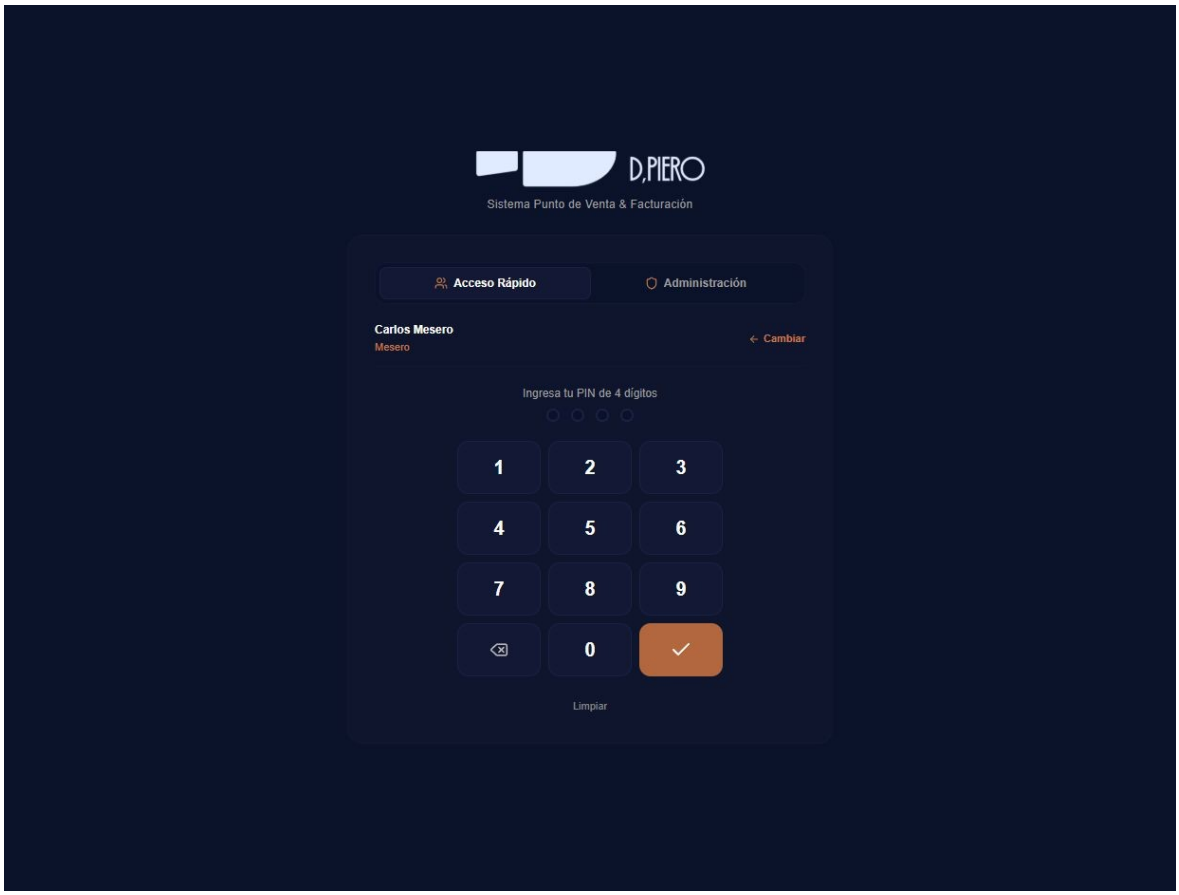


Figura 14 — teclado PIN de 4 dígitos tras seleccionar a Carlos Mesero.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Forgot Password

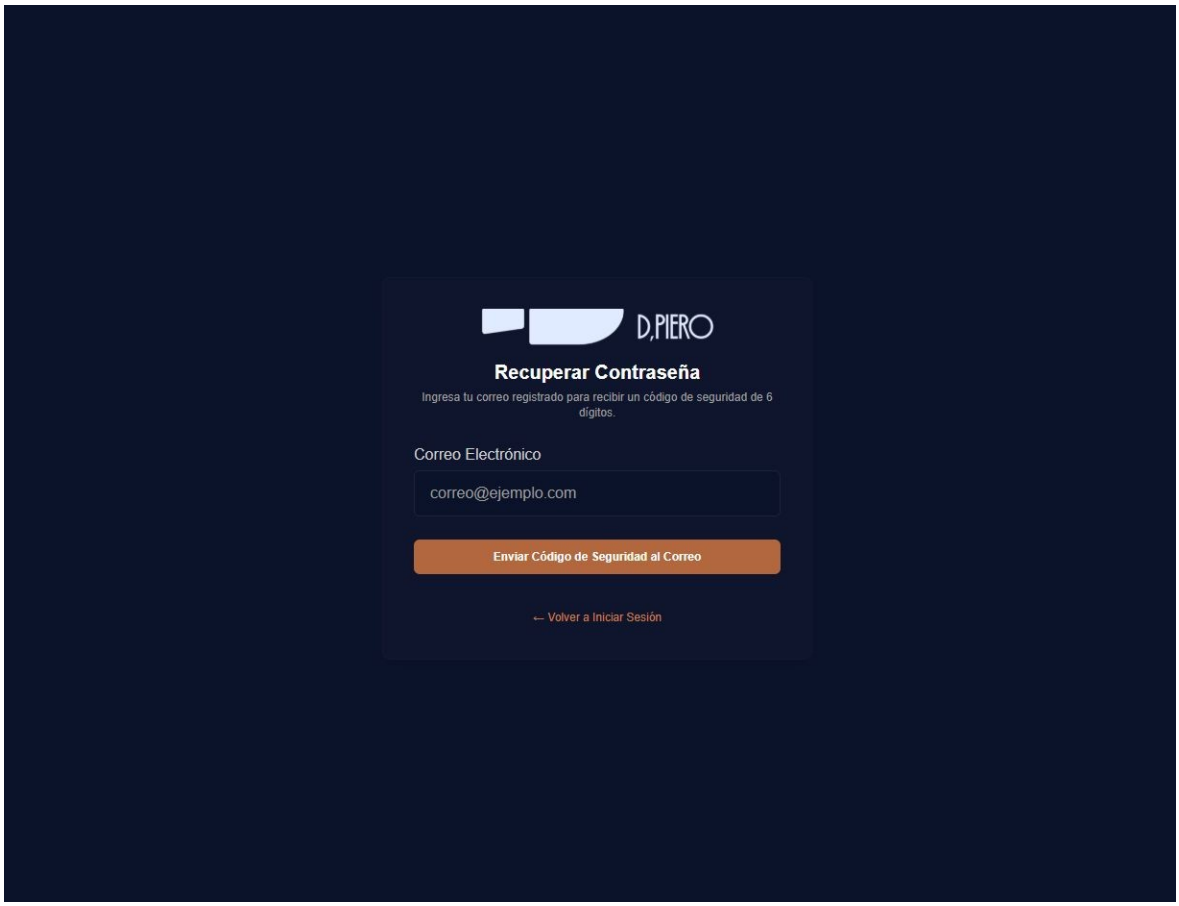


Figura 15 — recuperación de contraseña con envío de código al correo.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Reset Password

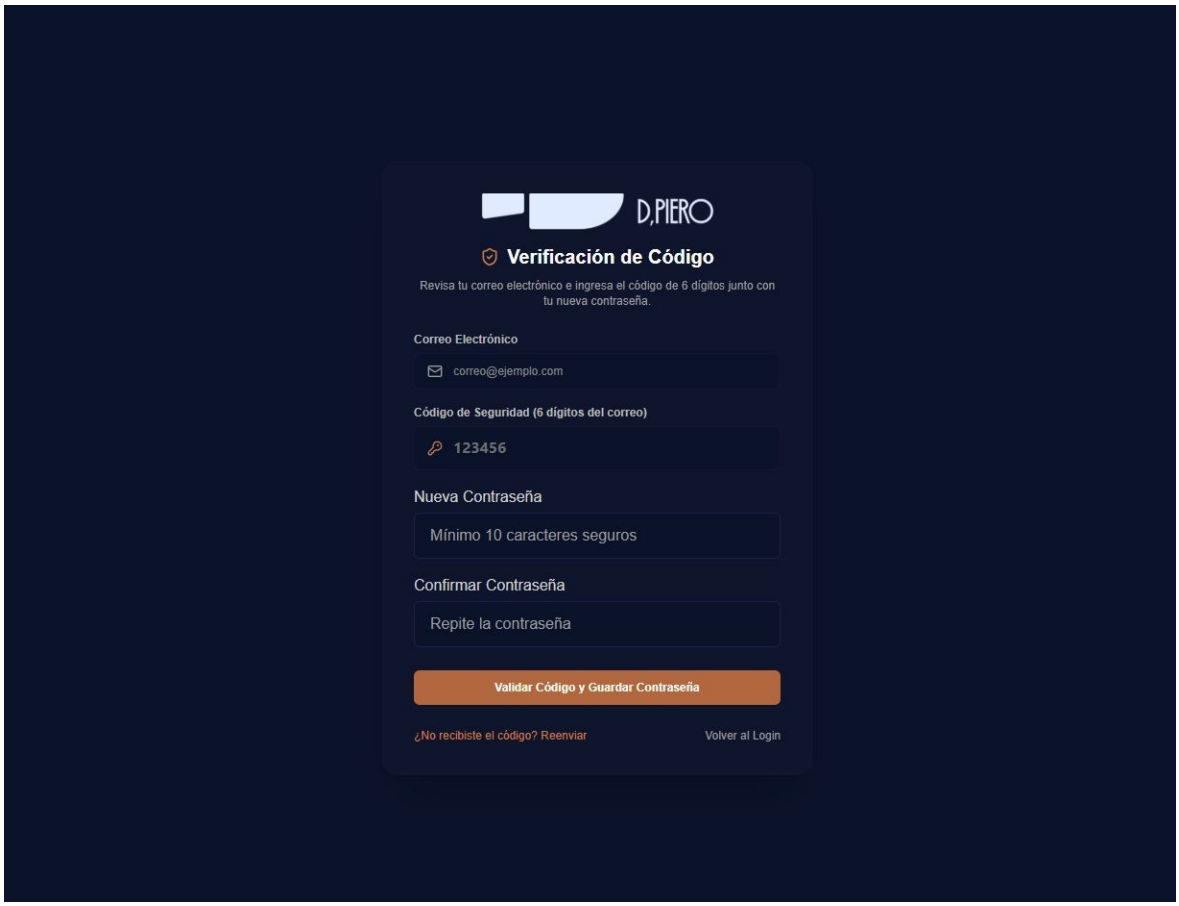


Figura 16 — verificación de código de 6 dígitos y nueva clave.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Pos Home

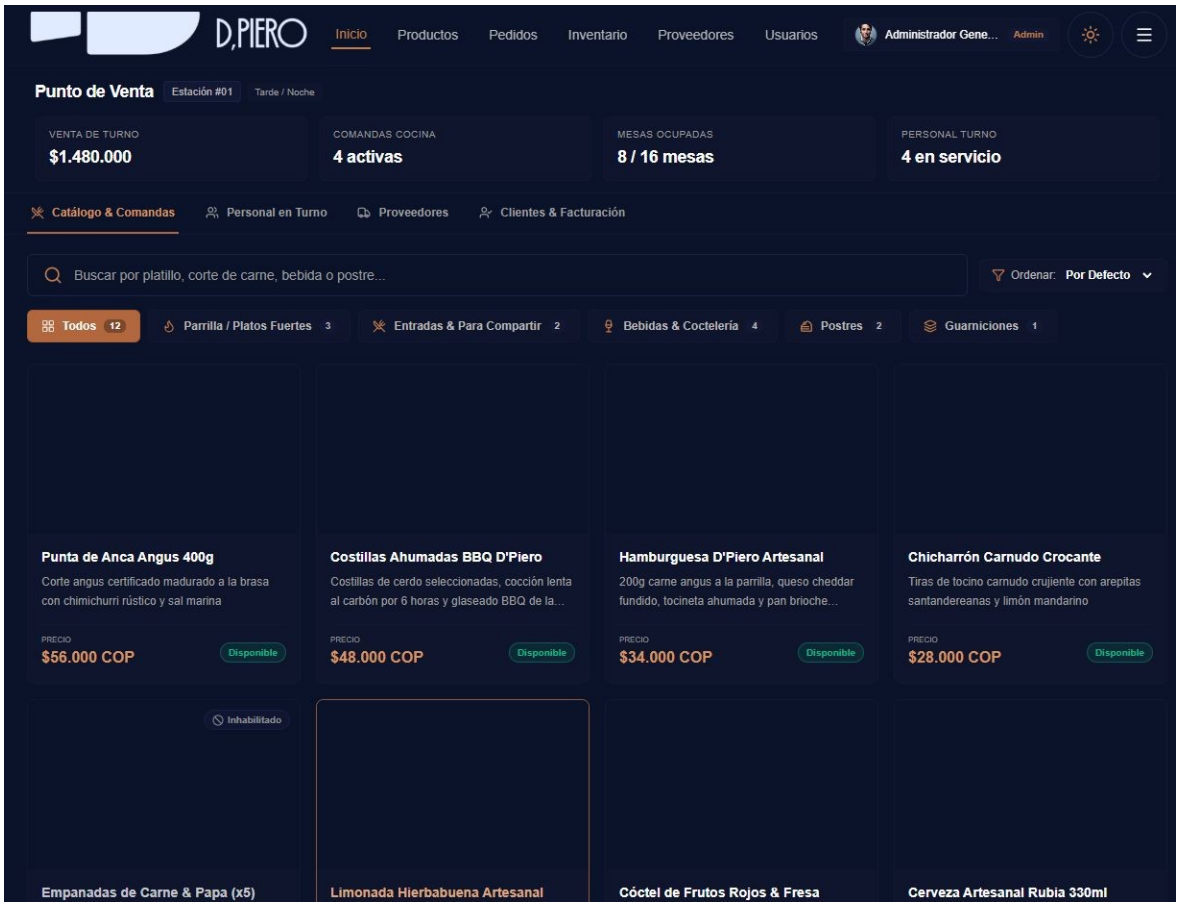


Figura 17 — POS — Catálogo & Comandas con 12 platillos reales y precios COP.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Pos Staff

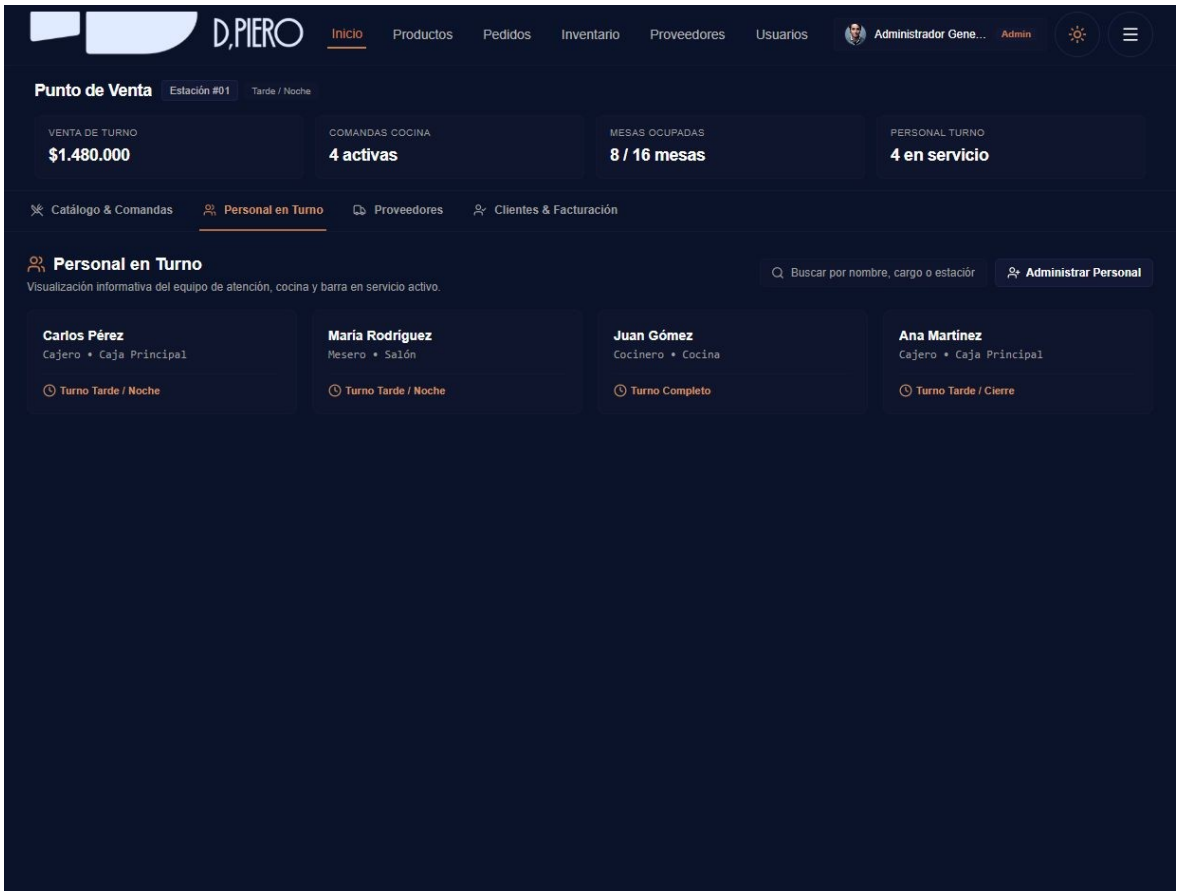


Figura 18 — pestaña Personal en Turno con roles y horarios.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Pos Suppliers Tab

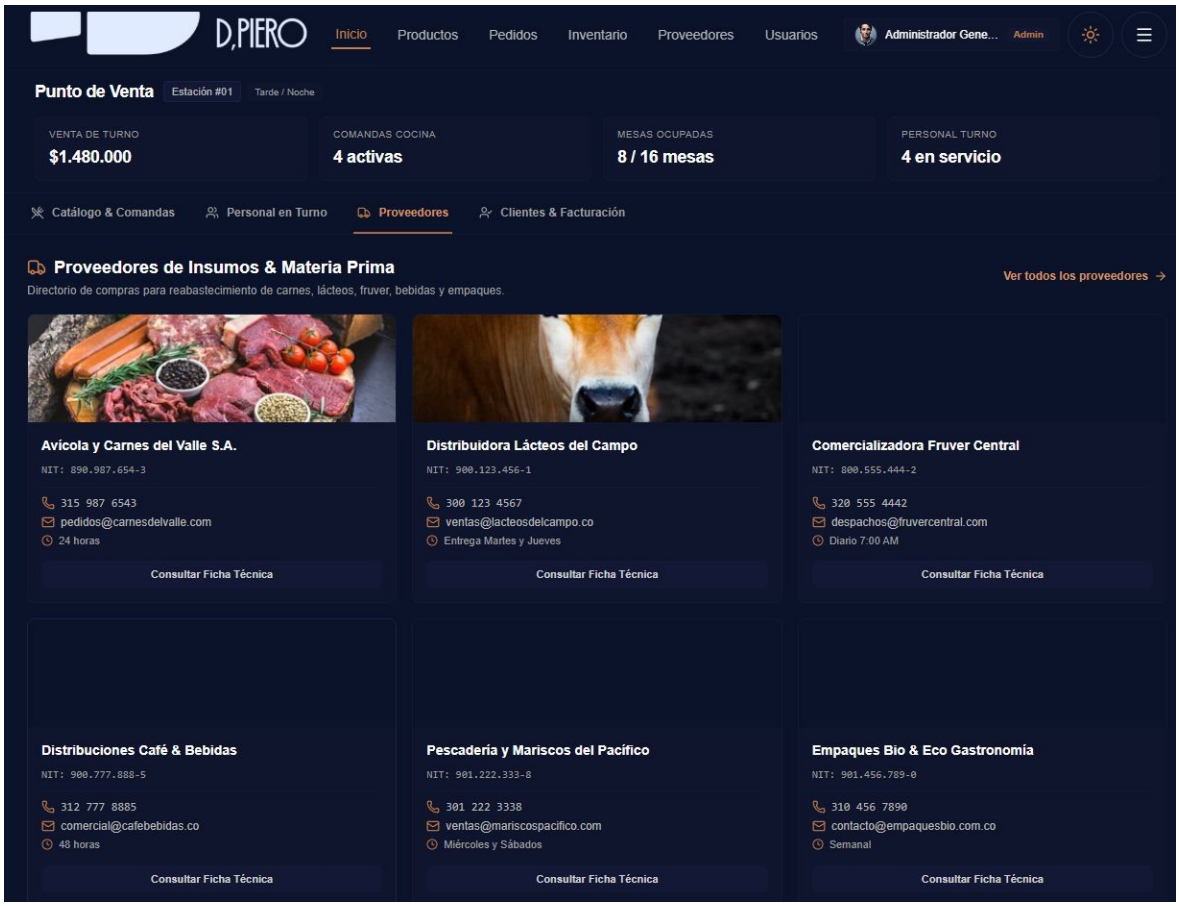


Figura 19 — pestaña Proveedores del POS con datos de contacto reales.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Pos Customers

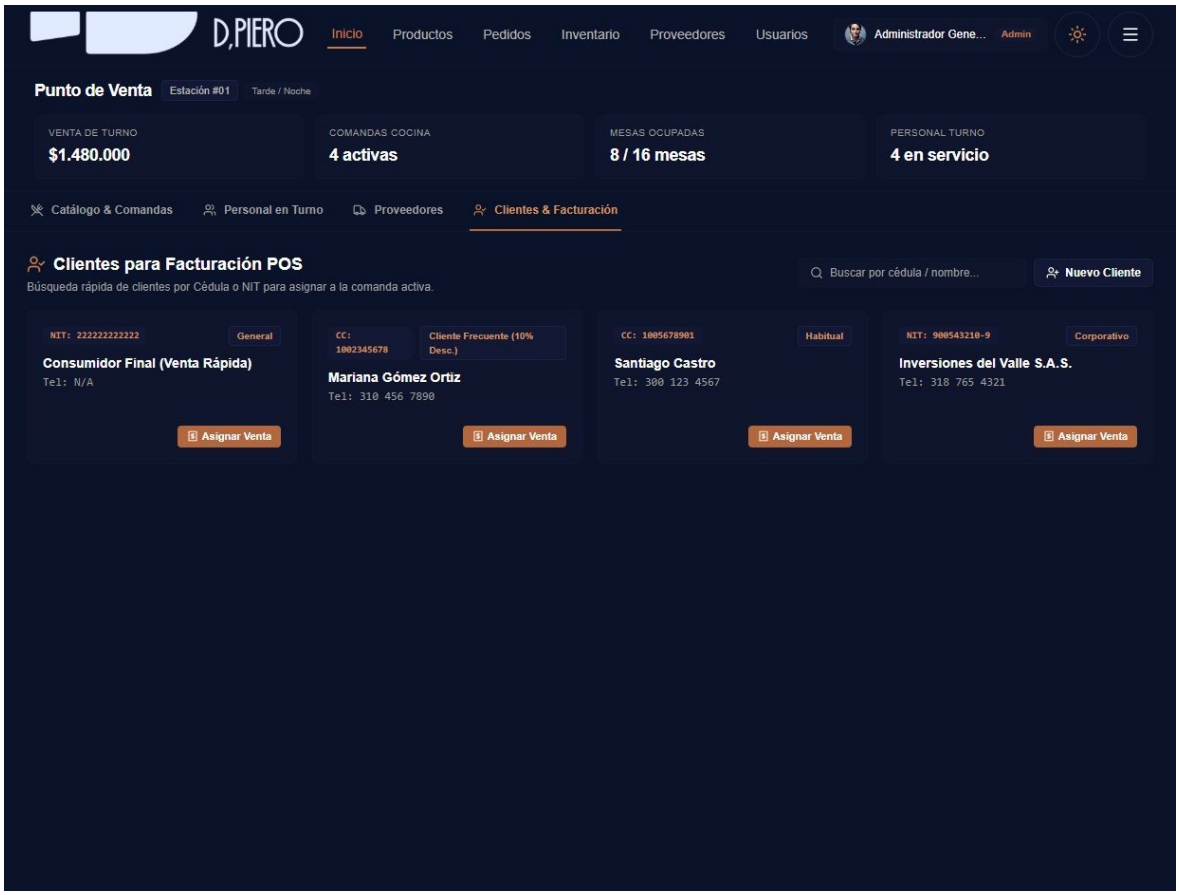


Figura 20 — pestaña Clientes & Facturación (datos de demostración).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Create Order

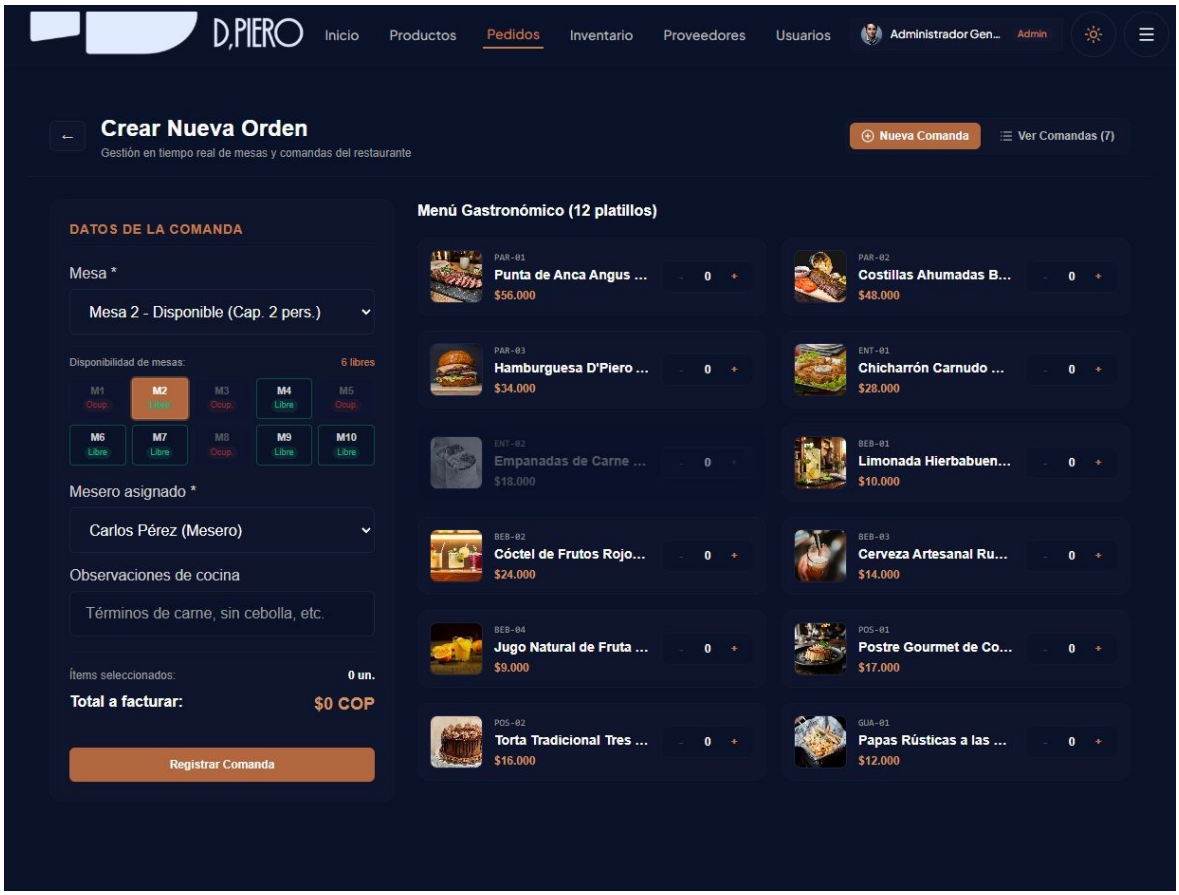


Figura 21 — creación de comanda con mesa y disponibilidad de 6 libres.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Order List

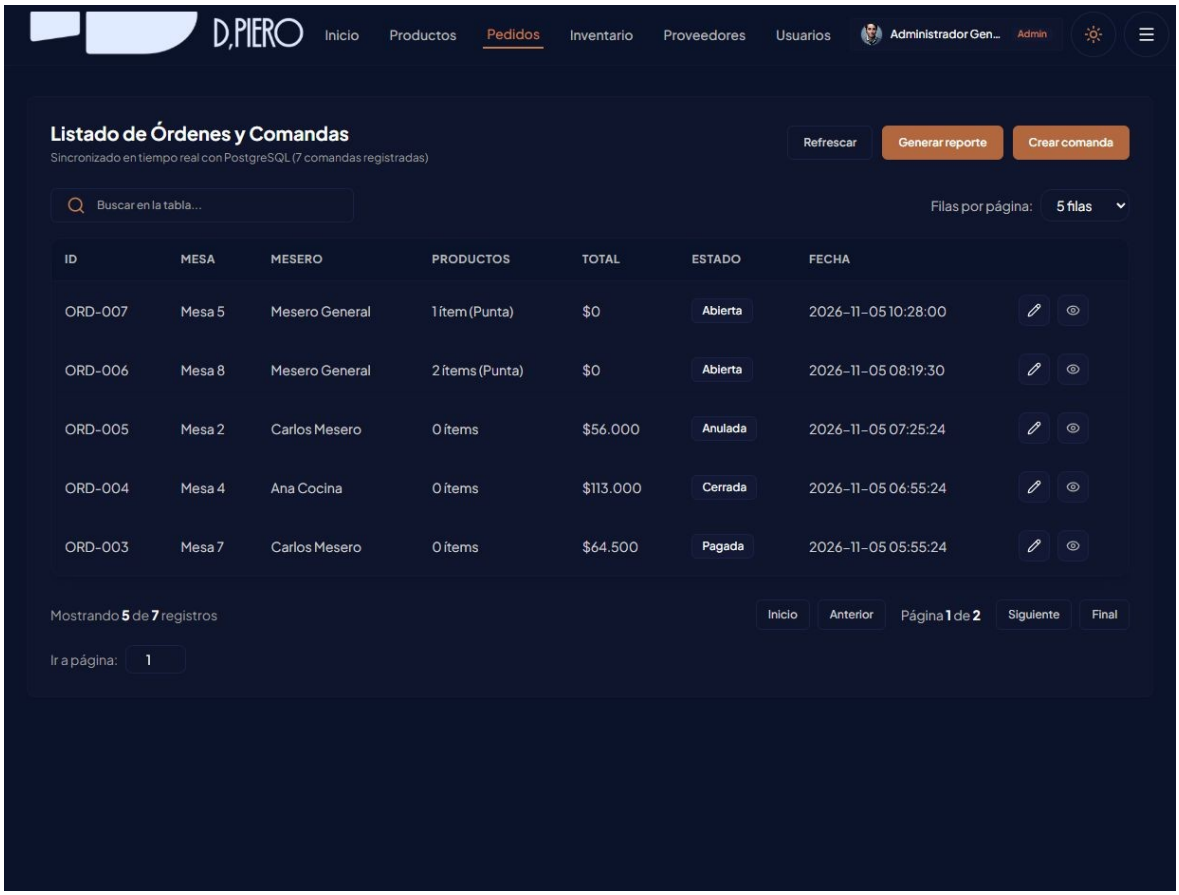


Figura 22 — listado de órdenes ORD-003..007 con estados y totales.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Order View

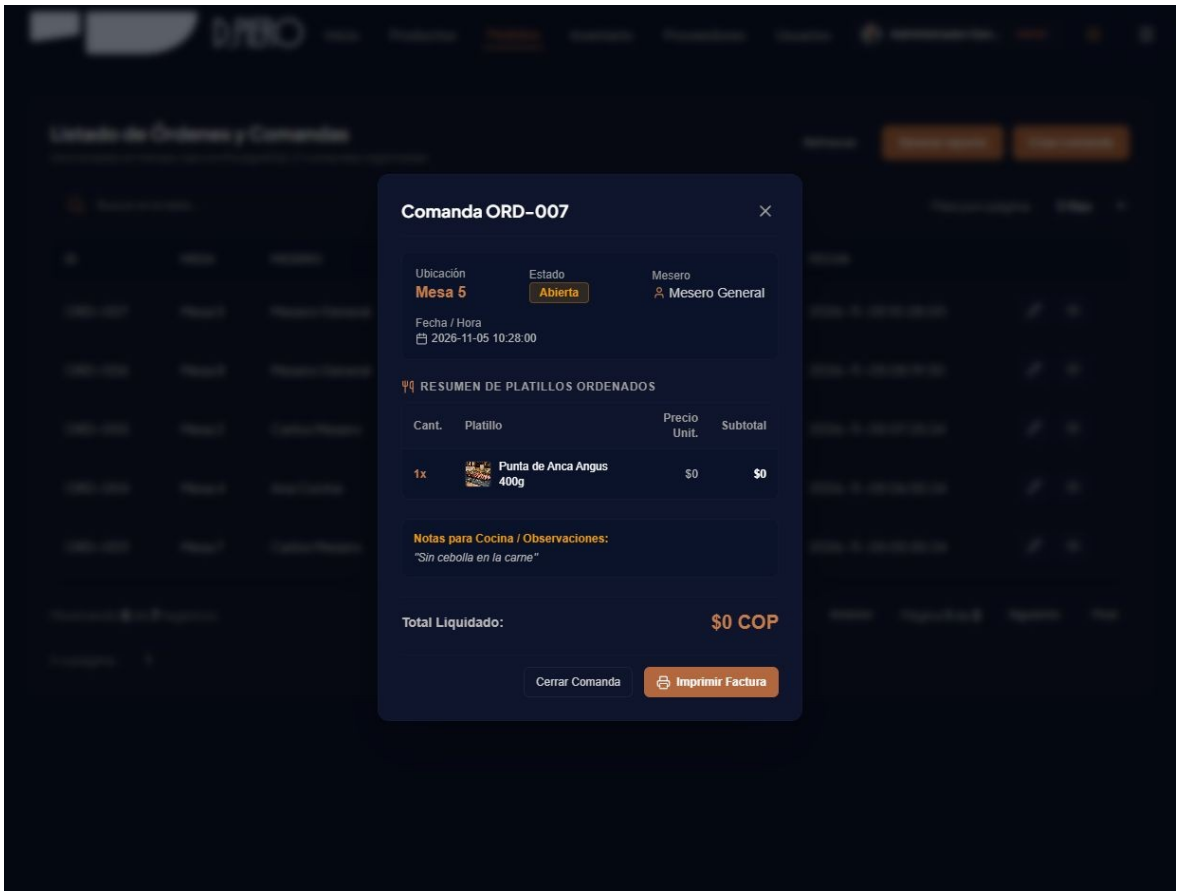


Figura 23 — modal de detalle de comanda con ítems y total.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Edit Order

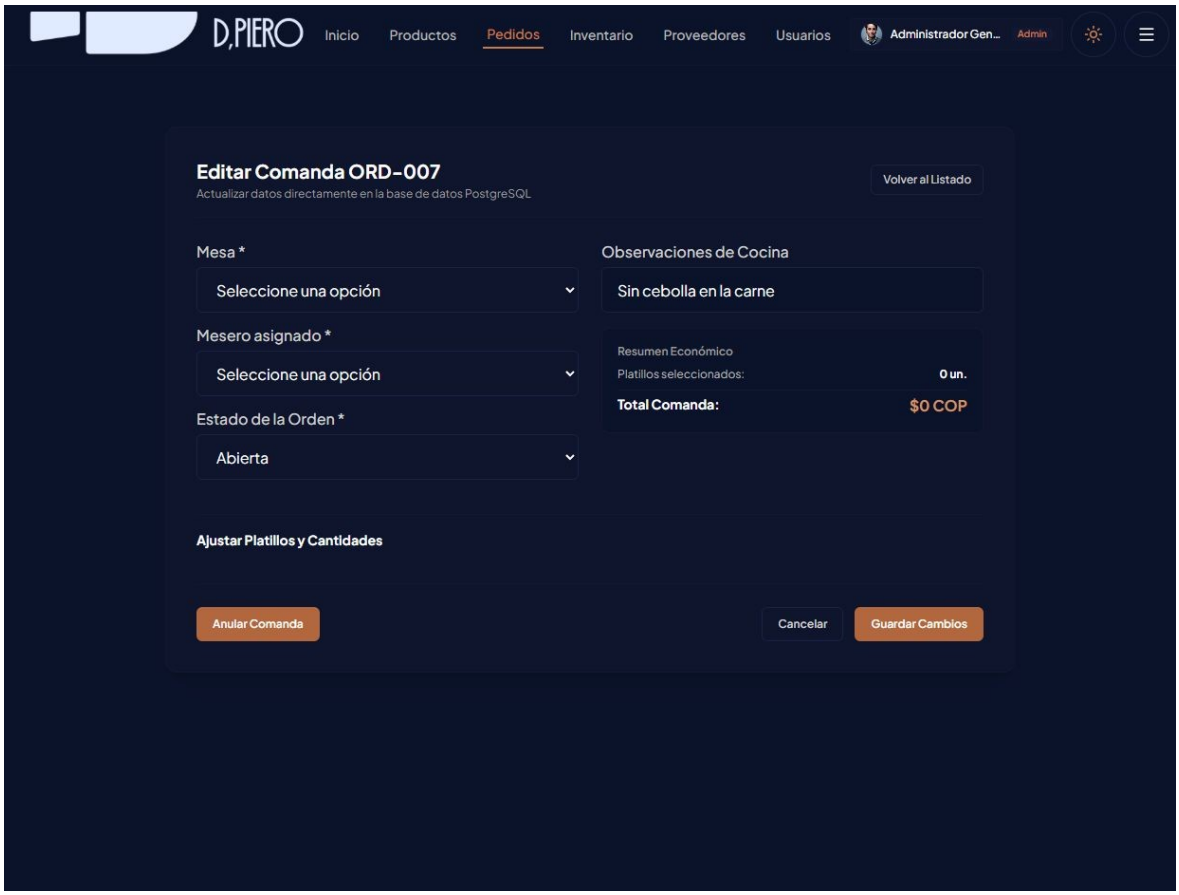


Figura 24 — edición de comanda (mesa, mesero, estado, ítems).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Product List

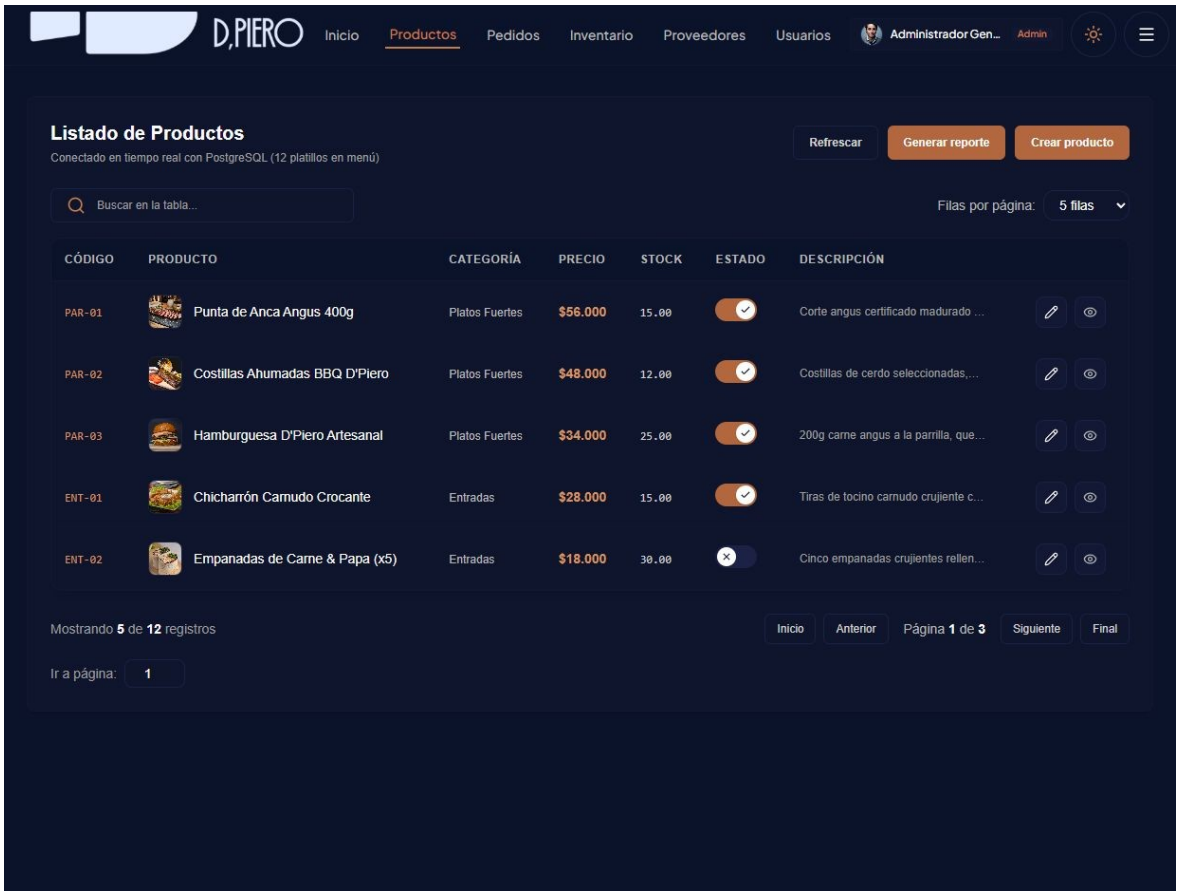


Figura 25 — listado de productos con stock real e INV válidos.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Product Create

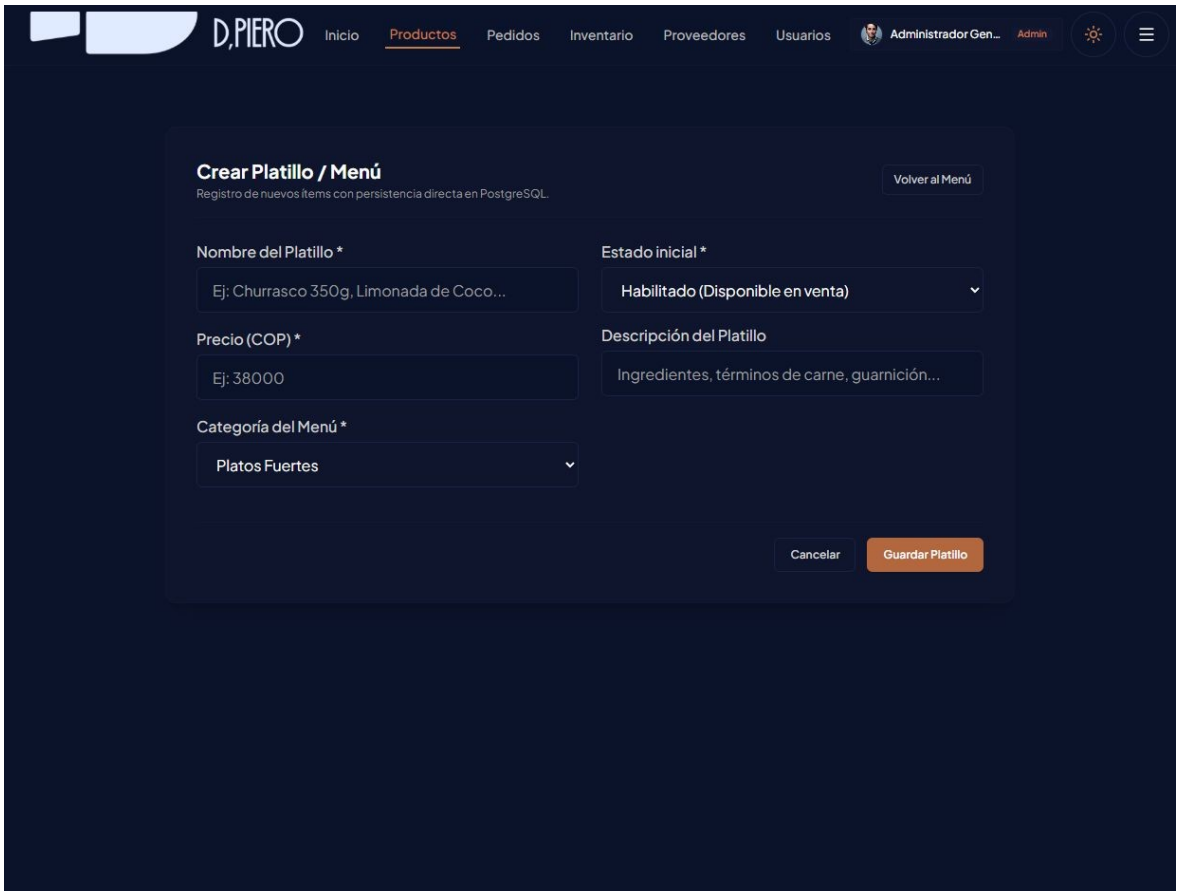


Figura 26 — formulario de creación de platillo.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Product Read

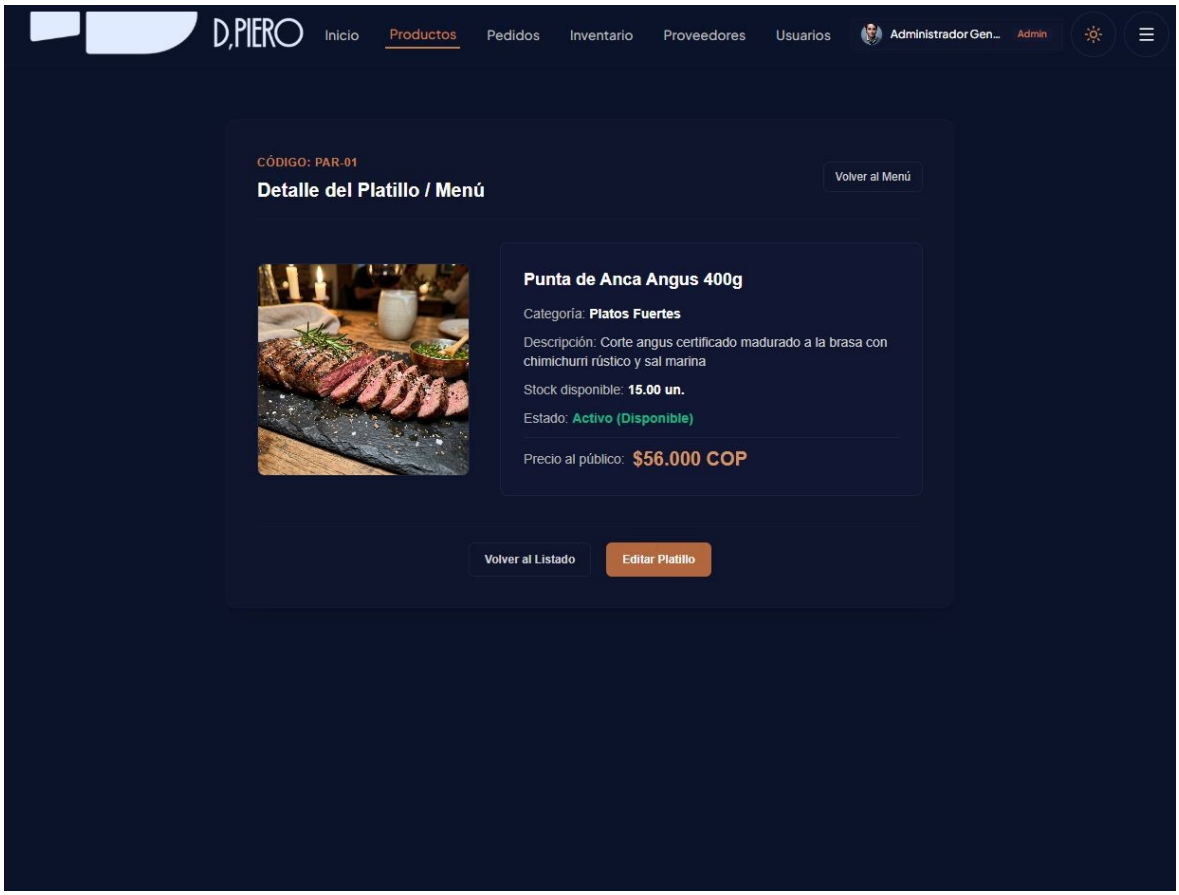


Figura 27 — ficha de solo lectura de platillo (Punta de Anca).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Product Edit

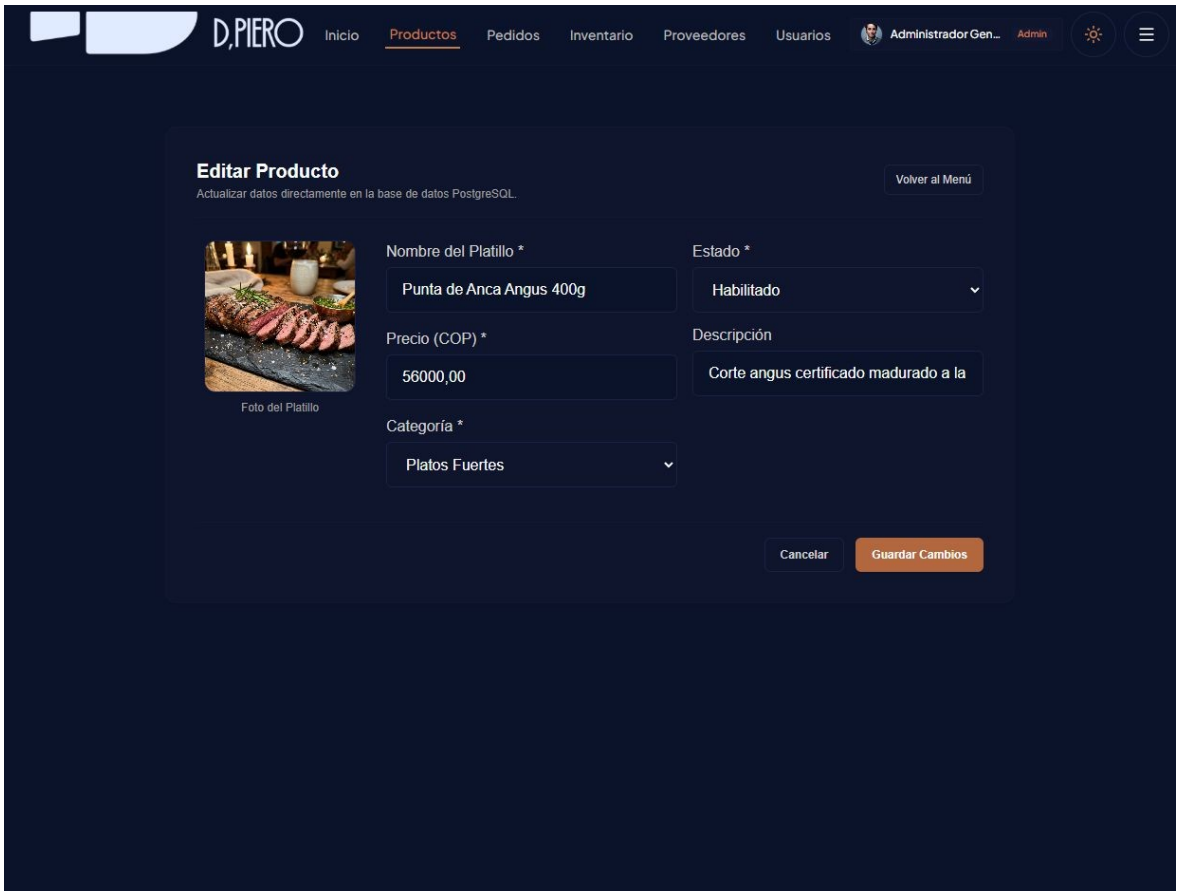


Figura 28 — edición de producto (estado Habilitado).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Inventory List

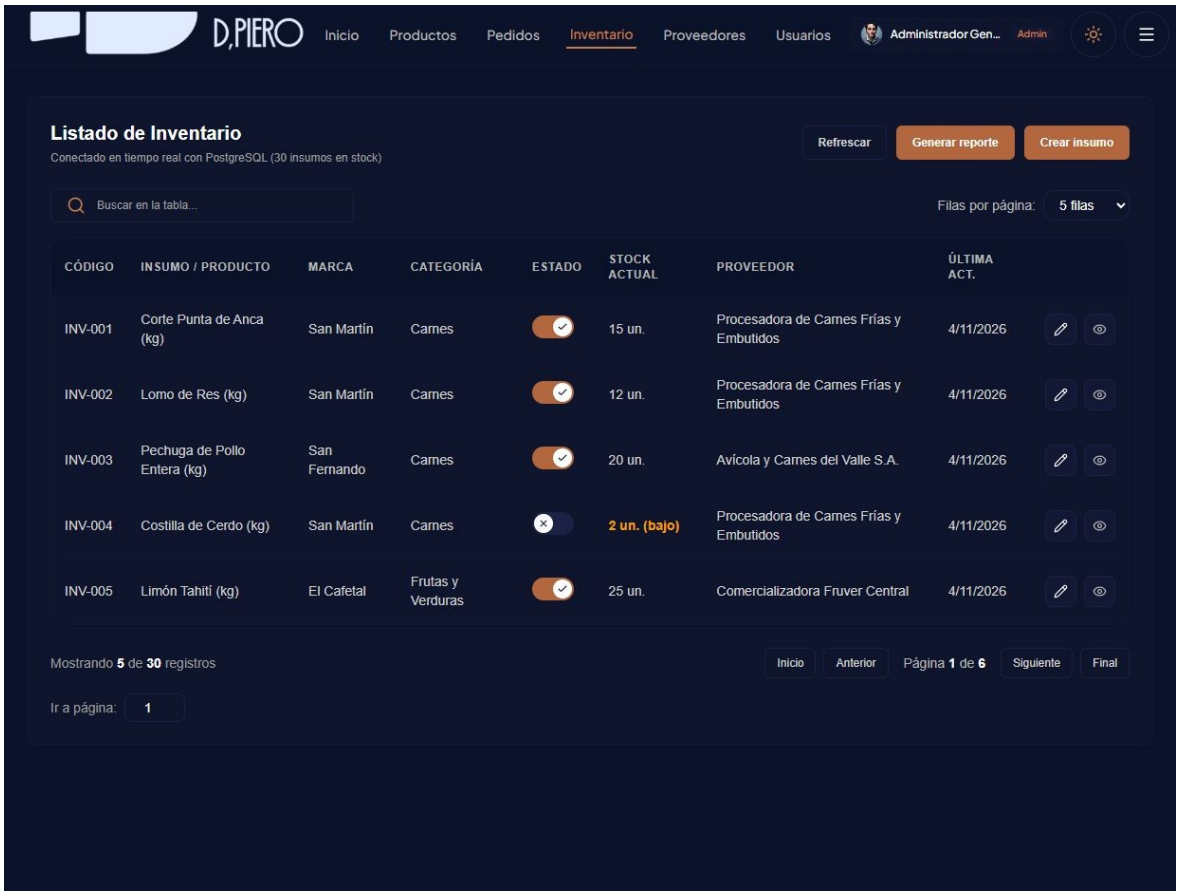


Figura 29 — listado de inventario con alerta de stock mínimo (INV-004).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Create Inventory

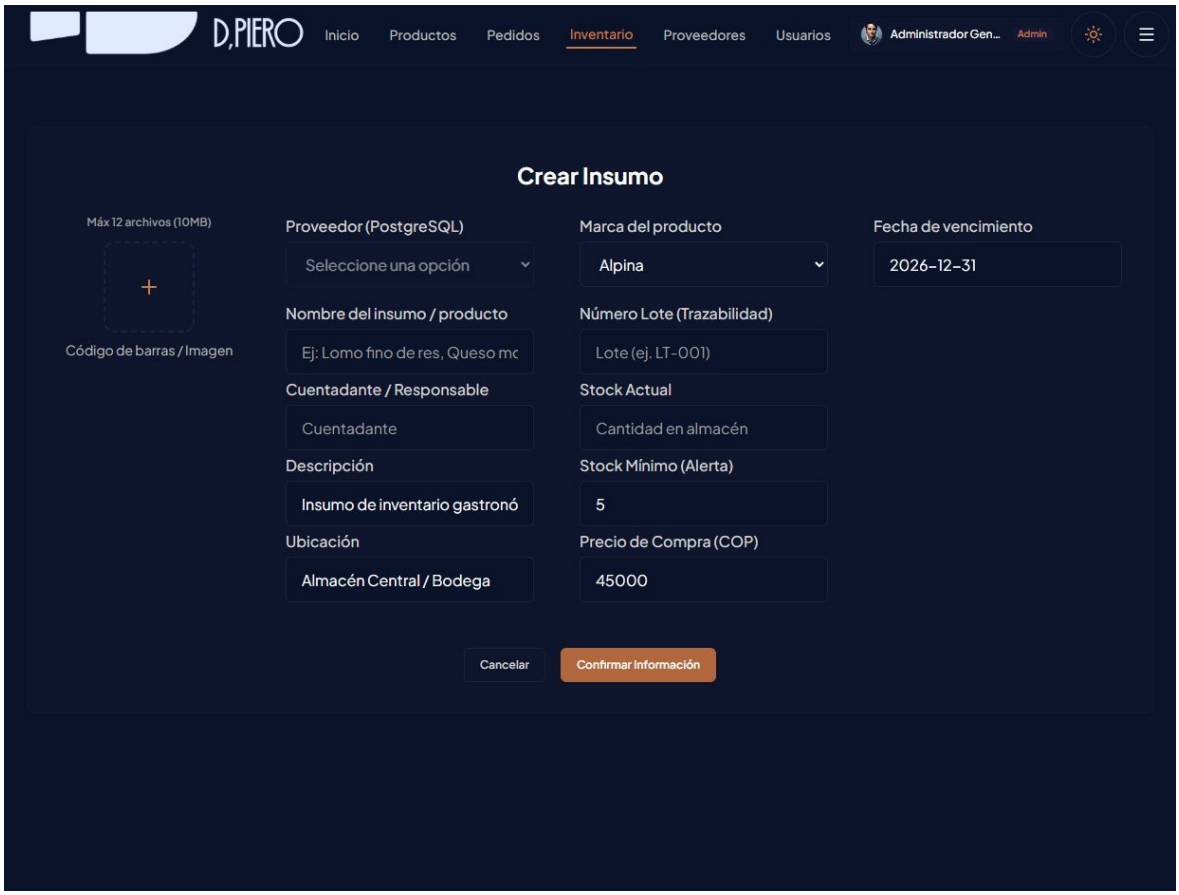


Figura 30 — alta de insumo con lote, vencimiento y proveedor.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Supplier List

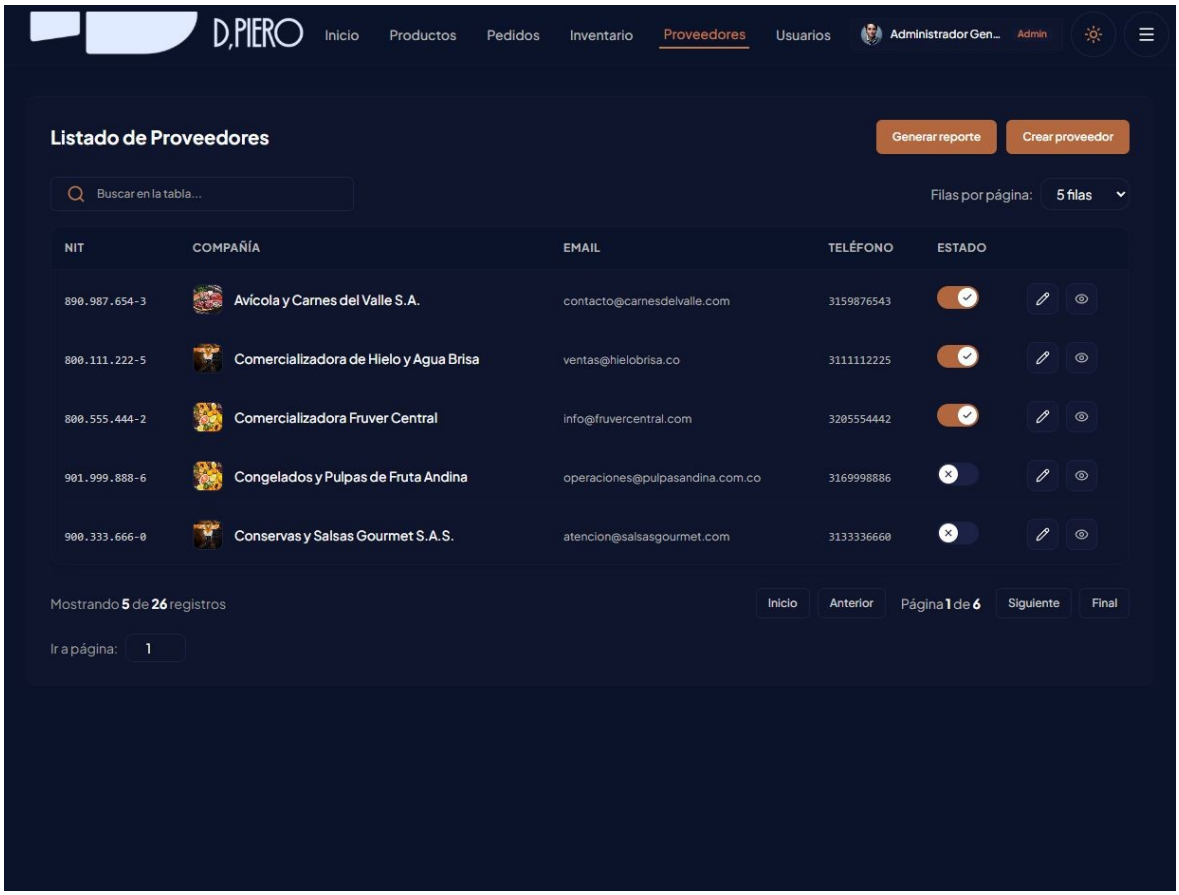


Figura 31 — listado de proveedores con 26 registros.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Supplier Read

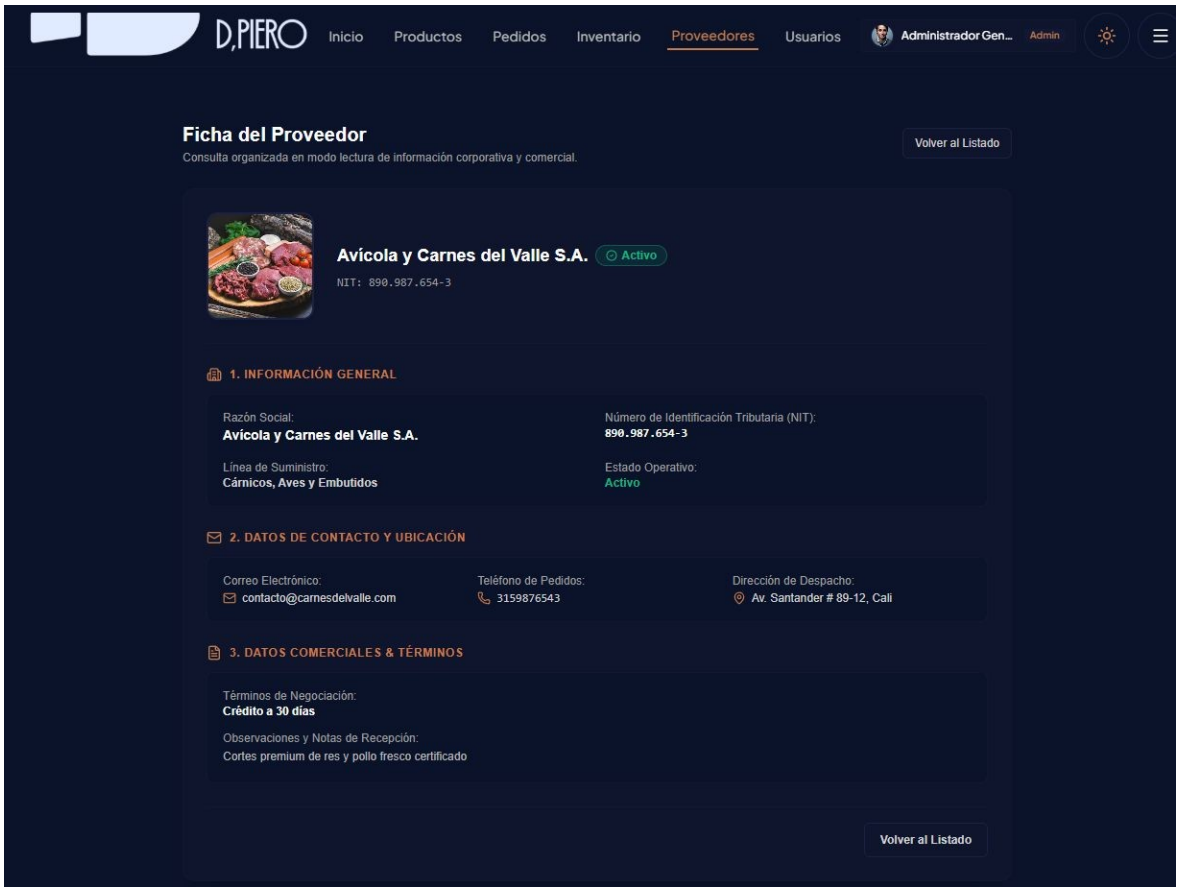


Figura 32 — ficha de proveedor (Avícola y Carnes del Valle S.A.).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Supplier Create

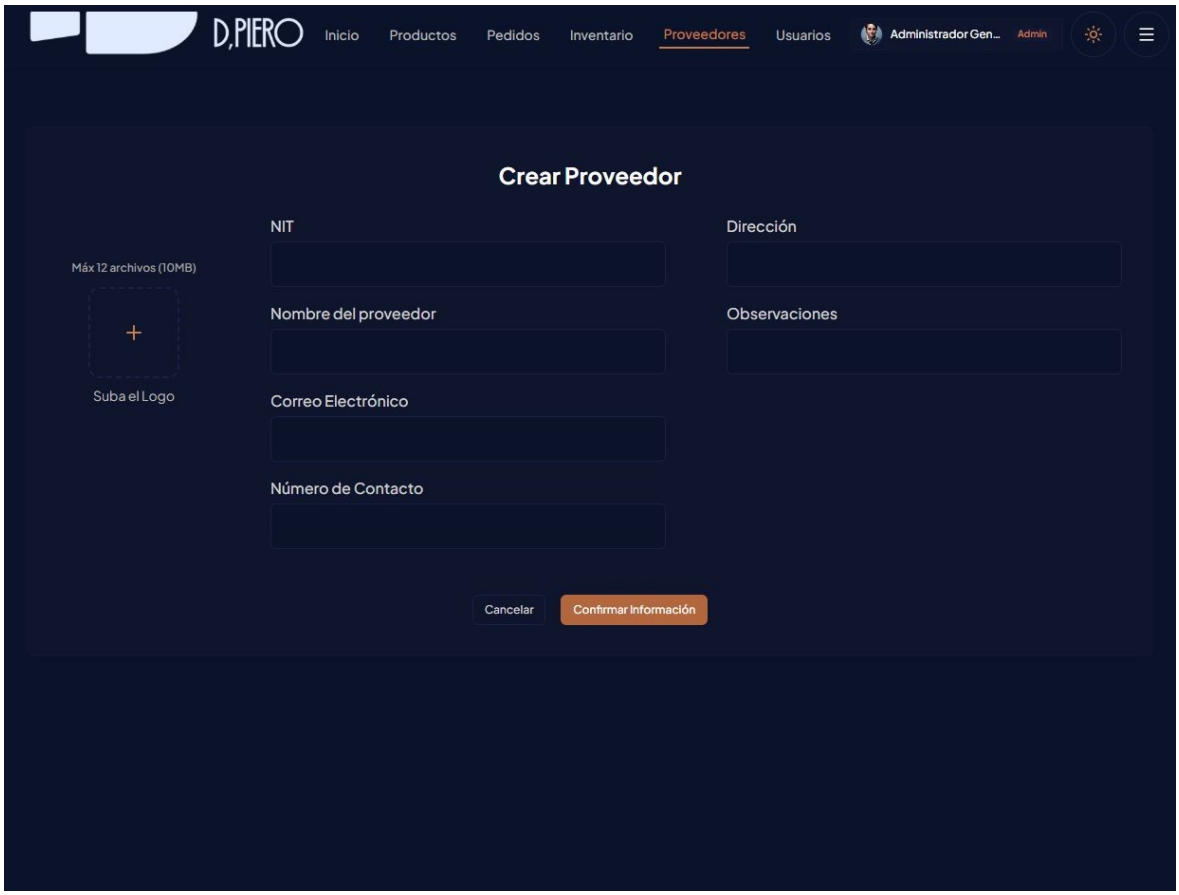


Figura 33 — alta de proveedor con NIT único.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Error Editar Proveedor

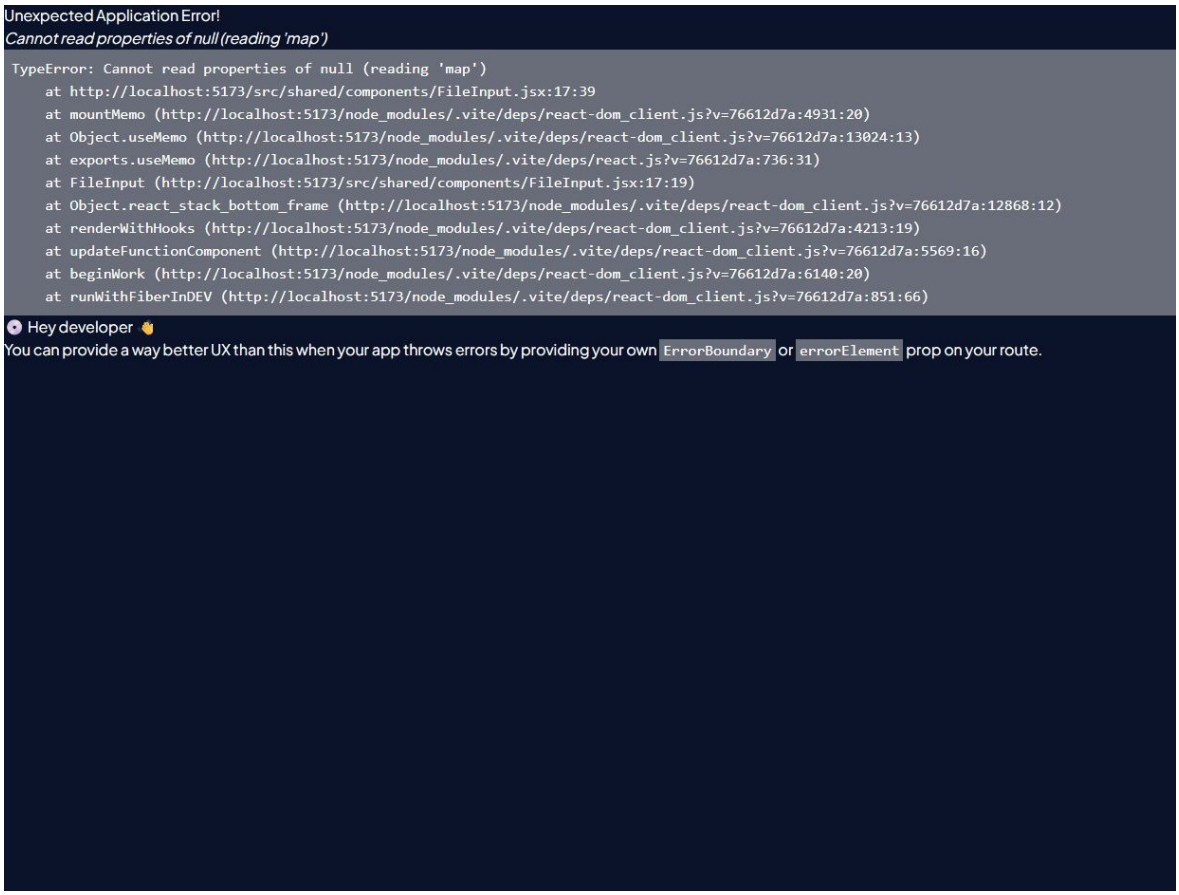


Figura 34 — 4.5: evidencia del fallo BUG-01 en /editsupplier (FileInput userImage=null).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

User List

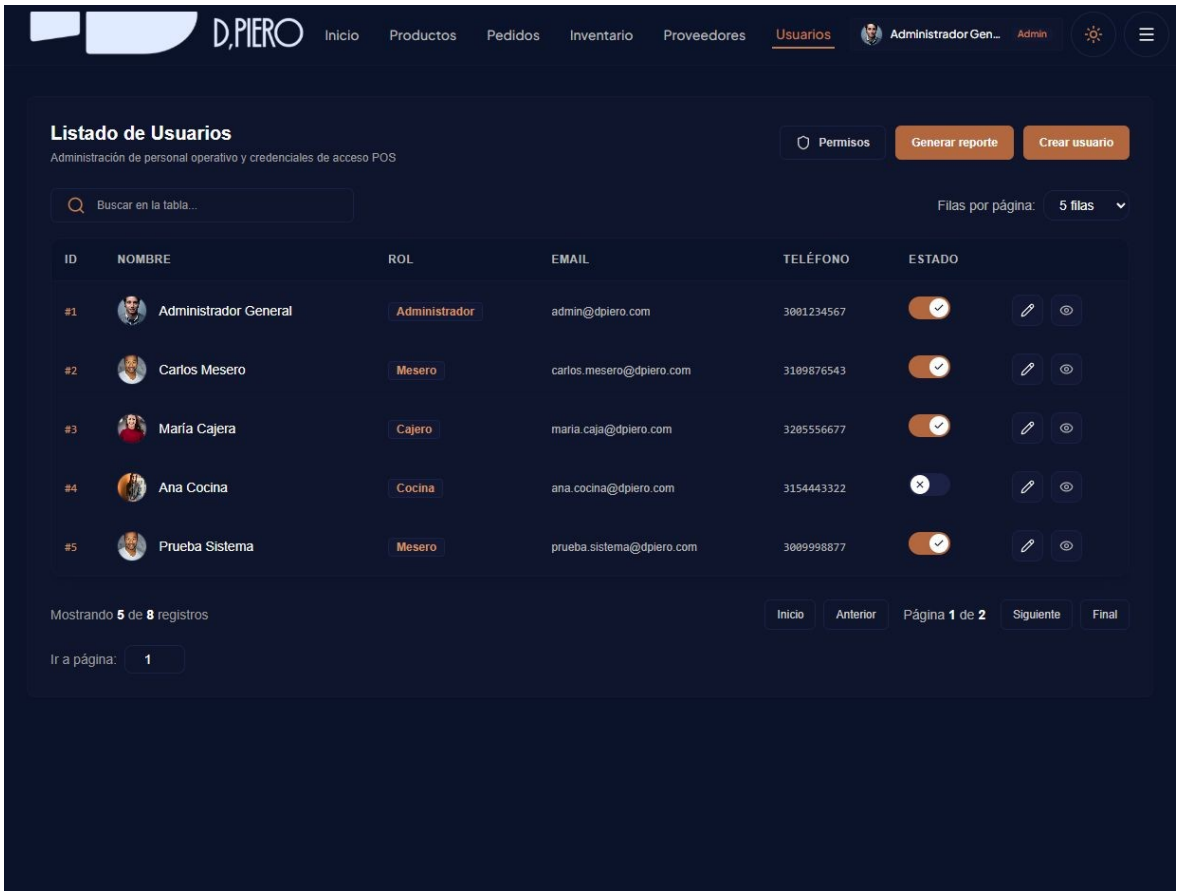


Figura 35 — listado de usuarios con acciones (editar/ver/estado).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

User Create

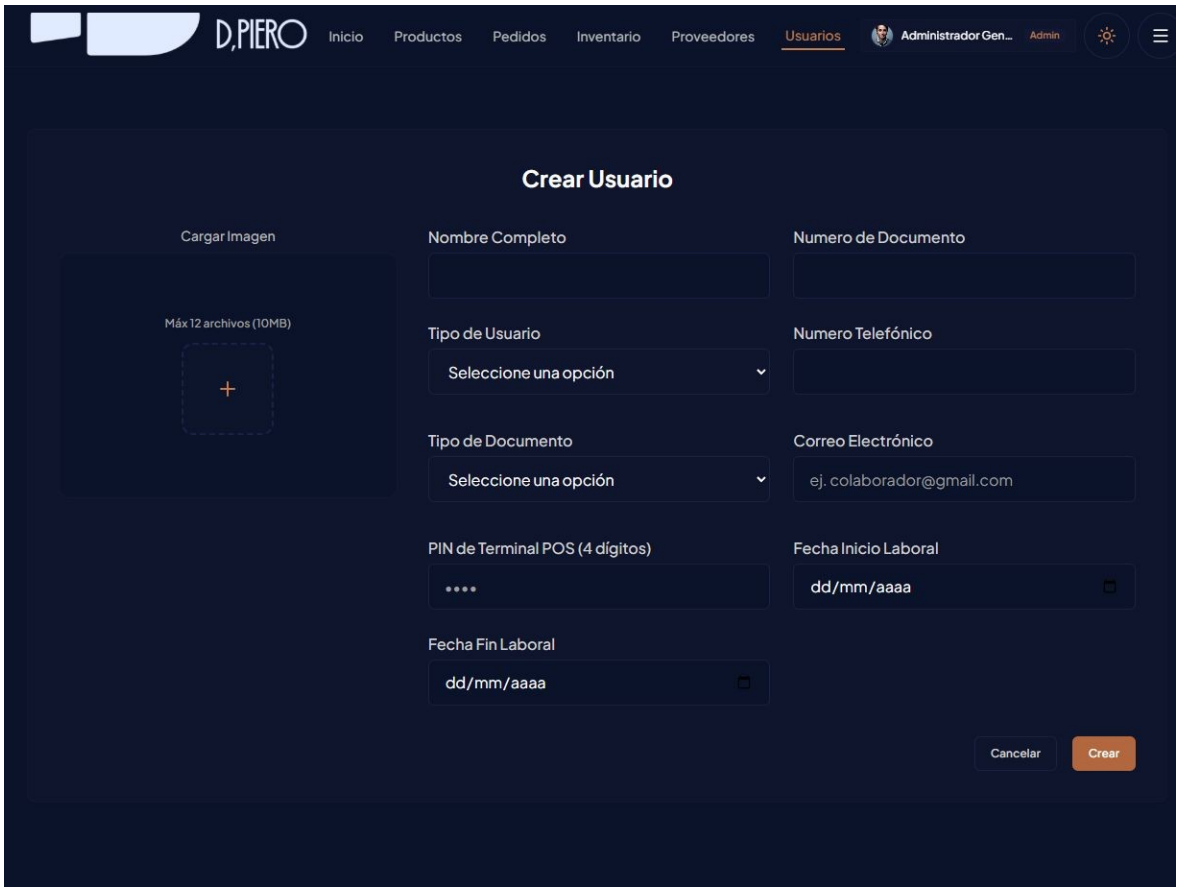


Figura 36 — alta de usuario operativo con PIN autogenerado.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

User Edit

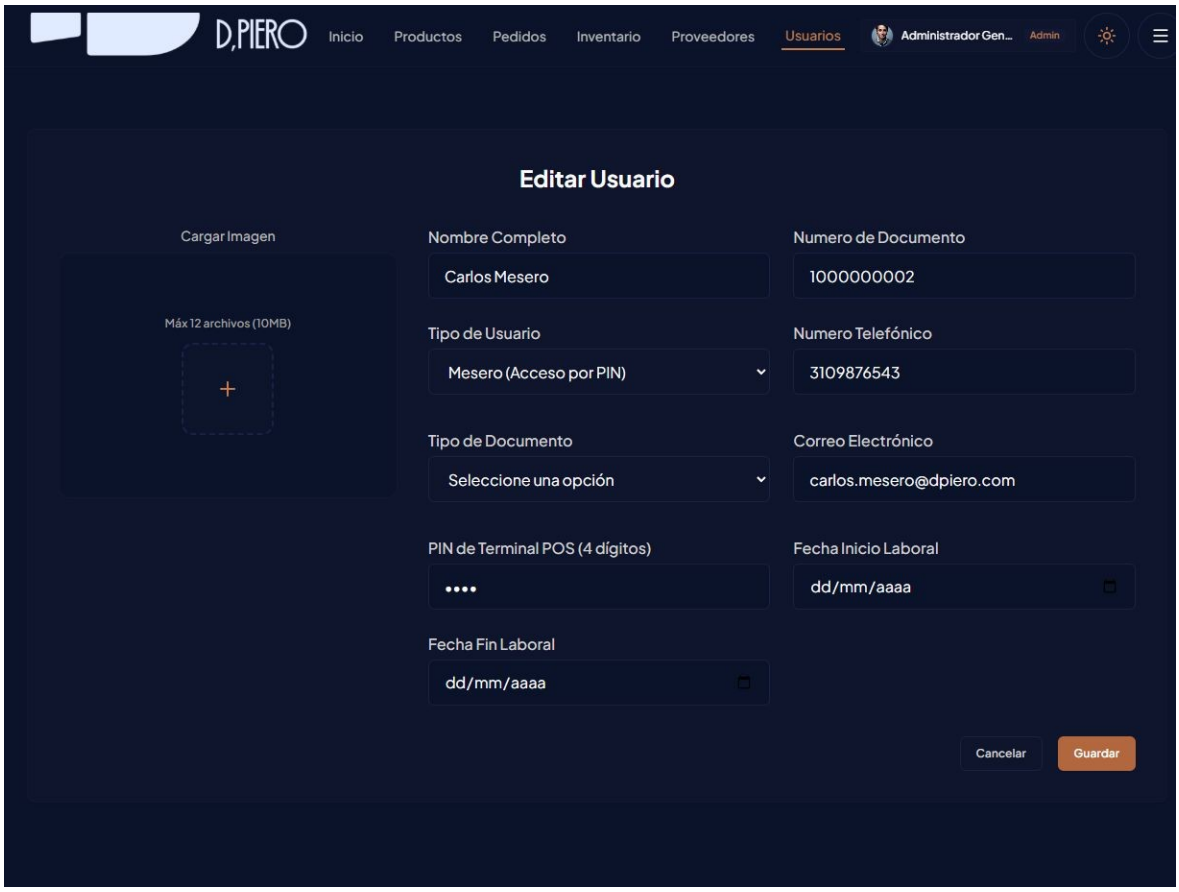


Figura 37 — edición de usuario (Carlos Mesero).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

User Read

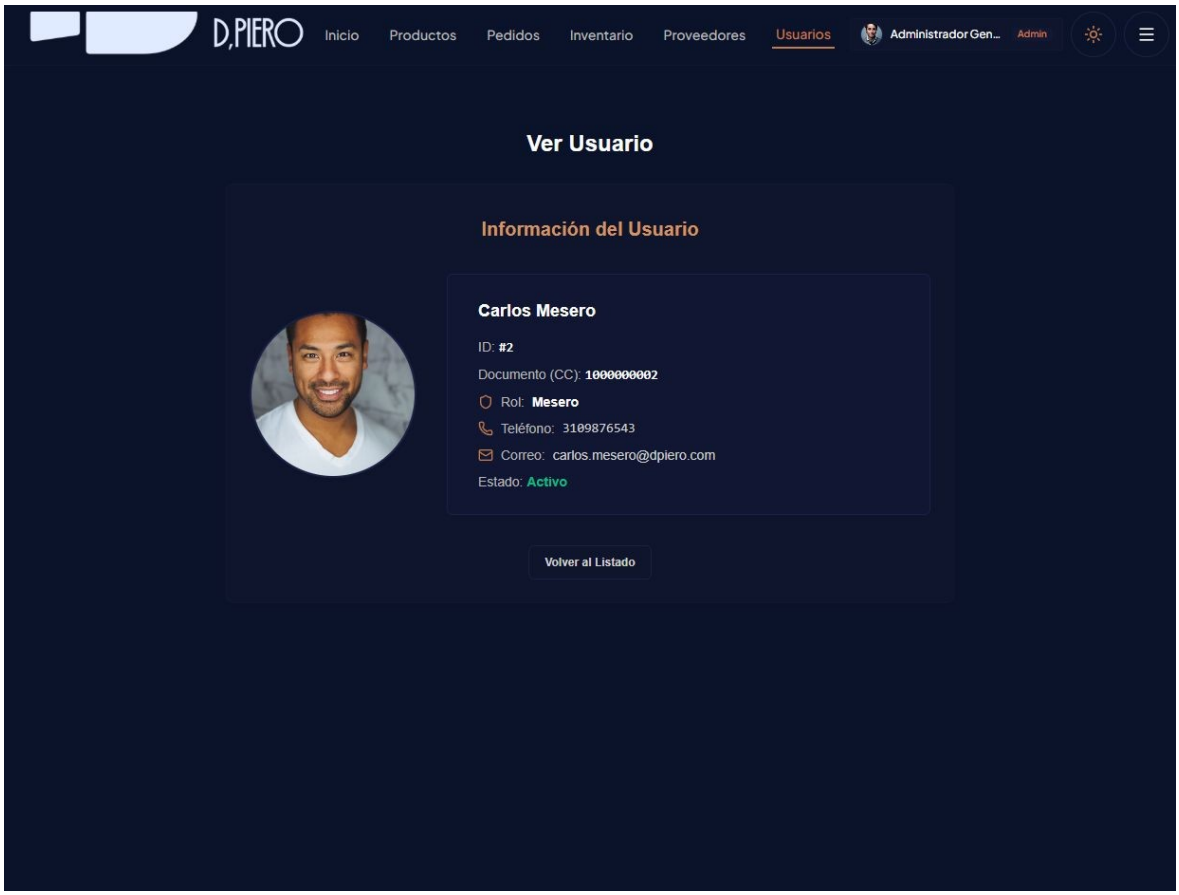


Figura 38 — ficha de usuario (información del usuario).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Users Management

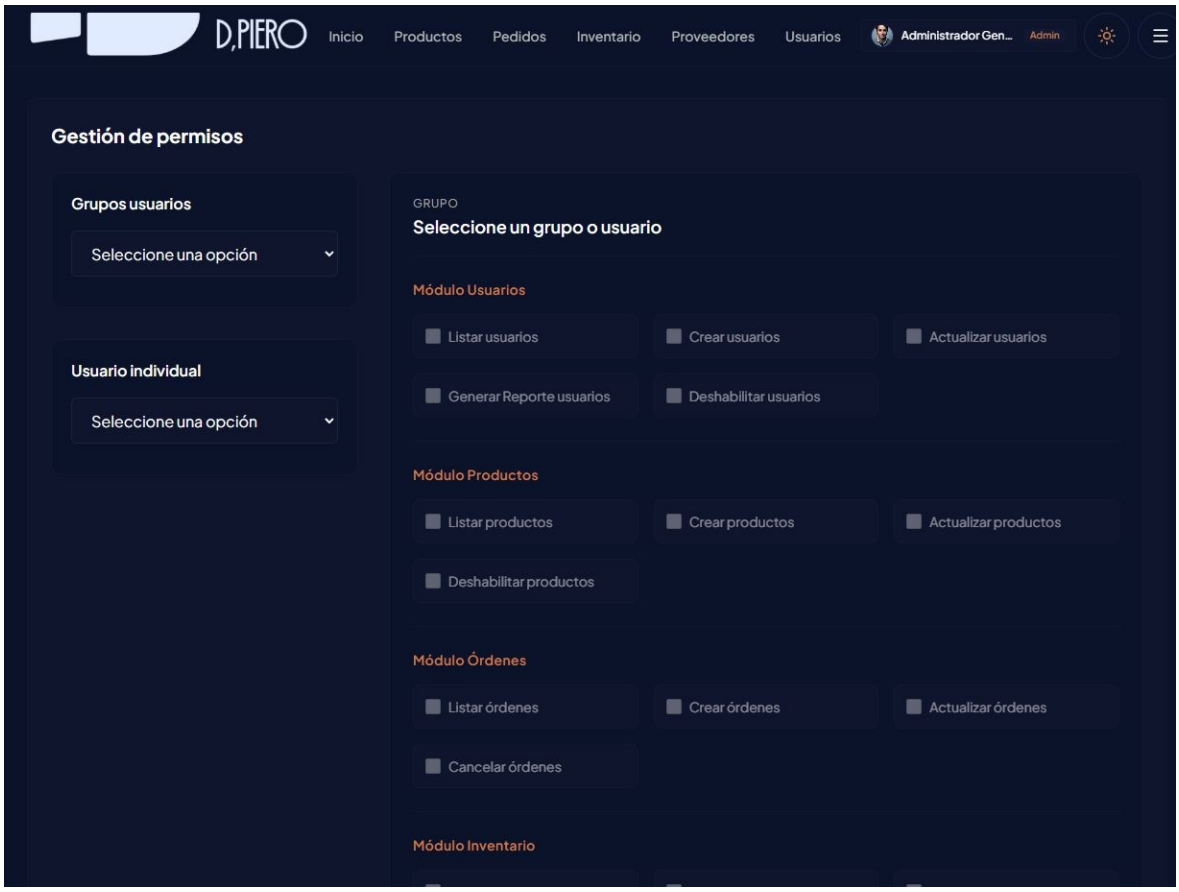


Figura 39 — gestión de permisos RBAC (grupos y usuario individual).

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Access Permissions

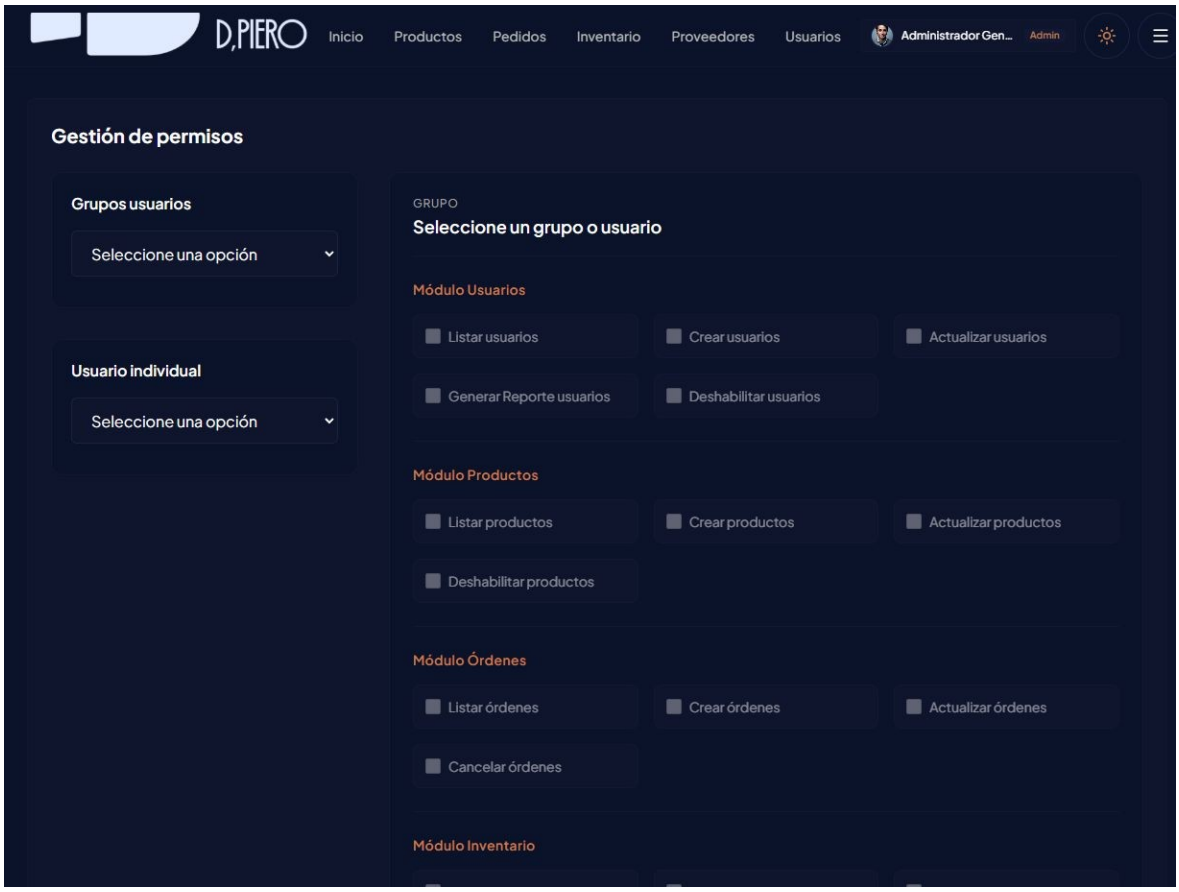


Figura 40 — matriz de permisos por módulo en /dashboard/access.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).

Report Modal

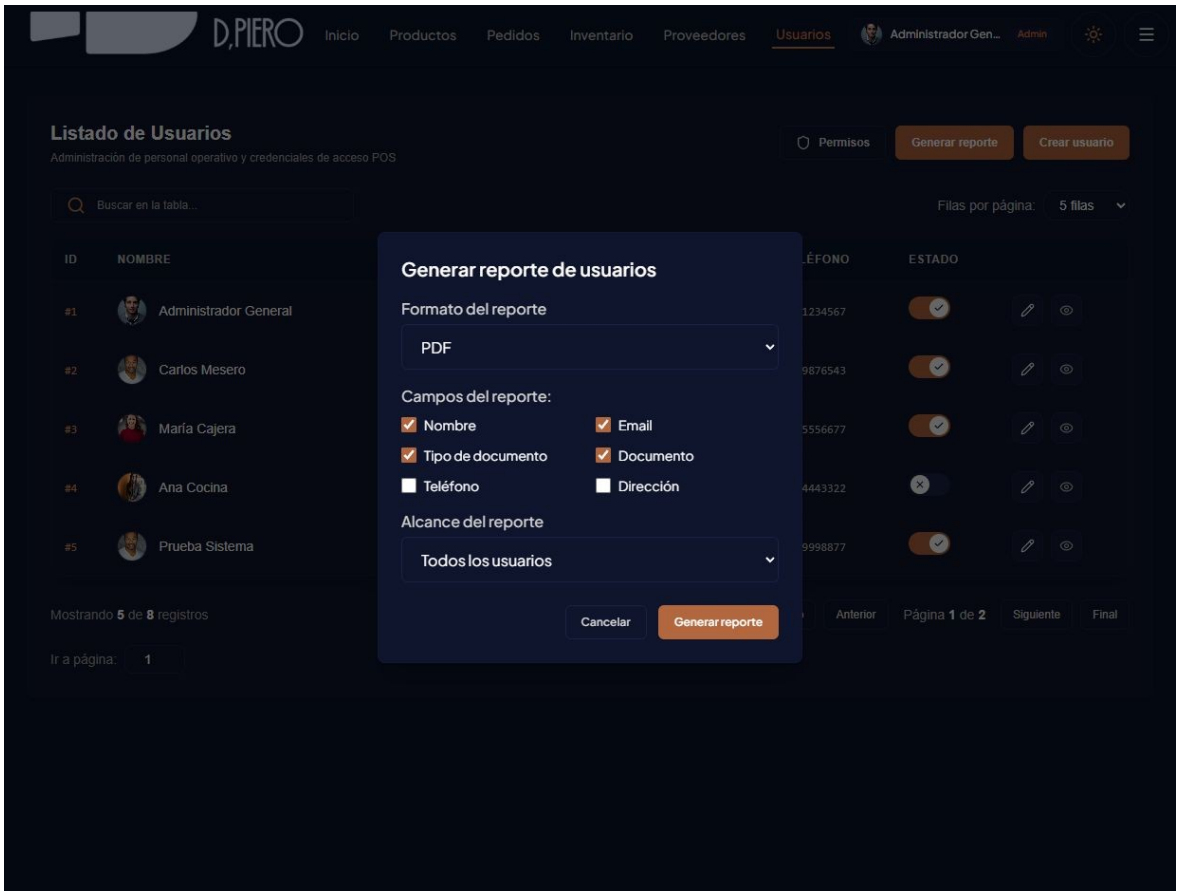


Figura 41 — modal 'Generar reporte de usuarios' con formato PDF por defecto.

Herramienta: navegador Chrome 130 en Windows 11, entorno local con backend en ejecución. Datos reales de la base `pos_project` (migraciones 0001–0027).